Team ID: 26FSDPYCPMID1029

Case-Study Report

On

AI-Driven Privacy Compliance in Heterogeneous IoT Environments

submitted as part of

BCSE318L Data Privacy

by

Registration No.: 23BCE0167
Student Name: Anuska Saha

To



Dr M Rajasekhara Babu

School of Computer Science and Engineering

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

Index

Sl. No.	Section Topic	Page No.
	Abstract	1
	Index Terms	1
1.	Introduction	1
2.	Problem Statement	3
3.	Background and Literature Review	4
4.	Requirements Analysis for AI-Driven Privacy Compliance	7
5.	AI-Driven Privacy Compliance Architecture	8
6.	IoT Data Acquisition and Classification Layer	10
7.	Edge Intelligence and Local Governance Layer	11
8.	AI-Powered Privacy Risk Assessment Layer	13
9.	Automated Compliance and Policy Enforcement Layer	15
10.	Privacy Preservation and Data Protection Layer	16
11.	Explainability, Transparency, and Auditability Layer	18
12.	Governance, Regulatory Mapping, and Data Sovereignty Layer	19
13.	Security and Threat Intelligence Layer	21
14.	Experimental Setup	22
15.	Evaluation Metrics	24
16.	Results and Findings	26
17.	Comparative Analysis	27
18.	Trustworthiness, Accountability, and Governance Assessment	29
19.	Industry Use Cases and Case Study Evaluation	30
20.	Business Impact Assessment	32
21.	Challenges and Limitations	33
22.	Future Research Directions	34
23.	Conclusion	36
	Data Availability	37
	Appendices (A–O)	38
	References	45

List of Figures

Fig. No.	Figure Title	Page No.
1.	AI-Driven Privacy Compliance Architecture	9
2.	End-to-End Privacy Compliance Pipeline	12
3.	Breakdown of an Illustrative Risk Score	13
4.	Risk Score Sensitivity to Uncertainty	14
5.	Conceptual Privacy–Utility Trade-Off	17
6.	Illustrative Response Time by Event Load	25
7.	Risk Scores for Synthetic Example Events	26
8.	Example System Output: Compliance Decision	37

List of Tables

Table No.	Table Title	Page No.
1.	Summary of Proposed Framework Contributions	3
2.	Research Challenges and Gap Analysis	3
3.	Literature Review Comparison	5
4.	Functional and Non-Functional Requirements Matrix	7
5.	IoT Data Classification Matrix	10
6.	Compliance Rules and Actions Matrix	15
7.	Regulatory Mapping Matrix	19
8.	Threat Categories and Security Measures	21
9.	Proposed Experimental Configuration	23
10.	Evaluation Metrics and Definitions	24
11.	Illustrative Test Cases - Not Measured Results	26
12.	Comparative Design Assessment	28
13.	Trustworthiness Assessment Matrix	29
14.	Industry Use Cases Matrix	31
15.	Business Impact Assessment Framework	32

Contributions (Individual Submission)

Sl. No.	Section Topic	Registration No. of Author	Words
	Abstract and Index Terms	23BCE0167	224
1.	Introduction	23BCE0167	1,587
2.	Problem Statement	23BCE0167	1,473
3.	Background and Literature Review	23BCE0167	2,225
4.	Requirements Analysis for AI-Driven Privacy Compliance	23BCE0167	1,438
5.	AI-Driven Privacy Compliance Architecture	23BCE0167	1,469
6.	IoT Data Acquisition and Classification Layer	23BCE0167	1,430
7.	Edge Intelligence and Local Governance Layer	23BCE0167	1,412
8.	AI-Powered Privacy Risk Assessment Layer	23BCE0167	1,742
9.	Automated Compliance and Policy Enforcement Layer	23BCE0167	1,397
10.	Privacy Preservation and Data Protection Layer	23BCE0167	1,654
11.	Explainability, Transparency, and Auditability Layer	23BCE0167	1,362
12.	Governance, Regulatory Mapping, and Data Sovereignty Layer	23BCE0167	1,332
13.	Security and Threat Intelligence Layer	23BCE0167	1,303
14.	Experimental Setup	23BCE0167	1,337
15.	Evaluation Metrics	23BCE0167	1,581
16.	Results and Findings	23BCE0167	1,396
17.	Comparative Analysis	23BCE0167	1,239
18.	Trustworthiness, Accountability, and Governance Assessment	23BCE0167	1,158
19.	Industry Use Cases and Case Study Evaluation	23BCE0167	1,592
20.	Business Impact Assessment	23BCE0167	1,189
21.	Challenges and Limitations	23BCE0167	1,326
22.	Future Research Directions	23BCE0167	1,304
23.	Conclusion	23BCE0167	1,125
	Data Availability	23BCE0167	93
	References	23BCE0167	1,644
	Appendices (A-O)	23BCE0167	6,608

Sl. No.	Registration No. of Student	Total Words Contributed
1	23BCE0167	41,640
	Total number of words (sole author)	41,640

AI-Driven Privacy Compliance in Heterogeneous IoT Environments

Anuska Saha
School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India
BCSE318L - Data Privacy | Registration No.: 23BCE0167 | Team ID: 26FSDPYCPMID1029

ABSTRACT

Heterogeneous Internet of Things (IoT) systems collect data through devices that differ in format, capacity, ownership, and purpose. These differences make privacy compliance difficult because a data record can pass through multiple gateways, applications, and cloud services. This case study proposes an AI-supported framework that joins device authentication, data classification, edge privacy controls, risk scoring, rule-based enforcement, explainable decisions, and audit records. The framework uses mandatory rules for known requirements and uses AI to identify unusual or uncertain events for further review. It defines a transparent risk score, a sequence of system actions, a set of privacy controls, and an evaluation protocol. It also discusses differential privacy, federated learning, encryption, data minimisation, explainable AI, threat monitoring, and regulatory mapping. The proposed design is intended to reduce unnecessary data movement and make each decision easier to inspect. The mathematical graphs use formula-based or synthetic examples and are labelled as illustrative. No real deployment or experimental result is claimed. Future work should implement the design, compare it with documented baselines, test it with properly labelled data, and publish the actual measurements and configuration. The paper is written as a detailed case-study design for privacy-aware governance in healthcare, smart cities, industry, energy, transport, logistics, and other IoT settings.

INDEX TERMS

AI-driven compliance, data protection, edge intelligence, Internet of Things, privacy governance, regulatory mapping, trustworthiness, IoT security.

1. Introduction

1.1 Background and context

The Internet of Things connects physical objects to computing systems. A sensor can measure temperature, a meter can record energy use, a wearable can collect health signals, and a camera can observe activity in a public or private space. A connected service may combine these readings to make a decision or to offer a useful service. This can save time and improve safety, but it also means data may be collected without a person seeing every step. Privacy therefore depends on the full data path, not only on the first device [1][3][4].

An IoT system is rarely one single product. It may include a device made by one vendor, a gateway made by another, a cloud platform, an analytics service, and an organisation that decides why the data are used. Each part may have a different software version and security policy. Some parts can store large records, whereas others have very little memory or battery power. A useful privacy framework must support this variety without assuming that every device can run the same model or encryption operation [6][14][15][16].

1.2 Why privacy compliance is difficult

Privacy compliance asks more than whether data are encrypted. It asks whether the data are needed, whether the stated purpose is valid, whether the receiver is approved, whether the data are kept for the correct period, and whether people can exercise their rights where the law gives them those rights. A system may use strong encryption and still misuse data if an application reads a record for an unrelated purpose. A system may also collect too much information even if no attacker ever sees it. Privacy protection must therefore include both technical controls and clear governance rules [2][12][31].

The problem is harder when records from different devices are linked. A location point may not identify a person by itself, but a long sequence can reveal a routine. An energy meter may show when a house is occupied. A health sensor can reveal information about a person's body. When several sources are joined, data that looked harmless can become sensitive. Data labels must consider the meaning of the record and the context in which it is used, rather than only the name of the field [3][4][21].

1.3 Role of AI and edge computing

Artificial intelligence (AI) can look for patterns that are hard to describe as a short list of fixed rules. For example, a device that normally sends a few readings may suddenly send thousands of records to a new receiver. An anomaly detector can flag this change. A classifier may help label incoming records or rank a queue of events for review. However, AI can also make mistakes, and an anomaly is not always a privacy violation. The system must check the event against approved policy rules before taking a high-impact action [1][3][20][30].

Edge computing places some processing near the devices that produce the data. This can reduce delay and limit how much raw data moves to a central cloud. The edge can validate fields, remove unnecessary identifiers, and apply local rules before a transfer takes place. Still, an edge gateway must be protected, updated, and managed. Local processing is a way to reduce risk, not a guarantee of privacy. The design in this report combines edge rules with AI risk support and a central audit view [6][14][15][16].

1.4 Aim, questions, and contributions

The aim of this case study is to propose a clear and testable way to check privacy compliance across different IoT devices. The first research question asks how a system can retain context about data purpose, sensitivity, receiver, and retention. The second asks how AI can support risk review without replacing mandatory rules. The third asks what evidence is needed to explain an action. The fourth asks how the design should be tested so that claims about performance, privacy, and cost can be checked later.

The report contributes an integrated architecture, an end-to-end pipeline, a transparent risk equation, data and policy tables, a literature comparison, a regulatory mapping approach, and a repeatable evaluation plan. It also explains the limits of the proposed approach. This is a design-focused case study. It does not claim that a working product has been built or that a benchmark has already been completed. A useful research contribution can define the system and the test method clearly before experiments begin. The next stage must measure the design using real implementation evidence.

1.5 Why compliance must cover the full data life cycle

Privacy risk changes as data move through a system. At collection, the main questions are whether the device is allowed to collect the field and whether the field is needed. During transfer, identity, access, and destination matter. During use, the purpose must still be valid. During storage, access and retention rules become important. At the end of the life cycle, copies in caches, backups, exports, and derived files must be handled under the approved retention plan. A single check at the network boundary cannot cover all these steps [2][4][12].

The proposed framework therefore treats each data use as an event that can be checked again when its context changes. A record that was allowed for one purpose is not automatically allowed for a second purpose. A receiver that was approved last month may have been removed from the approved list. A device may become unsupported or be revoked. The system must use current context and a versioned policy, while keeping enough historical evidence to explain previous decisions. This makes compliance a continuing process rather than a one-time box to tick.

1.6 What makes this study different from a product claim

This report presents a design and an evaluation plan, not a completed commercial product. The diagrams describe components that could be built; the equations describe choices that could be tested; and the graphs show how the proposed calculations behave under stated assumptions. This distinction is important because a well-drawn architecture does not prove that the system is secure, fast, or legally compliant. Those claims need implementation evidence, test cases, measured outcomes, and review by the responsible people.

The value of the present study is that it connects research ideas into one reviewable workflow. It describes where each control runs, what information it needs, what action it may take, what evidence it must keep, and what should happen if information is missing. The document also identifies the tests that could show whether the design works. In a later research stage, the design can become software and the illustrative graphs can be replaced by data from an actual testbed.

1.7 A lifecycle view of IoT privacy

Privacy has to be considered across the full life of a data item. The item is first created by a sensor or device. It may then pass through a gateway, a local service, a database, an analytics tool, and a third-party receiver. A control at only one point cannot guarantee that later use stays within the original purpose. For example, a location reading may be collected for traffic planning and later copied into a person-tracking system. The same value has moved through two different uses, so the privacy check must include the purpose and receiver, not only the security of the transfer. This lifecycle view is consistent with privacy-by-design guidance and research that maps privacy controls to different stages of IoT processing [2][12][42][45].

A practical way to apply this view is to assign an identifier to each event and connect it to a purpose, receiver, retention rule, and policy version. The identifier should not itself reveal a person's identity. The gateway can carry the metadata alongside the data or store a protected reference to them. A downstream service should verify that the metadata are still valid before using the event. If the purpose has changed, the service should not assume that the old permission still applies. This makes the data flow easier to audit and helps the team find the point where an unwanted use began.

1.8 Turning the research aim into testable questions

The broad aim of this case study can be broken into four questions. First, can one workflow represent different device types without losing the context needed for privacy checks? Second, can strict rules and AI-based risk signals work together without allowing a low model score to cancel a rule violation? Third, can the system make decisions near the data source while preserving a useful record for central review? Fourth, can the proposed design be assessed with repeatable tests rather than unsupported performance claims? These questions help connect the research discussion to the architecture, risk formula, and experimental plan.

Each question should have evidence that could support or challenge the design. The common event format can be tested by giving the system events from several device adapters and checking whether all required fields remain correct. The policy engine can be tested with allowed, denied, and uncertain requests. The edge design can be tested during normal operation and during a network outage. The audit service can be tested by rebuilding a decision from its recorded event ID, policy version, model version, reason, and action. The report does not answer these questions with measured results because the implementation has not been run. It defines a test plan that can provide those answers later.

Table 1. Summary of Proposed Framework Contributions.

Contribution	What is provided	Status
Integrated architecture	Connects devices, edge rules, AI risk, policy enforcement, review, and audit evidence.	Proposed design
Transparent decision process	Separates strict rules from AI risk signals.	Design specification
Mathematical model	Defines impact, likelihood, uncertainty, and scoring limits.	Illustrative formula
Research synthesis	Compares privacy, AI, edge, trust, and governance literature.	Narrative review
Evaluation plan	Defines cases, baselines, metrics, and repeatability details.	Not yet tested

2. Problem Statement

2.1 Main problem

The main problem is that privacy checks are scattered across devices, applications, gateways, storage systems, and business processes. Each component may understand only a part of the request. A device may know the record's type, but the cloud service may not know why it was collected. A policy may allow data to be sent to one receiver, while a later application routes the data to a different destination. The organisation then has difficulty proving that the data were used according to the approved purpose and retention rule [2][5][13].

The system must also manage uncertain events. A request may have missing metadata, the policy server may be offline, or the AI model may see a pattern it has not seen before. A system that always allows the request can leak information. A system that always blocks uncertain requests may stop important services. The design therefore needs a careful set of outcomes: allow, minimise, transform, block, hold, or send for human review. Each outcome must follow a rule that can be explained and tested.

2.2 Core challenges

The first challenge is technical diversity. Devices use different field names, clocks, units, network protocols, and computing resources. The second challenge is divided ownership. Device vendors, service operators, cloud providers, and data users may each control a different step. The third challenge is changing purpose. A record collected for safety may later be requested for research, marketing, or system optimisation. The fourth challenge is data linkage, where several records reveal more than each record alone. The fifth challenge is a lack of clear evidence after a decision is made [6][7][14].

The sixth challenge is model reliability. A model trained on older events may not recognise a new device or a new way of using data. The seventh challenge is alert volume. If the system flags too many normal events, reviewers may ignore alerts. The eighth challenge is legal context. A rule can depend on country, organisation role, data type, purpose, and effective date. A general tool must store this context rather than copying one rule into every environment [31][32][33][34][35][36][37].

2.3 Research gap

Prior research examines AI for privacy detection, federated learning, differential privacy, edge computing, trust management, and regulatory mapping. Each area offers useful methods, but the methods are often assessed with different data, metrics, and assumptions. A privacy classifier may report accuracy without measuring policy actions. A federated-learning paper may focus on model quality but not whether the trained model correctly handles purpose and retention. A governance paper may define duties but not show how an event is checked at runtime [1][3][7][8][9][10][11].

The gap addressed here is the connection between the parts. The proposal links data context to a policy check, AI risk ranking, a final action, and an audit record. It defines the rules that AI is not allowed to override and specifies how failures and uncertainty should be handled. This is not a claim that no prior system has ever combined these components. It is a practical synthesis designed for a case-study report, with explicit requirements and a test plan that can support fair comparison.

2.4 Scope and assumptions

The proposed system covers incoming events, local privacy checks, risk estimation, rule enforcement, audit records, and human review. It assumes that each organisation can define approved purposes, data categories, receivers, retention periods, and policy owners. The design does not attempt to interpret every law automatically. Legal mapping must be reviewed by a qualified person. The model is treated as a decision-support component, and its output is only one input to the final process.

The scope also excludes a claim of full anonymity, guaranteed protection against every attacker, or automatic proof of legal compliance. Encryption, anonymisation, federated learning, and differential privacy have specific limits. Those limits must be visible to the person deploying the system. The report focuses on a design that can be checked with a documented event set and measured in a future implementation. This keeps the project achievable and reduces the risk of making unsupported performance claims.

Table 2. Research Challenges and Gap Analysis.

Challenge	Why it matters	Proposed response
Device heterogeneity	Formats, clocks, and resources differ.	Use validated adapters and a common event schema.
Distributed data	Purpose and retention context may be lost.	Keep source, purpose, receiver, and retention metadata.
Rule complexity	Legal applicability depends on context.	Use dated and reviewed policy mappings.
Unknown events	Fixed rules may miss new patterns.	Use AI to rank unusual cases for review.
Unclear decisions	A score alone cannot explain an action.	Save reason codes and policy versions.
False-alert burden	Reviewers may ignore noisy alerts.	Measure false positives and workload.

2.5 Example failure path

Consider a sensor whose records are used for a safety task. The gateway checks the device, but a later application forwards the record to a new analytics service. If the later application does not repeat the purpose and receiver check, a valid first transfer may become an invalid second transfer. The issue is not necessarily a failure of encryption. The data may have been securely transferred to a destination that was not approved. A privacy framework must therefore check context at important points in the data path, not only at the first network connection.

A second example is a device that sends a malformed event because its firmware was updated. If the classification step treats missing fields as low risk, the event may continue with the wrong label. If the audit service stores only “allowed,” a later reviewer will not know that the system lacked important information. The expected behaviour should be to mark the event as uncertain, stop the risky step, and request a corrected record or human review. These examples show why safe handling of missing information is a core requirement.

2.6 Questions that a later evaluation must answer

The evaluation should answer whether the rule engine correctly handles known conditions, whether AI adds useful detections beyond rules, and whether the complete system stays within operational limits. Each question needs a separate measure. Rule correctness can be measured by comparing expected and observed actions. The added value of AI can be measured by the additional labelled violations found and the extra false alerts introduced. Operational suitability can be measured with latency, throughput, resource use, and failure-recovery time.

The evaluation should also ask whether a person can reconstruct the decision from the audit record. This is different from whether the software returned an action. A system could block an event for the correct reason but fail to store the policy version. It could store a complete record while applying the wrong rule. The test plan must therefore inspect both actions and evidence. These questions shape the methods and metrics used in later sections.

2.7 Failure conditions and system boundaries

A useful problem statement must say how the system can fail. It may receive malformed data, lose contact with a policy service, find an unknown receiver, use an expired rule, receive a model result that is outside the model’s known range, or fail to save an audit record. These cases are not rare details that can be ignored. They decide whether the system can be trusted when conditions are not perfect. Each failure needs a defined response, such as reject, hold, minimise, quarantine, or request review. The response must be chosen before deployment and must be included in tests.

The system also needs clear boundaries. It can check metadata, compare events with known rules, flag unusual behaviour, and record decisions. It cannot prove that a legal interpretation is correct by itself. It cannot stop a receiver from misusing data after a valid transfer unless the receiver is also controlled or monitored. It cannot guarantee that a model will find every new attack. Stating these limits prevents the report from claiming more than the design can provide. It also helps researchers identify which external controls, agreements, people, and audits are needed alongside the software.

2.8 Why fragmented control creates practical risk

When each subsystem keeps its own records, the organisation may know that a transfer happened without knowing why it was allowed. The device gateway may record an address, the analytics service may store a model score, and the data store may keep a retention date. If these records do not share an event reference, an auditor may not be able to reconstruct the path. The problem is therefore not only a lack of controls; it is also a lack of connection between controls. A linked event ID, policy version, decision reason, and outcome can reduce this gap without storing the full personal payload in every log.

Fragmentation also causes inconsistent responses. One system may block an expired request while another accepts it because it has an older policy. An edge node may continue after a cloud connection is lost without knowing that its local rule has expired. A central service may assume that an AI alert means a policy violation, even when the alert is only a signal for review. The proposed framework treats these as interface and governance problems. It defines a shared event record, a policy authority, a strict order of checks, and a safe fallback so that local and central services follow the same basic decision logic.

3. Background and Literature Review

3.1 AI-based privacy detection

Giordano, Palomba, and Ferrucci reviewed 152 primary studies on the use of AI to detect privacy concerns or protect privacy in IoT systems [1]. Their review is useful because it considers the tasks attempted by researchers, the model families used, and the way those models were validated. One important lesson is that a high model score does not automatically mean that the system is suitable for a real deployment. Studies need to explain their use case, data labels, validation method, and the limits of the reported result.

Rodríguez, Otero, and Canal survey machine-learning and deep-learning methods for privacy protection in IoT [3]. Their work helps place classification and detection methods within the larger group of privacy controls. Kaur and colleagues provide a later survey covering privacy-preserving approaches across collection, transfer, and storage [4]. These surveys show that privacy protection is not one algorithm. It is a set of methods applied at different points in the data life cycle. This supports the layered structure used in the proposed framework.

3.2 Privacy engineering and regulatory mapping

Kounoudes and Kapitsaki map user-centred IoT privacy approaches to the General Data Protection Regulation [2]. The value of this work is the connection between user rights and technical design. It shows why a system should keep information about purpose, user control, and data handling, rather than only applying a security mechanism. Product-design research also finds that privacy must be considered while products are designed and developed, not only after a product is released [12].

A compliance framework must turn a requirement into a system control and then into evidence. For example, a retention requirement can map to a timer, a deletion job, a record of its outcome, and a review procedure when deletion fails. A data-access requirement can map to a role policy and an access log. This mapping is more useful than a long list of legal words without clear implementation steps. However, the mapping cannot determine legal applicability on its own. A legal or privacy owner must confirm the context and keep the mapping current.

3.3 Federated learning and privacy-enhancing technology

Federated learning allows several devices or organisations to train a shared model while keeping raw records at their local sites. Surveys discuss its use in IoT networks and mobile edge systems, including health, transport, and industrial applications [9][10][26][27]. This can reduce direct transfer of raw data, but it does not make the system automatically private. Model updates may reveal information about the training records, and a malicious participant may send poisoned updates. The coordinating server and the communication channel also need protection.

Privacy-enhancing technologies can add protection around federated learning. Secure aggregation can help a server obtain a combined update without viewing each participant's individual update [48]. Differential privacy can limit the effect of one record on a release or model update, when its assumptions and privacy budget are correctly defined [8][47]. These methods have costs. They may increase communication, consume processing resources, or reduce model usefulness. A deployment needs to measure the cost and state what kind of privacy guarantee it can honestly provide.

3.4 Edge computing and heterogeneity

Research on edge computing explains why processing near a device can lower response time and reduce network traffic, while also creating security and management challenges [14][15][16][37]. Edge nodes may have limited memory and energy. They may also operate at many locations and be managed by different teams. The same privacy policy cannot be implemented by assuming that every device has the same hardware or software. A practical design needs adapters, a shared event format, and a clear way to update or retire local policy rules.

Mengistu and colleagues examine heterogeneity across IoT, wireless sensor networks, and federated learning [7]. They identify differences in devices, architectures, models, networks, and data distributions. These differences affect both privacy and model quality. A model can perform well for one device type and poorly for another. Therefore, a future evaluation should report results by device category where possible and test low-resource devices separately. Averaging every result into one number could hide important failures.

3.5 Security, trust, and auditability

Surveys of IoT trust and security show that devices must be authenticated and their behaviour assessed over time [17][18][19]. Trust is not the same as privacy compliance. A trusted device can still send data for an unapproved purpose, and a privacy-compliant request can come from a device that has weak security. The proposed framework keeps identity, behaviour, and policy compliance as separate inputs. This separation makes the decision easier to explain and reduces the risk that one good score hides a different kind of problem.

Security surveys also show that edge and IoT deployments face threats to devices, networks, management systems, and data [15][30]. Auditability makes it possible to review how the system responded to such events. An audit record should state which policy version was used and what action followed. It should not copy every personal field into a permanent log. The log is itself a data store and must have access restrictions, a purpose, and a retention period.

3.6 Literature synthesis and research gap

The literature supports a combined approach but also warns against treating a single technology as a complete solution. AI can flag patterns but can be wrong. Federated learning reduces central raw-data movement but can leak information through updates. Differential privacy provides a formal guarantee only under a defined mechanism and budget. Edge processing reduces some transfers but adds distributed security work. Legal mapping makes requirements more actionable but still requires human interpretation [1]–[12][14]–[27].

Table 3 compares the research strands, their strengths, and the gaps that remain. The design in this report focuses on the hand-off points between them. Each event receives context, the strict rules run before any soft AI recommendation is accepted, uncertain requests are not silently allowed, and the final decision is recorded. The framework should be evaluated as a whole, not only by measuring the AI model. This gives the future experiment a clearer purpose than reporting one model's accuracy alone.

Table 3. Literature Review Comparison.

Research strand	Main contribution	Gap relevant to this case study
AI for IoT privacy [1], [3]	Uses AI for detection, classification, and risk support.	Different papers use different datasets and labels.
Regulatory privacy engineering [2], [12]	Connects user rights and design controls.	Policy-to-runtime evidence is not always shown.
Differential privacy [8], [46]	Provides a formal release-level guarantee.	Parameters and utility costs must be reported.
Federated learning [9]–[10], [26]–[27]	Keeps raw training data at local sites.	Updates and participants still create risks.
Edge computing [6], [14]–[16]	Supports local processing and lower delay.	Edge management adds new security tasks.
Trust and security [17]–[19], [30]	Studies identity, trust, and threat controls.	Security alerts do not directly prove a privacy violation.
Recent IoT privacy reviews [4]–[5], [13], [20]–[25]	Map methods, applications, and open problems.	Need common tests across rules, model, and evidence.

3.7 Privacy protection methods solve different problems

Encryption protects the confidentiality of data in transit and at rest. Access control limits which account or service can read or change a record. Pseudonymisation reduces direct exposure of identifiers but may allow re-linking. Data minimisation reduces the amount of information collected or released. Differential privacy limits the effect of a defined record on a supported calculation. Federated learning reduces the need to centralise raw training data, but it does not remove all leakage from model updates [8][9][10][11][12][26][27][28][29].

These methods are not interchangeable. A system that uses encryption may still collect too much data. A pseudonymised dataset may still identify people through detailed timing or location. A federated model may leak information through its updates. Differential privacy may protect a release but does not stop a service from sending a raw record to the wrong receiver. The proposed architecture places each method at the point where it provides value and avoids making a broad claim that one technology protects the complete system.

3.8 What published evaluations often leave unclear

An important issue across privacy and AI research is the difference between a model result and a system result. A model may report accuracy or F1, but that does not show whether a privacy rule was correctly enforced or whether the final action was logged. Similarly, a privacy-enhancing method may report lower leakage under one threat model, but that does not mean all data sharing is safe. Reports should state the data source, target task, threat model, baseline, and evaluation conditions so the result can be interpreted correctly [1][3][8][9][10][11].

Another problem is that studies may use different labels and assumptions, which makes direct comparison difficult. One paper may call a network anomaly a privacy event; another may only label a policy violation. The meaning of "accuracy" will then differ. A fair comparison requires a shared event definition and an expected action for each case. The present report responds by separating technical security alerts, privacy policy violations, uncertain events, and normal events in the proposed evaluation process.

3.9 Review method and limits of this synthesis

This report uses a focused narrative review of peer-reviewed journal articles, selected conference papers, and official standards or legal documents. The sources were selected because they relate directly to AI-based privacy detection, edge processing, federated learning, differential privacy, IoT security, trust management, or regulatory mapping. The review is intended to inform the design rather than to claim a complete systematic search. No claim is made that every paper in the field has been captured or that a formal screening process was performed.

The findings are therefore used as design evidence, not as a count of how often one method succeeds across all deployments. Journal surveys help show the range of techniques and recurring limits, while primary studies provide specific mechanisms or examples. Official standards and regulations define context that journal surveys alone cannot supply. A full systematic review would require a documented database search, inclusion and exclusion rules, duplicate handling, screening by reviewers, and a complete study-selection record. That could be a separate future research task.

3.10 Recent research and what it adds

Recent reviews continue to treat IoT privacy as a problem that spans device design, data handling, AI, security, and governance. A 2025 review on AI-based methods for IoT privacy discusses approaches such as federated learning, anomaly detection, and encryption, while identifying open questions about practical deployment [4][31]. A 2024 review of data analytics and privacy paradigms also connects AI use with privacy requirements across sectors [5]. These studies matter because they show that privacy is not solved by selecting one model. The design must state what problem a method is meant to solve, what assumptions it needs, and how its effect will be tested.

Work on federated learning and edge IoT gives more detail about privacy leakage and the extra protection that may be needed around model updates [7][8][9][10][48]. Secure aggregation can reduce the server's view of individual updates, and differential privacy can limit the effect of one record under a defined mechanism and budget. These methods are not interchangeable. Secure aggregation changes what the coordinator can see, while differential privacy gives a mathematical bound on a release. Both can add resource costs, and neither removes the need to protect credentials, policy files, and output data. The proposed framework therefore treats privacy-enhancing technology as a set of selectable controls rather than as one all-purpose solution.

3.11 Comparing methods across the same questions

Literature comparisons are most useful when each approach is judged against the same questions. Does it reduce movement of raw data? Can it enforce purpose and receiver rules? Does it explain the reason for a decision? Can it work on low-resource devices? Does it create evidence for review? What are its known failure modes? A method may perform well for one question and poorly for another. For example, encryption protects a channel but does not decide whether the receiver has a valid purpose. An anomaly detector may flag unusual data flow but may not know whether the flow is legally permitted. A policy engine can enforce known conditions but may miss behaviour that was never encoded as a rule.

This report uses that common set of questions to bring the research strands together. Table 3 shows that privacy engineering, federated learning, edge computing, AI detection, and trust management each solve a different part of the problem. The proposed contribution is the order and connection of their outputs. The device layer supplies trustworthy context; strict policy checks decide the rules that must not be broken; AI helps rank unclear or unusual events; and an audit service records the final action. This is a design synthesis based on prior research, not a claim that a new algorithm has outperformed existing methods.

3.12 Limits of the literature base

The literature review in this case study is focused rather than fully systematic. It uses selected journal papers, reviews, standards, and official legal sources that are directly relevant to IoT privacy, AI risk detection, edge computing, federated learning, and governance. It does not report a complete database search, screening flow, or formal quality score for every paper. Therefore, it should not be described as a systematic review. The distinction matters because a reader needs to know how broad the evidence is and which claims the method can support.

A later full review should specify the databases searched, search strings, date range, inclusion and exclusion rules, screening process, and reasons for excluding records. It should separate studies that report real experiments from papers that propose an architecture or discuss a concept. It should also record the type of data, application area, evaluation method, and limitations reported by each study. Such a matrix would allow the team to check which findings transfer across smart health, smart cities, industry, and other sectors. Until that work is completed, the present literature section is best read as a focused evidence synthesis used to motivate the proposed design.

4. Requirements Analysis for AI-Driven Privacy Compliance

4.1 Stakeholders and needs

The main stakeholders are device owners, service operators, privacy officers, security teams, data users, auditors, and people whose data are collected. Device owners need a reliable way to register devices and withdraw access when a device is retired. Service operators need the system to work without adding unacceptable delay. Privacy officers need to check purpose, data minimisation, retention, and exceptions. Security teams need identity checks, update controls, and incident records. Auditors need a trail that links a request to the policy version and final action.

People whose data are processed need meaningful controls where the relevant law or service design provides them. The system should not hide a decision behind a technical score. It should be possible to explain in plain words why a request was allowed, changed, blocked, or sent for review. A reviewer should be able to correct a wrong data label or a policy rule without changing the historical record of what happened. Corrections should be versioned so that later investigators can see both the original and the corrected state.

4.2 Functional requirements

Functional requirements state what the proposed system must do. It must register device identities, validate event formats, add or check metadata, classify data sensitivity, verify declared purpose, check receivers, apply retention rules, calculate a risk score, choose an action, save evidence, and provide a review queue. It must also record policy versions and model versions so that a past decision can be reconstructed. If a required field is missing, the event must enter a defined uncertain state instead of being treated as automatically compliant.

The system should support data minimisation before a transfer. For example, it may remove a direct identifier, reduce location precision, or send a count rather than raw records when the intended task only needs an aggregate. It should also allow a reviewer to accept, reject, or request more details for cases that cannot be handled safely by a rule. Any override must include the reviewer's identity, reason, time, and expiry where an exception is temporary. The framework must not allow an AI score to override a mandatory deny rule.

4.3 Quality requirements

Quality requirements describe how well the system should operate. The design should be explainable, secure, responsive, interoperable, recoverable, and maintainable. The time needed to act on an event should be measured under an agreed workload. Memory and processor use should be measured on actual edge hardware. Interoperability should be tested with several event formats instead of being assumed from one successful device. Recovery tests should include expired policies, invalid signatures, unavailable services, and a broken network connection [14][15][16][36].

Privacy quality must also be measured. The system should collect the minimum event fields required to make and explain a decision. It should not copy raw data into central records without a defined need. If differential privacy is used, its parameters and privacy-budget accounting must be stated. If pseudonymisation is used, the report must explain who can access the mapping key. These are not only implementation choices; they affect what privacy claims the organisation can safely make [8][11][46].

4.4 Acceptance tests

Every important requirement should have a test with a clear expected result. The device test should include a valid device, an unknown device, and a revoked device. The metadata test should include a complete event and an event with a missing purpose. The receiver test should include an approved destination and an unapproved destination. The retention test should include an active period and an expired period. The AI test should include a normal pattern, a suspicious pattern, and an input outside the model's training range.

A test passes only when both the system action and the evidence record match the expected result. For example, a blocked transfer is not enough if the audit log cannot show which rule caused the block. Likewise, a complete log is not enough if the data were sent before the check completed. Requirements, test cases, and results should share stable identifiers. This creates a traceable link from the original requirement to the final evidence and allows a reviewer to repeat the evaluation.

Table 4. Functional and Non-Functional Requirements Matrix.

ID	Requirement	Acceptance check
F1	Authenticate device and check status.	Unknown or revoked device is rejected.
F2	Validate schema and required metadata.	Malformed or incomplete event is held.
F3	Check purpose, receiver, and retention.	Every decision names the policy version.
F4	Compute risk score with reasons.	Score, inputs, and uncertainty are recorded.
F5	Apply a defined action.	Observed action matches the expected policy result.
F6	Record minimal audit evidence.	All mandatory fields exist without raw payload duplication.
Q1	Work with different device types.	Adapters pass schema tests.
Q2	Remain responsive at target load.	Latency and throughput are measured.
Q3	Support review and correction.	Reviewer can reconstruct and challenge decisions.

4.5 Requirement priority and failure cost

Not every requirement has the same priority. A failure to authenticate a device or block an unapproved receiver can create an immediate exposure. A missing dashboard label may be less urgent, although it can still harm auditability. Requirements should therefore be ranked by likely harm, frequency, and the ability to recover. The ranking should identify which controls must fail closed, which can be retried, and which can continue under an approved local policy. This makes the system's response more predictable when several services fail at once.

Priority should not become an excuse to ignore lower-ranked requirements. For example, audit completeness may not decide whether one event is allowed, but it is needed to understand the system's later behaviour. Each requirement should have a test, an owner, and a release condition. A release should not be approved only because the main algorithm runs. It should also show that policy checks, error handling, logging, and review functions meet their acceptance conditions.

4.6 Traceability from requirement to evidence

A traceability matrix connects each requirement to the component that implements it, the test that checks it, and the evidence that proves the test ran. A device-identity requirement can link to the registration service, invalid-credential tests, and a rejection record. A retention requirement can link to the policy service, expired-retention test, and deletion outcome. A model-explanation requirement can link to the reason-code interface, reviewer test, and a saved explanation. The links should use stable identifiers rather than relying on a person's memory.

Traceability is useful when a policy changes. The owner can identify the affected component and rerun the linked tests. It is also useful when an incident occurs because the investigation can find which requirement was intended to prevent the event. The same matrix can be used during review meetings, internal audits, and future development. The appendix provides a starting structure, but a real project should keep the matrix in a version-controlled file alongside code and test results.

4.7 Requirements traceability

A requirements list is not enough unless each requirement can be traced through the design and the tests. A traceability record should name the requirement, the reason it exists, the system component that implements it, the test case that checks it, the result, and the person who approved it. For example, “reject a revoked device” connects device registration to the gateway identity check, a test with a revoked credential, and an expected rejection reason. “Record the basis for a decision” connects the policy engine to the audit service and a test that rebuilds a past decision from its event ID and rule version.

Traceability helps when a rule changes. If a retention requirement changes, the team can identify the policy rule, deletion task, dashboard field, test case, and documentation that need to be updated. Without traceability, changes may be made in one component while other components keep old assumptions. The traceability record should therefore be maintained as a controlled artefact rather than as an informal checklist. It should also record when a requirement is not implemented and explain whether that gap blocks deployment or needs a documented mitigation.

4.8 Acceptance criteria and negative tests

Acceptance criteria should be written in terms of visible behaviour. A successful request should not simply produce a score; it should pass all mandatory rules, carry the necessary metadata, and complete the allowed action. A denied request should stop the transfer and create the required evidence. An uncertain request should be held or reviewed rather than silently treated as compliant. A transformation request should not continue until the system has checked that the output has the required form. These criteria can be used by a developer and by a reviewer who does not know the internal model code.

Negative tests are especially important. They should include missing identity, revoked credentials, unknown data category, invalid timestamp, expired policy, receiver mismatch, missing purpose, model timeout, audit-store outage, and attempted use of an old policy. The test plan should check both the decision and the side effects. For example, it should confirm that a denied event does not still appear in the downstream receiver and that an error message does not reveal unnecessary personal information. A test passes only when the complete expected behaviour is observed, not merely when the main function returns an error code.

5. AI-Driven Privacy Compliance Architecture

5.1 Architectural overview

The architecture has five main operational parts: the IoT sources, secure gateway, edge privacy controls, AI risk engine, and policy decision service. A governance service supports all parts with policy versions, model records, audit logs, and human review. Figure 1 shows this design. The architecture is modular so that a hospital, city, or factory can use the same decision process while changing device adapters, data categories, and legal rules for its own setting.

The design separates mandatory rules from the AI risk score. The policy engine checks facts that must be true for a request to proceed, such as whether the destination is approved and whether the retention period has expired. The AI engine looks for unusual behaviour and ranks cases by risk. The final decision combines the outputs but does not let a model score cancel a strict rule. This separation is central to the proposed design because it keeps clear requirements from becoming uncertain model predictions.

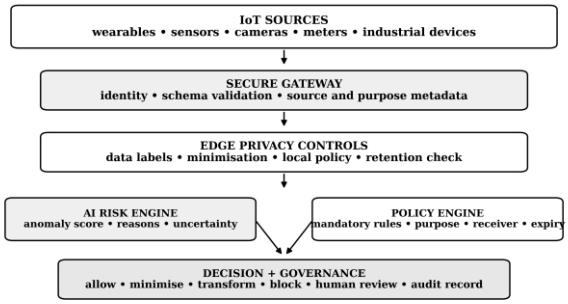


Fig. 1. AI-Driven Privacy Compliance Architecture.

The proposed modules are connected through explicit checks. The diagram is a design, not a deployed system.

5.2 Data flow and service boundaries

The process begins when a device sends an event and the metadata required for a decision. The gateway authenticates the source and checks the event structure. The edge layer applies data labels, purpose rules, and minimisation. If the event passes the hard checks, the risk engine may estimate whether the event is unusual. The policy service then selects the action. A minimal audit record stores the event ID, policy version, outcome, and reason codes. Raw data remain near the source when the task does not require central access.

Each service boundary needs a clear input and output. The gateway should return either a validated event or a rejection reason. The classifier should return a data label and its confidence or uncertainty, not a legal conclusion. The risk engine should return a score and reason codes. The policy engine should return a final action and the rule that determined it. The audit service should verify that required fields are present before accepting a decision record. This makes failures easier to detect and simplifies testing.

5.3 Policy and model management

A policy should have an owner, version, effective date, review date, approval status, and a set of tests. If a legal rule changes, the owner should update the policy and rerun its tests before deployment. The system should not silently replace a policy while a decision is being processed. It should store the exact version used by that decision. The same rule applies to models: each model needs a version, feature list, training-data description, evaluation results, and an approved use range.

Model updates require care because the data may change over time. A new sensor or a new service may produce input unlike the training examples. The system should monitor for missing features, unusual distributions, and changes in performance. A model that fails its quality checks should be rolled back or removed from automatic decisions until it is reviewed. During model uncertainty, the rules still run and sensitive cases can be sent to a human. The system should never treat a missing model as proof that an event is safe.

5.4 Fault handling and safe operation

A privacy system must define what happens when a service is not available. A gateway may use a signed local policy if it is still valid and has not expired. If it cannot verify a policy, the action should be conservative for sensitive data. It may hold the event, reduce the data, or block a transfer until a reviewer can assess it. The correct fallback depends on the service and the potential harm, but it must be selected in advance and tested. Silent fail-open behaviour is not acceptable for a high-risk path.

When the connection returns, the gateway should synchronise decision records and check for new policy versions. It should detect duplicate event IDs, missing log entries, and conflicting versions. The system should also record whether an action happened offline. This allows an audit to distinguish an intentional policy decision from a fallback caused by an outage. The proposed architecture aims to support service continuity without giving up privacy controls when central components are unavailable.

5.5 Deployment views: local, central, and hybrid

A local deployment keeps most checks inside the device network. It is useful when data cannot leave a site or when response time is important. A central deployment may simplify management and use larger models, but it can increase data movement and create a central service that needs strong protection. A hybrid deployment places identity checks and basic policy enforcement at the edge, while central services manage approved policies, model versions, and aggregated audit reporting. The hybrid arrangement is the main design assumed in this report, although it is not the only possible choice.

The deployment decision should be based on the data category, service delay, connectivity, hardware resources, and governance needs. If the edge device cannot safely store a policy, the team may need a hardened local gateway instead. If the service needs a large model, it may send a minimal feature summary to a central service where permitted. Each transfer should be part of the approved architecture. Figure 1 shows the logical components; a deployment diagram for a real system should name the actual hosts and trust boundaries.

5.6 Interface contracts and version control

Each module should have a documented interface. An event schema defines required fields and validation errors. The risk service defines how it returns a score, reasons, and an uncertainty state. The policy service defines the final action and any hard-rule failure. The audit service defines which fields are mandatory and how duplicate events are handled. A clear interface prevents two components from silently using different meanings for the same field, such as a retention date or receiver identity.

When a schema or API changes, it should have a version and a compatibility plan. Older devices may continue to send the previous format for a period, but the gateway should not guess how to interpret it. It should use a validated adapter or hold the event. Model and policy versions should also be recorded. This makes a decision reproducible even after the software has changed. It also allows the team to roll back a faulty release without losing the history of earlier actions.

5.7 Service interfaces and event contracts

The architecture depends on clear interfaces between the device adapter, gateway, risk engine, policy engine, action executor, and audit store. Each interface should define required fields, accepted types, error codes, timeout behaviour, and version rules. If the risk engine returns only a score with no model version or explanation, the decision service may not be able to explain the result later. If the action executor reports “blocked” before it has actually stopped the transfer, the audit log may describe an action that never happened. Interface contracts should therefore specify both the requested action and the confirmed outcome.

An event contract should be versioned. When a field is added or its meaning changes, the team should check whether old gateways and new policy services can still exchange records safely. Unknown fields should not automatically be trusted, and missing mandatory fields should not become empty strings that appear valid. Compatibility tests should cover devices with older firmware and gateways that connect after a period offline. This is a practical way to handle heterogeneity without requiring every device to run the same software version.

5.8 Deployment patterns and trust boundaries

The same logical architecture can run in several deployment patterns. In an edge-heavy design, identity checks, data minimisation, and strict local rules run next to the sensors. The central service maintains policy versions, model packages, and audit summaries. In a cloud-heavy design, most decisions happen centrally, which may simplify model management but increase network dependence and raw-data movement. A hybrid design can keep strict and time-sensitive checks at the edge while sending selected signals to a central model. No one pattern is best for every use case; it depends on the sensitivity of the data, the network delay, device capacity, and the risk if the central service is unavailable.

Every deployment should map its trust boundaries. A device can be compromised; a gateway can be misconfigured; a model service can be unavailable; a policy administrator can make an incorrect change; and an auditor account can be abused. The design should not assume that every internal service is trustworthy just because it is inside the network. Service-to-service identity, least-privilege access, signed policy files, protected secrets, and logs of administrative changes are required. The architecture should also define which failure can be handled locally and which failure requires processing to stop.

6. IoT Data Acquisition and Classification Layer

6.1 Device onboarding

Device onboarding creates a record of which device is allowed to send data. A useful record includes a unique device ID, owner, device type, model, software version, supported fields, network identity, and retirement status. Onboarding should also record the service purpose and the person responsible for the device. Device credentials should be unique where possible, protected in storage, and revocable. If a device is lost, sold, replaced, or retired, its access must be removed rather than left active indefinitely [36].

A registered device is not automatically trusted forever. The system should check whether the device is still active, whether its software is supported, and whether its certificate or key has expired. A change in owner or data purpose may require a new review. An event from a valid device may still violate privacy rules, so identity is only the first check. Onboarding creates a reliable starting point for the later data and policy checks; it does not replace them.

6.2 Common event format

Heterogeneous devices often use different names and formats for similar fields. A gateway should map these records into a common event structure. A simple event can contain an event ID, device ID, event time, data type, purpose, sensitivity label, intended receiver, retention class, and policy version. It can also contain a pointer to the original data where necessary. The gateway should validate types, ranges, timestamps, and required fields. If it cannot map a field safely, it should hold the event instead of guessing.

The common format should preserve provenance, which means information about where the event came from and what changes were made. If a location value is rounded, the record should state that the precision was reduced. If direct identifiers are removed, the record should state which transformation was applied. Provenance helps a reviewer understand whether the final value is still suitable for the original purpose. It also makes it easier to repeat tests because the same event format can be used across device types.

6.3 Data classification

Data classification determines how strong the controls should be. Public environmental values may need integrity protection but little access restriction. Operational secrets may need limited access because they reveal a company's process. Personal data need purpose, access, and retention checks. Sensitive personal data, such as health measurements, normally need stronger control and a smaller list of approved users. A field name alone is not enough: a numeric value can become sensitive when it is linked to a named person or a precise location [2][3][4].

The classification engine should use the content, source, purpose, and linkability of the event. It should also allow a person to correct a label when the context is unusual. Unknown data should not default to the lowest sensitivity class. It should be marked as unknown and sent for review or treated according to a cautious default. Table 5 presents example categories and their normal controls. The exact classification policy must be adjusted for each organisation and verified with the people responsible for the data.

Table 5. IoT Data Classification Matrix.

Class	Example	Normal controls
Public	Public weather value	Validate source and integrity.

Class	Example	Normal controls
Operational	Machine state or process metric	Limit access and retention.
Personal	Linked location or device use	Check purpose and reduce unnecessary detail.
Sensitive personal	Health measurement	Strict access, minimisation, high-risk review.
Unknown	New schema or unclear context	Hold, label cautiously, or review manually.

6.4 Purpose and ownership metadata

Purpose metadata tells the system why the data were collected and what the current request intends to do. The declared purpose should be selected from an approved list rather than written as any free-form sentence. The intended receiver should be an identifiable service or organisation. Retention metadata should describe how long the record may be kept and what event starts the retention clock. Ownership metadata should show which team can approve changes or answer a review request.

Missing metadata are a risk signal. If the system cannot tell whether a request matches the approved purpose, it should not quietly continue. It may ask the requesting service to supply missing information or send the request for human review. The system should also check whether metadata are stale. A purpose that was valid last year may no longer be valid after a service change. The event should be checked against the current approved policy before the data are used or sent to another receiver.

6.5 Metadata lifecycle and change handling

Metadata should be checked when it is created and whenever an important part of the context changes. The data owner may change, the service purpose may be updated, the retention period may be shortened, or a receiver may be removed. A record that was labelled correctly at onboarding can become stale. The system should store the source and time of the last metadata review and should flag cases that use expired values. This is especially important when devices remain in place for years and the original team no longer manages them.

A metadata update should be versioned, and the system should state whether it applies only to new events or also affects stored data. A retention change may require a cleanup task for existing records. A new purpose may require a separate approval process. The gateway should not rewrite historical records in a way that hides which metadata were used when a decision was made. It should preserve the versioned context needed to explain past decisions.

6.6 Common schema does not mean common meaning

A common event structure improves interoperability, but it does not guarantee that two fields mean the same thing. One device may report local temperature, while another reports a temperature calculated after smoothing. One location field may use a site coordinate, while another uses a person's device location. Adapters should document units, precision, sampling interval, and transformation history. The receiving service should not combine fields merely because they share the same name.

Classification should therefore include semantic checks as well as format checks. The system should validate units and allowed ranges, and it should preserve the device type and transformation details. If a field cannot be interpreted safely, the event should be held or marked unknown. These rules reduce the risk that a technically valid value is used for the wrong purpose. They also make model inputs more consistent because the model receives well-defined features rather than ambiguous values.

6.7 Data quality and lineage checks

Data quality affects both privacy and model quality. A timestamp that is wrong may make an old permission appear current. A data field with the wrong unit may be assigned an incorrect category. A duplicated event may create repeated alerts or be counted twice in an evaluation. A gateway should therefore check schema, time, units, required values, duplicates, and device version before the event reaches the risk engine. The system should not guess a correction when the correct value cannot be established. It should record the error and send the event for review or follow a safe rejection rule.

Lineage records the main steps that changed the data. For example, a gateway may remove a direct identifier, reduce location precision, aggregate several readings, or convert a value into a standard unit. The lineage record can describe the transformation without storing the full original payload. This helps a reviewer understand what was sent to the next service. Lineage should be protected because it can still reveal which data source was used, when an event was created, or which service processed it. It should also have a retention rule, rather than being stored forever by default.

6.8 Data ownership and context are not the same as device ownership

The owner of a device is not always the person whose data it contains. A hospital may own a monitor, while the patient is the person described by the measurement. A transport company may own a vehicle sensor, while its location data can describe a driver or passenger. The system should distinguish device owner, data controller or responsible organisation, data subject where known, service operator, and intended receiver. These roles can differ, and the role that approves a device does not automatically approve every use of the data it creates.

The metadata should record the purpose that was approved and the scope of that approval. A broad label such as “analytics” may not be enough to decide whether a specific use is allowed. A clear purpose should describe the task in a way that a policy can check. If the purpose is changed later, the team should review whether the new processing has a valid basis and whether notice or permission steps are required in that context. When the system cannot establish the context, it should use an uncertain state and avoid treating missing information as a low privacy risk.

7. Edge Intelligence and Local Governance Layer

7.1 Why process at the edge

Edge computing places checks near the device or within the same local network. This can reduce response time and reduce how much raw data moves to central storage. A camera may send a count rather than a full video stream. A wearable may apply a local quality check before sending a summary. A factory sensor may remove a worker identifier before sending machine statistics. The benefit depends on the task: if the remote service needs the full record for a valid reason, the edge cannot remove it without changing the purpose [6][14][15][16].

Local processing can also keep a system useful when internet access is unreliable. The gateway may apply a valid signed policy while the central service is temporarily unavailable. It should not invent new rules during the outage. When the connection returns, the gateway can synchronise the decision record and obtain updated policies. Edge-first design therefore needs both local autonomy and central governance. The local device must know which decisions it can make and which decisions require a central service or a human reviewer.

7.2 Lightweight AI at the edge

Small models can classify common events or flag unusual activity without sending every raw record to a cloud service. The model may use event rate, destination history, device type, or the presence of required metadata. These features should be selected because they help answer a defined question. The system should not collect extra personal details simply because they might improve a model. The model's input list should be documented and reviewed with the privacy owner before use [3][9][10].

A compressed model is not automatically as good as the original. Quantisation, pruning, and smaller input sets can change its performance. The system should measure model quality on each target device class and record the memory, processor, and energy cost. If the edge hardware is too limited, the system can apply explicit rules locally and send a minimal event summary for further analysis. A weak edge model should not be presented as equivalent to a larger model without evidence.

7.3 Local policies and safe fallbacks

A gateway should store only the policy versions it needs for its assigned tasks. Each policy must be signed, versioned, and time-limited. A signature helps the gateway detect unauthorised changes, while the version and expiry prevent old rules from being used forever. The system should check whether the local policy is still valid before it allows a sensitive transfer. If the policy is expired or damaged, the gateway should hold or block high-risk actions according to the local fallback plan.

The local policy should include a clear list of actions: allow, minimise, transform, block, or review. A transformation must be defined in a way that can be tested. For example, reducing location precision should specify the allowed resolution. If the action cannot be completed, the transfer should not proceed as though it succeeded. The audit entry should show both the requested action and whether it was completed. This helps prevent an apparent policy pass from hiding a failed transformation.

7.4 Edge security and management

Edge devices are often installed in locations where physical access is possible. They need protected boot, unique identity, restricted administrative access, supported software, and a safe update process. The device inventory should show when each device was last checked and whether the vendor still supports it. Devices that no longer receive security updates should be reviewed for isolation or replacement. These operational details matter because a privacy rule is ineffective if an attacker can alter the gateway that applies it [15][30][36].

Monitoring should detect changes in local policy, repeated authentication failures, unusual outbound connections, and missing audit records. Monitoring records must be minimised because the monitoring system can itself reveal behaviour. The organisation should decide who can view alerts and how long they are retained. Figure 2 shows the main event pipeline; the same order should be used for local and central processing so that a reviewer can understand where an event was checked and where it was finally handled.

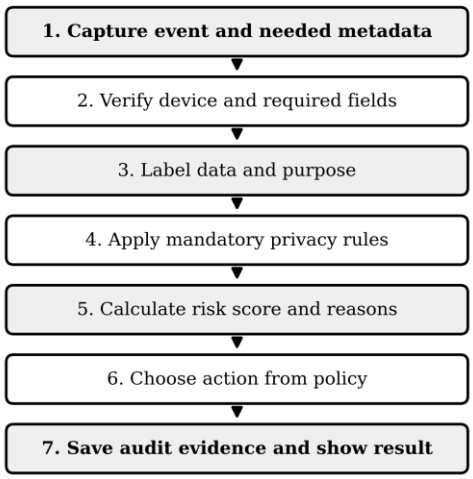


Fig. 2. End-to-End Privacy Compliance Pipeline.

The same event ID should connect each stage and the final audit entry.

7.5 Policy update and rollback procedure

Policy updates should be delivered as signed packages with a version, issue date, expiry, owner, and list of required tests. The gateway should verify the signature and compatibility before activating the package. If the new package fails a local validation, the gateway should retain the last valid policy only if it remains within its approved expiry. The system should log that the update was rejected and notify the owner. A rollback should not erase the evidence that a faulty policy was installed or attempted.

The update process should include a staged rollout. A small group of gateways can receive a new policy first, and their status can be checked before wider deployment. If a policy changes a high-impact action, the owner should verify the new test results and the affected use cases. This reduces the chance that one incorrect rule affects every device at the same time. The process should also define what happens if the central management service cannot reach a gateway for an extended period.

7.6 Energy, network use, and placement trade-offs

Edge placement affects more than delay. A local check may reduce network traffic and avoid sending large raw records, but it uses device energy and processor time. A central model may use more computing resources in one place but require network transmission and create a dependency on connectivity. The appropriate choice depends on the event frequency, data size, acceptable delay, and the effect of a lost connection. These choices should be measured on actual target hardware, not estimated only from the logical architecture.

Figure 6 is included as an assumption-driven example of how latency may respond to load. It is not an observed result. A real benchmark should record event rate, payload size, network conditions, hardware, and whether the measurement includes policy and audit time. If edge and cloud approaches are compared, they must use the same workload and record the same start and end points. These conditions make the plotted difference meaningful and repeatable.

7.7 Choosing which work belongs at the edge

Not every task needs an AI model on every device. A small gateway may only have enough memory for identity checks, schema validation, simple thresholds, and a limited policy set. A stronger edge server may run a small classifier, aggregate data from several devices, and detect unusual transfer patterns. A central service may be better for long-term analysis, model training, policy management, and comparisons across sites. The placement decision should be made task by task, using sensitivity, delay requirements, available resources, connectivity, and the cost of failure.

A model that works in a powerful server may not work on a small gateway. Its memory use, start-up time, response time, energy use, and update process should be measured on the intended hardware. Quantisation or model compression may reduce cost but can also change model behaviour. The compressed model must be tested again on the relevant event types. A deployment record should state which model version runs at which location, how it is updated, and what the system does if the model fails to load.

7.8 Policy synchronisation and safe fallback

Local policies can improve response time, but they can become stale. Each policy package should have a version, issuer, signature, activation time, expiry time, and set of supported devices or regions. The edge node should verify the signature before loading the package and confirm that it is within its valid period. If a new package is received, the node should check that it is complete and compatible before replacing the current package. A failed update should not leave the node in a half-updated state.

The fallback decision should depend on the type of operation. A low-risk temperature reading may be queued during an outage, while a high-risk transfer of identifiable health data may need to be stopped. The rule must be written before the outage. The local node should mark the event as processed under offline conditions and later synchronise the record. It should not pretend that a central review occurred when it did not. Recovery testing should include an expired local policy, a missing signature, partial network connectivity, delayed audit upload, and conflicting policy versions.

8. AI-Powered Privacy Risk Assessment Layer

8.1 Risk is not the same as a legal verdict

A risk score helps decide which events need attention first. It does not decide by itself whether data processing is legal. Risk can depend on possible harm, the chance of a harmful event, the sensitivity of the data, uncertainty, and the scope of the request. Legal compliance also depends on facts such as purpose, role, legal basis, and jurisdiction. These factors may be represented by explicit rules instead of a model score. The final system must keep risk ranking and compliance status separate [1][3][8].

The score proposed in this report is used for teaching and design. It gives a simple way to combine impact, likelihood, and uncertainty into one number. The chosen weights are not a standard and have not been fitted to real cases. The score can help organise a review queue, but it must not create a false feeling of precision. A real implementation would need expert review, labelled test cases, calibration where relevant, and a threshold chosen for the use case.

8.2 Inputs and feature design

The impact input should reflect the possible harm if the event is mishandled. A health record sent to an unknown receiver may have higher impact than a public temperature reading. The likelihood input should reflect how likely the harmful event is under the available evidence. The uncertainty input should rise when important facts are missing, the model sees an unfamiliar device, or the event is outside the range covered by validation. These inputs should have definitions that two reviewers can apply consistently.

Feature design affects privacy. A risk model may be tempted to use detailed user profiles or unrelated personal data. Such features should be rejected unless there is a clear and approved reason to use them. The model should use the smallest set of fields that can support its task. It should also record when a feature is missing. Replacing missing information with a default value can hide the very uncertainty that should trigger review. A missing-data policy should be tested as carefully as normal data paths.

8.3 Proposed risk-score equation

Let I be the impact score, L be the likelihood score, and U be the uncertainty score. Each input is scaled from 0 to 1. Equation (1) defines the proposed score. The weights make impact the largest contribution, likelihood the second largest, and uncertainty the third. They reflect a design choice for this report, not a result from a trained model. A later study should test other weights and should explain why the final weights were selected.

For an example event, let I = 0.90, L = 0.85, and U = 0.80. The three contributions are 36.00, 29.75, and 20.00 points. The total is 85.75, which can be rounded to 86 for display. Figure 7 shows sample event scores, while Figure 3 breaks down this illustrative risk score. All shown values are synthetic. No measured deployment data were available. A real report must replace these examples with values from a documented dataset before claiming that the score performs well.

$$R = 100 \times (0.40I + 0.35L + 0.25U) \quad (1)$$

I is impact, L is likelihood, and U is uncertainty. Each input is scaled from 0 to 1. The weights are design choices for this case study and require validation before operational use.

$$x_{norm} = (x - x_{min}) / (x_{max} - x_{min}) \quad (2)$$

Equation (2) scales a value to the range 0 to 1 when x_max is greater than x_min. If the range is zero, the implementation must use a separate defined rule rather than divide by zero.

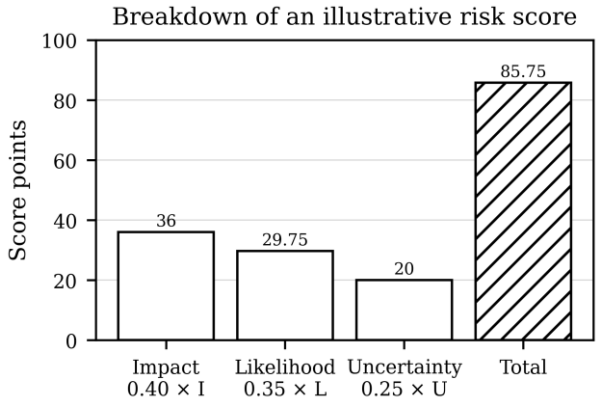


Fig. 3. Breakdown of an Illustrative Risk Score.

The graph shows the formula contributions for I=0.90, L=0.85, U=0.80; it is not measured data.

8.4 Risk bands and uncertainty

A possible starting point is to define low, medium, and high risk bands, but the thresholds must be selected after testing. A low score may still be blocked when a mandatory rule fails. A medium score may request extra metadata or a second check. A high score may lead to blocking, minimising, or human review. The action depends on the policy and the possible harm, not only on the number. Thresholds should also account for the cost of a false negative and the burden of false alerts.

Uncertainty deserves special care. If a model has not seen a device class or the required fields are missing, the score may not be reliable. In those cases, the system should state that the score is uncertain and use a safe fallback. The score should not be displayed as a calibrated probability unless calibration has been measured. The sensitivity graph in Figure 4 shows how a score changes as uncertainty changes while the other inputs remain fixed. It explains the equation rather than claiming model performance.

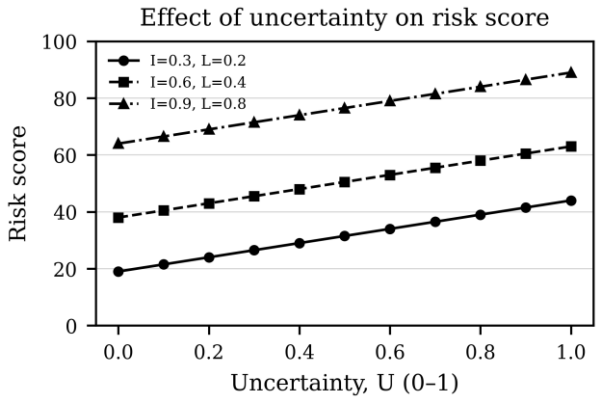


Fig. 4. Risk Score Sensitivity to Uncertainty.

Formula-based curves. Impact and likelihood are fixed in each curve; this is not a model benchmark.

8.5 Model training, drift, and review

A model should be trained on events that match its intended task. Network attack data can help identify suspicious traffic, but they are not the same as labels for an unlawful or unapproved data use. A privacy-risk model needs labelled cases linked to purpose, receiver, retention, and context. Labels should be reviewed by people who understand the use case. The dataset should include normal events, violations, uncertain events, and different device types. The report should state the number of examples in each class and the known limits [1][3][29].

The model can become less reliable when devices, software, or user behaviour change. The system should monitor drift and re-test performance after a significant update. It should not retrain itself on every new event without a controlled process. New training data must have a known source and permitted use. A model change should create a new version, pass the agreed tests, and receive approval. If the model fails, the system can fall back to explicit rules and human review rather than silently using a weak model.

8.6 Threshold calibration and reviewer workload

A threshold determines which events enter a higher-risk queue. If the threshold is too low, many normal events may be sent to review. If it is too high, the system may miss events that require attention. The correct threshold depends on the relative harm of a missed event and the cost of reviewing an alert. It should be selected on a validation set, then checked on a separate test set. The report should avoid choosing a threshold after looking only at the final test results.

Thresholds may differ by context, but each difference should be justified and tested. Health data may need a lower threshold than public weather data, yet a lower threshold may increase review volume. The system should measure both detection quality and queue burden. A reviewer workload that exceeds the available staff capacity can make the control ineffective even if the score is mathematically well defined. The threshold should therefore be part of the governance policy and be reviewed when data or staffing conditions change.

8.7 Calibration, fairness, and out-of-range inputs

A score can rank cases without representing a true probability. If the system displays a probability, it must be calibrated against labelled outcomes. A model that says “80 percent risk” should be correct about that frequency across the defined evaluation setting; otherwise the number may mislead users. The proposed weighted score in this report is only a ranking score. It should not be described as a probability. A calibrated model would require suitable training data, validation, and periodic checks.

Out-of-range inputs require a defined response. A new device may provide values that were not seen during training, or a field may be missing. The system should not silently force every input into a normal range and continue. It should create an uncertainty flag, apply mandatory rules, and choose a safe review path. Model output should also be checked for large differences across device types. These steps do not remove all bias, but they make uncertainty and uneven performance visible to the people responsible for the system.

8.8 Threshold selection and model calibration

A risk threshold controls how much work goes to the human review queue. If the threshold is very low, the system may catch more unusual events but create many false alerts. If it is too high, reviewers may see fewer alerts but the system may miss important events. The threshold should therefore be selected using labelled validation events and a stated cost for different errors. The cost of missing an unauthorised transfer may be much higher than the cost of reviewing a valid transfer. Thresholds should not be picked only because they create a visually attractive score distribution.

Calibration is a separate question. A model that reports a confidence of 0.8 should be correct about 80 percent of the time on comparable cases if the confidence is well calibrated. This is not guaranteed by training accuracy. Calibration must be checked on data that were not used to fit the model, and it may change across device types or over time. If the system uses an uncalibrated anomaly score, it should label it as a ranking signal rather than a probability. The interface should also show uncertainty when the event is outside the model's known range.

8.9 Fairness and unequal error costs

Privacy errors can affect groups differently. A model may receive more examples from one device type, service, language, region, or user group than another. If the test set is unbalanced, an overall score can look good while the model performs poorly on a smaller group. Where lawful and appropriate, the evaluation should examine errors by relevant data source or use case. It should avoid collecting sensitive group data just to create a fairness table unless there is a justified and protected method for doing so.

The cost of an error also depends on the use case. A false alert in a public sensor system may create extra work. A missed violation in a medical or financial system may create greater harm. The score should be used to support a policy decision, not to rank people or make a consequential decision without review. Any change to features, thresholds, or labels should be recorded. A model owner should be able to explain why a score changed between versions and what tests were used to approve the update.

9. Automated Compliance and Policy Enforcement Layer

9.1 From written rule to runnable policy

A written privacy requirement is not enough for a live system. The organisation needs to turn it into a policy with inputs, conditions, actions, exceptions, and evidence. A policy should state the data category, allowed purpose, approved receiver, permitted location, retention rule, and responsible owner. The policy also needs a version, effective date, and approval record. An AI tool may help draft a rule or suggest test cases, but a qualified owner must check the meaning before the rule is used.

Policies should be stored in a format the gateway and central service can both understand. A common format reduces differences between device types. It should also allow jurisdiction-specific rules and sector-specific exceptions. A system must not copy a rule from one country to another without checking whether the same law applies. The policy service should explain which conditions were tested, which conditions passed, and which conditions failed. This makes a later audit more useful than a simple "compliant" or "not compliant" label.

9.2 Hard rules and soft signals

Hard rules represent requirements that cannot be ignored. Examples include a revoked device identity, a forbidden receiver, a missing legal approval required by policy, or an expired retention period. If a hard rule fails, the system must apply the prescribed action. A soft signal is different. It may indicate that activity is unusual, data are sensitive, or extra review is sensible. A soft signal can change the review priority, but it should not cancel a hard rule [1][2][17].

This separation reduces two common mistakes. The first is allowing a request because an AI model assigns a low risk score, even though the receiver is not approved. The second is treating every unusual event as a violation when a valid reason may exist. Clear rules handle facts that are known, and AI helps surface events that need attention. Where facts are uncertain, the system should ask for more evidence or send the case to a reviewer.

9.3 Policy tests and change control

Every policy should have test cases for valid use, invalid use, missing information, expired approval, and attempted bypass. The test record should include the input event, expected action, actual action, and whether the audit record was complete. Policy changes should be reviewed when a law changes, a device is added, a new receiver is introduced, or an old purpose is no longer used. Tests should run before deployment and after a policy is changed. A policy should not go live merely because it can be written in the chosen format.

An exception must be limited and traceable. It should name the person who approved it, give the reason, state its scope, and include an expiry when appropriate. The audit should preserve the original rule version and show that an exception was used. If a service repeatedly relies on exceptions, the policy owner should investigate. Repeated exceptions can mean that a rule is too strict, the purpose is not described correctly, or a team is trying to bypass the normal process.

9.4 Actions and evidence

The policy service should return an action from a small, known set: allow, minimise, transform, block, quarantine, or review. The system should record the action requested and the action actually completed. For example, if a request is allowed only after location precision is reduced, the audit record must show that the transformation succeeded. If the transformation failed, the event must not continue as though the data were safe. This is important because the act of enforcing a policy can fail even when the rule itself is correct.

Table 6 gives sample conditions and actions. The table is a design example rather than a universal legal rule set. Each organisation must define which laws and internal policies apply to its service. The rule engine should keep the source of each requirement and a link to a test. A reviewer should be able to follow the path from a requirement to the policy, from the policy to an event decision, and from that decision to its audit evidence.

Table 6. Compliance Rules and Actions Matrix.

Condition	Policy status	Expected action
Device is revoked	Hard rule failed	Reject and record the reason.
Required purpose is missing	Cannot assess	Hold and request review.
Receiver is not approved	Hard rule failed	Block transfer.
Retention has expired	Retention rule failed	Run approved deletion or isolation.
AI flags unusual behaviour	Risk signal	Review or apply defined risk action.
All mandatory checks pass	Allowed under current policy	Continue and record outcome.

Condition	Policy status	Expected action
Policy service is offline	Degraded mode	Use valid signed policy or fail safely.

9.5 Rule conflicts and precedence

Two policies may apply to the same event and appear to require different actions. A purpose rule may allow a transfer, while a retention rule may say that the data should already have been deleted. A local rule may allow a transfer to a gateway, while a jurisdiction rule may prohibit the requested cross-border route. The system needs a documented precedence model. Hard legal and security prohibitions should not be overridden by a permissive local default. When the conflict cannot be resolved automatically, the event should be held for review.

Precedence rules should be tested with deliberate conflict cases. The policy engine should report which rule won and why. It should not only return a block without identifying the condition that caused it. The organisation should also review repeated policy conflicts because they may show that the catalogue is inconsistent or that a new use case was not properly assessed. Policy conflict handling is part of the system design and should be included in the acceptance tests.

9.6 Policy performance and explainable outcomes

A rule engine can be correct but still create operational problems if rules are too broad or if the event schema is too strict. The team should record the number of events held because data are missing, the number blocked by hard rules, and the number referred to human review. These counts help identify a policy that prevents legitimate work or a gateway that fails to collect required metadata. They do not prove that one outcome is right, but they show where review is needed.

Every action should have a readable explanation and machine-readable reason codes. The reason code should be stable enough for analysis across software versions. The readable message should describe the rule without revealing sensitive details to a user who is not authorised to see them. This allows dashboards to summarise events while keeping full evidence restricted to approved roles. A real system should test that explanations are consistent with the policy result and do not disclose extra personal data.

9.7 A rule format that can be tested

A policy language should make the main conditions clear. A rule can name the data category, permitted purpose, approved receiver, jurisdiction, retention class, required transformation, and action when a condition fails. The rule should also carry an owner and version. It should be possible for a reviewer to read the rule and understand what it means without reading program code. The executable form should be generated from a reviewed source or maintained alongside it, so the written rule and the running rule do not drift apart.

Rules should be checked for conflict before deployment. One rule may allow a transfer while another requires it to be blocked. One policy may say that a field must be deleted while another process attempts to retain it. Conflict tests should find these cases in advance. The team should define which rule has priority, such as a strict deny rule taking precedence over a general allow rule. When no clear priority exists, the system should not invent one at run time; the request should be held for an authorised decision.

9.8 Exceptions, rollback, and policy ownership

Exceptions are sometimes necessary, but they should be limited and traceable. Each exception should state its reason, approving person, start time, end time, data scope, and compensating control. A permanent exception with no review date can silently become the new normal. The system should report repeated exceptions so the organisation can decide whether the base policy is too strict, the workflow is broken, or a user is trying to bypass a control.

Policy rollout should be staged. The team can first test the rule against historical events, then run it in a monitoring mode, then enable enforcement for a limited group before wider deployment. A rollback plan should restore a known-good policy if the new version causes errors. The audit log should retain the version used for each decision, even after a rollback. That record lets the team distinguish a bad policy from a software defect and makes it possible to repeat the evaluation later.

10. Privacy Preservation and Data Protection Layer

10.1 Data minimisation and purpose limitation

Data minimisation means collecting and using only the fields needed for the task. A traffic service may need the number of vehicles passing a junction, not the full images of every driver. A hospital dashboard may need a trend or alert, not every raw sensor reading. A factory service may need machine temperature, not a worker's name. Removing unnecessary fields before transfer reduces what can be leaked or misused later. It also lowers the amount of data that needs encryption, storage, access control, and deletion [2][3][4][5].

Purpose limitation means that the data should be used for the purpose approved at collection or for another use that is permitted under the applicable rules. An AI system should not make a new purpose valid merely because it can discover a useful pattern in the data. If the purpose changes, the organisation must assess whether the new use is allowed and update its metadata and policies. This decision may require human or legal review. The system should record the approved purpose and test that each request matches it.

10.2 Encryption, access control, and key management

Encryption protects data while they travel across a network or remain in storage. It does not stop a service that already has permission to misuse the data. Access control limits which users and services can view, change, or export information. The system should use least privilege: a service should receive only the rights it needs for its work. Administrative access should be limited, reviewed, and logged. Keys and credentials must be protected separately from the data they unlock. A lost key can make data unavailable, and a leaked key can weaken protection across many records.

Encryption should be selected to fit the device, data type, and performance constraints. Low-power sensors may use a gateway for some cryptographic work, but the gateway becomes a critical security point. The organisation should record how keys are created, rotated, revoked, backed up, and destroyed. It should also test what happens when a device is lost or a key is suspected to be exposed. Encryption is one part of privacy and security, not a substitute for correct purpose, retention, and receiver checks [15][30][36].

10.3 Pseudonymisation and anonymisation

Pseudonymisation replaces direct identifiers with a code or token. It can reduce exposure because an application may work with a code instead of a name. However, the code may still be linked to a person if a key or lookup table exists. The mapping information must be protected, and access should be limited. A pseudonymised dataset should not be described as anonymous merely because names are hidden. Other fields, such as exact time and location, can still identify a person when linked with outside information [3][4][12].

Anonymisation aims to reduce the chance that a person can be identified from the released data. Whether a dataset is truly anonymous depends on its detail, the available outside information, and the likely attacker. Removing names alone is rarely enough for rich IoT streams. Aggregation, generalisation, suppression, or formal privacy methods may be needed. The system should record the transformation used and the purpose of the resulting dataset. If the data can still be linked back to a person, stronger rules may still apply.

10.4 Differential privacy

Differential privacy gives a mathematical limit on how much the output of a supported calculation changes when one person's record is added or removed from a dataset. The common parameters are epsilon, written ε, and sometimes delta, written δ. Smaller ε generally means a stronger formal privacy guarantee, but more noise may be required and the result may become less useful. The guarantee depends on the neighbouring-dataset definition, the mechanism, and the way repeated releases are counted [8][47].

Differential privacy does not automatically protect every part of an IoT system. It does not hide an unencrypted network request, stop a policy engine from sending a record to an unapproved receiver, or fix an insecure device. It is most useful for defined releases, summaries, or learning operations where the privacy parameters can be tracked. The organisation should state the protected unit, the chosen ε and δ, and how the privacy budget is accumulated. If these details are missing, a claim of differential privacy is incomplete.

$$Pr[M(D) \in S] \leq e\hat{}\varepsilon \, Pr[M(D') \in S] + \delta \qquad (3)$$

Equation (3) states the (ε, δ)-differential privacy condition for neighbouring datasets D and D′, a random mechanism M, and output set S. The neighbouring-data definition and the privacy budget across releases must be recorded.

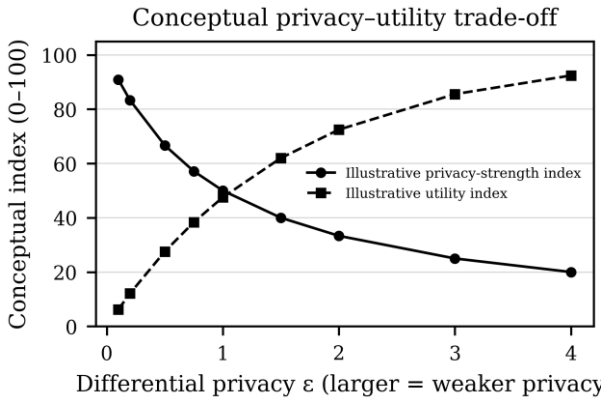


Fig. 5. Conceptual Privacy–Utility Trade-Off.

The curves are conceptual indexes to explain the expected trade-off. They are not measurements of privacy or accuracy.

10.5 Federated learning and secure aggregation

Federated learning trains a shared model across several devices or organisations while keeping the raw training records where they were collected. Each participant computes an update and sends it to an aggregation service. This can reduce direct transfer of personal records, which is useful in hospitals and distributed IoT settings [9][10][11][26][27]. However, updates can still reveal information. Participants can also be malicious or have data that differ greatly from the other participants.

Secure aggregation can help the server receive a combined update without seeing every individual's update [48]. Differential privacy may be applied to limit the influence of individual records [8][46]. Both methods bring extra cost and have assumptions. The system must record which participants joined, which model version they used, how updates were protected, and what happened if a participant failed. Federated learning is a useful option when raw data should stay local, but it is not a complete compliance framework by itself.

10.6 Retention, deletion, and copies

Retention control must cover more than the main database. IoT systems may copy data into local caches, message queues, dashboards, exports, backups, or model-training sets. A deletion request should identify which copies fall within scope and which records must be retained for a documented reason. The system should not report “deleted” if the data remain in a cache or an export. It should record the outcome of the deletion task and make failures visible to an owner.

Audit evidence should not defeat deletion by storing the full payload indefinitely. In many cases, the event ID, policy version, action, and time are enough to show that the process ran. If a legal or operational rule requires longer retention, the owner should document the reason and limit access. The retention plan should also consider derived features and model-training data. A data lifecycle review is necessary whenever the organisation adds a new storage layer or a new analytic service.

10.7 Selecting a privacy control for each threat

Control selection should begin with the information that needs protection and the likely misuse. Encryption addresses exposure during storage and transfer. Access control addresses unauthorised viewing or change. Data minimisation reduces the amount of information collected. Pseudonymisation reduces direct identification but can leave linkability. Aggregation reduces detail in a result. Differential privacy can limit the influence of a defined record on a release. Federated learning can reduce raw-data centralisation. These controls can be combined, but each should have a clear purpose and test.

The team should record the expected benefit and the cost of each control. Encryption requires key management. Differential privacy uses a privacy budget and can reduce accuracy. Federated learning requires communication and secure coordination. Aggregation can make a result less useful for small groups. The selected combination should match the use case, data type, and hardware. A control should not be included only because it is popular in the research literature; it should address a stated risk and be tested under the expected workload.

10.8 Selecting a privacy tool by threat and purpose

Privacy tools should be selected by asking what information must be protected and from whom. Encryption is useful when storage or communication may be observed, but authorised services still see data after decryption. Access control limits who may use a record, but it does not hide the record from a user who is allowed access. Aggregation can remove unnecessary detail when a task only needs a total or average, but small groups may still reveal a person. Pseudonymisation lowers direct exposure, while differential privacy can provide a formal bound for a defined statistical release or learning process [8][29][48].

The team should record the threat model, selected method, parameters, expected protection, cost, and remaining risk. For differential privacy, this includes the neighbouring-data definition and privacy budget. For federated learning, it includes the participants, update visibility, protection for the communication channel, and response to a malicious participant. For encryption, it includes key storage, rotation, revocation, backup, and recovery. This information makes it possible to compare methods fairly and prevents a broad claim such as "the data are private" from hiding the limits of one control.

10.9 Retention, deletion, and downstream copies

Retention is a whole-system issue. A record may be copied to a cache, message queue, export file, analytics table, backup, or model-training set. Deleting it from the main database does not automatically remove these copies. A retention policy should state which stores are in scope, how deletion is triggered, how failures are reported, and what proof can be kept. The proof should contain a minimal reference and status, not a full copy of the personal record that was deleted.

Derived information also needs review. A model trained on personal records may continue to reflect patterns from those records even when the original table is deleted. Whether the model needs retraining or special handling depends on the model, the data, the legal setting, and the stated policy. The report does not claim that simple deletion removes all influence from a trained model. A future implementation should document how it treats training data, backups, exports, and derived artefacts, and should test the deletion workflow from end to end.

11. Explainability, Transparency, and Auditability Layer

11.1 Explain each decision

A person who reviews a blocked request should be able to understand what happened without reading model code. An explanation should name the event, the policy rule, the relevant facts, the action, and the reason. For example, "the request was blocked because the receiver is not on the approved list" is clearer than "risk is 86." A risk score can support prioritisation, but it does not tell the user which policy was broken or whether the policy was valid for the situation.

The explanation should separate a model signal from a mandatory rule. The AI may say that a transfer pattern is unusual. The policy engine may say that the destination is forbidden. These are different reasons and should be recorded separately. If a model has low confidence or missing inputs, the output should show that uncertainty. A reviewer should not be given a confident-sounding explanation that hides weak evidence. Explanations help people understand the decision, but they do not prove that the decision is correct [1][3][43][48].

11.2 Audit-record design

A minimal audit record can contain an event ID, time, device reference, data category, purpose, receiver, policy version, model version where used, risk score, reason codes, action, completion state, and reviewer status. It should avoid storing the full personal payload unless that is necessary and permitted. The record must be protected from unauthorised access and change. It should have a clear retention period and access policy. Otherwise, the audit system could become a new source of privacy risk.

Each record should link to the version of the rule and model used at the time of the decision. Policies and models will change, so a later reviewer must be able to reconstruct the older state. Hashes or signatures can help detect modifications, but they do not prove that the original event data were true. Audit design needs both technical protection and good operational practice, including access reviews, time synchronisation, backup, and deletion of records after the retention period ends.

11.3 Human review and challenge

The review queue should explain why an event needs a person. Reasons can include a high risk score, conflicting metadata, a rule exception, low model confidence, or an action that may significantly affect a person. The reviewer should see only the information needed to make the decision. Access to particularly sensitive records should require the right role and a recorded reason. The interface should permit the reviewer to ask for more information, approve a defined action, reject the request, or return the policy for correction.

Human review must not be only a formal step that automatically accepts the AI suggestion. Reviewers need enough time, authority, and information to disagree. The organisation should measure how often reviewers override the model and why. Frequent overrides may show model drift, unclear policy, poor metadata, or an unreasonable threshold. The reviewer's decision should be recorded separately from the original automated decision so that later analysis can identify where the system helped and where it failed.

11.4 Transparency reporting

A compliance dashboard should display the state of the system, not only a large green indicator. Useful items include the number of decisions made, the number blocked by hard rules, unresolved high-risk events, missing metadata, policy failures, model drift warnings, audit-record completeness, and the age of the latest review. These counts should include a time period and definition. A graph without its denominator or time range can give a misleading impression of improvement.

Reports should distinguish measured facts from model estimates. For example, "120 policy failures were recorded this month" is a measured count if the log is complete. "The system prevented 120 privacy breaches" is a stronger claim that requires evidence that the actions would otherwise have caused a breach. The dashboard should not turn blocked requests into a claim of harm prevented without explaining the basis. Transparent reporting supports trust because it shows the boundary between what the system knows and what it estimates.

11.5 Protecting the audit system itself

An audit log should be treated as sensitive information. It may reveal when devices are used, which receivers request data, which policies are triggered, and which people review exceptions. Access should be limited by role, and every read or export of the log should be controlled where appropriate. The log should have its own retention period. Backups and replicas should follow the same rules. It should not be possible for an ordinary operator to edit a decision record without leaving evidence of the change.

Integrity controls can include append-only storage, digital signatures, or tamper-evident hashes. These controls can reveal some changes, but they do not prove that the original source event was truthful. The system still needs authenticated devices, secure time, and reliable software. The audit design should state the threats it protects against and the threats that remain. A strong statement is narrower and more useful than saying that the logs are “immutable” without explaining the implementation.

11.6 Reporting to different audiences

A technical operator may need a device ID, error code, and suggested repair. A privacy officer may need purpose, receiver, retention, and legal mapping. An auditor may need the policy version, decision chronology, and evidence completeness. A person whose data are involved may need a plain explanation of the action and a way to ask a question where the service and law allow it. The same system can provide different views without giving every user access to all underlying data.

Transparency reports should state the period covered, the meaning of each metric, and the size of the population. They should distinguish counts, estimates, and outcomes. For example, the number of blocked requests is a count; the number of breaches prevented is usually an estimate unless there is evidence for the counterfactual. Clear language prevents a dashboard from overstating what the system has achieved. The organisation should also record when a report was generated and which data were available at that time.

11.7 Minimum audit schema and integrity checks

A useful audit record should answer six questions: what event was checked, when it was checked, which policy version applied, whether an AI model was used, what action occurred, and why the action was selected. A suitable record may include event ID, device reference, policy version, model version, reason codes, action request, action result, reviewer status, and the time of completion. It should avoid unnecessary personal content. A central record can refer to protected source data when access is justified, rather than copying that source data into the log.

Audit integrity can be supported by restricted write access, append-only storage, signed records, hashes, and monitoring for gaps. These controls can help reveal changes, but they do not guarantee that a compromised system records every event correctly. The organisation should compare records across system boundaries and alert on missing sequence numbers or unexpected gaps. It should also protect audit access because logs can reveal usage patterns, medical events, and internal operations. The audit store requires its own data purpose, retention period, access policy, and recovery plan.

11.8 Testing the quality of an explanation

An explanation should be judged by whether a person can understand and check it. A reviewer should be able to identify the rule that failed, the input that triggered the risk alert, and the action that followed. An explanation should distinguish a strict rule from an AI recommendation. It should not imply that a model has proved a violation when it has only found an unusual pattern. It should also say when required information was missing or the model could not assess the event reliably.

Explanation tests can use a set of labelled decisions. Reviewers can be asked to identify the rule, expected action, and reason using only the explanation shown by the system. The test can measure correctness, time required, and agreement between reviewers. It should include intentionally wrong model scores and missing data to see whether the interface reports uncertainty honestly. Explanation methods such as local feature explanations or feature attribution can be helpful, but they are not proof of legal compliance or model correctness [42][50].

12. Governance, Regulatory Mapping, and Data Sovereignty Layer

12.1 Legal context must be explicit

Privacy law depends on the jurisdiction, type of data, organisation role, and purpose of processing. The EU General Data Protection Regulation (GDPR) includes principles such as purpose limitation, data minimisation, storage limitation, and data protection by design [44]. These principles can be supported by technical controls, but the software cannot decide every legal question. The organisation must determine which obligations apply and how they apply to its actual processing activity.

India's Digital Personal Data Protection Act, 2023 and the Digital Personal Data Protection Rules, 2025 establish a framework for digital personal data. The Rules include phased commencement: some provisions commenced on publication, while others are scheduled to commence later [45]. A compliance system should therefore keep effective dates and transition periods in the mapping. Before live use, an organisation must check the current official text, commencement notification, applicable rules, and any later amendments. A static table in a student report is not a substitute for legal review.

12.2 Mapping requirements to controls

A mapping record should connect a requirement to a practical control and to evidence. For example, a retention requirement may map to a stored retention class, a timer, a deletion process, and a record of deletion completion. A requirement to protect access may map to role-based permissions, authentication, and an access review. A requirement to explain a decision may map to reason codes and an audit record. The mapping should state the source, the effective date, the system owner, and the next review date.

The mapping should not claim that one system control proves full legal compliance. Encryption may support security but does not prove that the purpose is lawful. A consent record may support a workflow but may not be sufficient for every legal basis. The system should record the facts that were checked and allow a reviewer to verify whether the control is suitable for the context. Table 7 gives example mappings as an engineering aid; it is not legal advice or a complete list of obligations.

Table 7. Regulatory Mapping Matrix.

Source / context	Possible system control	Important limitation
GDPR where applicable [44]	Purpose, minimisation, storage, design controls.	Role, legal basis, and processing context must be checked.
India DPDP framework [45]	Applicable notice, consent, security, and rights workflows.	Check effective dates and applicable provisions.
HIPAA Privacy Rule [46]	Use and disclosure controls for covered entities.	Does not apply to every health-related app.
NIST Privacy Framework [41]	Risk mapping, governance, and measurable outcomes.	Voluntary framework, not a law.
NIST AI RMF [42]	Govern, map, measure, and manage AI risk.	Use with sector and organisation requirements.
IoT device baseline [43]	Device identity, configuration, protection, and update controls.	Device capabilities must be assessed in context.

12.3 Data sovereignty and cross-border transfer

Data sovereignty concerns where data are stored, processed, accessed, and controlled. A cloud service may offer several regions, but choosing a region does not automatically make a transfer lawful. The organisation must consider the relevant law, contracts, data type, recipient, and permitted transfer conditions. The policy can list approved locations and receivers, but the list must be maintained and reviewed when a vendor, service, or purpose changes.

A cross-border rule should be part of the same policy pipeline as other mandatory rules. If the destination is not permitted, the system should block or hold the request according to the approved procedure. It should not allow the request because the AI score is low. The record should state which destination was requested, which policy version was checked, and what action followed. This creates evidence of the system's behaviour without keeping unnecessary copies of the data itself.

12.4 Sector-specific rules

A hospital, city authority, factory, and consumer application may have different duties. HIPAA in the United States applies to defined covered entities and business associates; it does not automatically cover every health-related app [46]. Similarly, an IoT vendor may not have the same role as an organisation deciding why the data are used. The policy system must record the role and the service relationship before it applies a sector-specific rule.

A regulatory catalogue should be curated by people who understand the law and the business process. AI may help find a new document or identify a change, but a person must confirm the effective meaning and approve the revised control. Each catalogue entry should include a source link, summary, applicability conditions, control mapping, test cases, and review date. The legal review process itself needs an audit trail so that later readers can see why the organisation chose a certain rule.

12.5 Keeping the regulatory catalogue current

A regulatory catalogue should have a named owner and a scheduled review. Changes can come from new legislation, commencement notifications, regulator guidance, contracts, or a change in the organisation's role. An AI tool may help detect a new source or compare text, but it should not automatically turn an unreviewed interpretation into a production rule. The catalogue should store the official source, summary, effective date, applicability conditions, mapped controls, test cases, and approval record.

India's DPDP Rules were published in November 2025 and specify staged commencement for different rules. This means that an implementation should not assume every provision starts on the same day [45]. A live compliance project must check the latest official notification and legal interpretation. The report's regulatory table is only an example of how to store a mapping; it does not replace a current legal analysis or account for later amendments and judicial interpretation.

12.6 Cross-border control and accountability

Cross-border control requires more context than a cloud region name. The organisation should know the sender, recipient, purpose, data category, storage location, access location, contract terms, and applicable transfer conditions. These details can be represented as policy attributes, but a legal reviewer must confirm which conditions are sufficient. A change in cloud provider or subcontractor should trigger a review because a new party may gain access to the data.

The system should retain a compact record of the destination check and the policy version used. If the destination is not allowed, the action should be blocked or held. If an exception is approved, the exception should name the approving authority, basis, scope, and end date. This allows a later reviewer to understand why a transfer was made. The process should also prevent old destination approvals from being reused after a supplier or legal arrangement changes.

12.7 Legal mapping as a maintained work item

A legal mapping should never be treated as a one-time spreadsheet created at the start of a project. New devices, new business purposes, a new destination country, or an amendment to a rule can change what the organisation must do. Each mapping entry should name the legal source, the provision or requirement, the context in which it applies, the technical control, the evidence needed, the approving role, and the date of the next review. The rule should also record what it does not cover so that a reviewer does not assume the mapping is complete.

Legal texts often contain conditions, exceptions, and duties that depend on the type of organisation and processing. For example, a healthcare rule may apply to defined entities and relationships rather than to every app that handles health-related data. A privacy system should therefore store the jurisdiction and organisational role for a request and send uncertain cases to a qualified reviewer. The software can help track dates and identify affected rules, but a human must confirm the meaning and applicability of a legal requirement.

12.8 Cross-border transfer and data location controls

A data location policy should list which destinations are approved, what transfer condition is required, who approved it, and when the approval must be reviewed. The location of a cloud data centre alone does not answer every question about remote access, support staff, replicas, backups, or service providers. The review should consider the full service path and the contractual arrangement, not only the region selected in the cloud console.

The system can enforce the technical part by checking the receiver identity, service region, route, and permitted data class. It can hold the request when those details do not match the rule. The audit record should state which location rule was checked and what happened. It should not claim that the transfer is legally approved simply because the technical destination matched. Legal review, vendor review, contract terms, and organisational accountability remain separate controls. This distinction keeps technical automation useful without giving it authority that it cannot safely exercise.

13. Security and Threat Intelligence Layer

13.1 Threat model

The threat model should identify what must be protected, who might attack it, and what access the attacker may have. Assets include raw sensor data, device credentials, policies, models, audit records, and the service that decides what action to take. Threat actors may include outside attackers, malicious insiders, compromised vendors, and participants attempting to poison a learning process. Different threats need different controls, so the report should avoid using the word “secure” as though it means every risk has been addressed [15][30][32].

Typical attacks include stolen credentials, unauthorised commands, insecure updates, data interception, service interruption, model evasion, and exposure of logs. Some attacks affect confidentiality, some affect integrity, and others affect availability. A privacy breach can also occur without an external attacker, such as when an authorised service uses a record for a purpose that was not approved. Security monitoring must therefore link technical alerts to the data context and policy outcome.

13.2 Device and network protection

Devices should use unique identities, supported firmware, secure update paths, restricted ports, and a way to revoke credentials. Gateways should be hardened and should expose only required services. Network traffic should be authenticated and protected, and the system should track failed authentication attempts. The device inventory should record software and update status. Unsupported devices should be isolated or replaced when their risk cannot be managed. These requirements align with IoT cybersecurity guidance such as the NIST baseline [43].

A gateway is a high-value target because it applies local rules to many devices. It should protect its configuration, verify policy signatures, monitor unexpected changes, and keep secure logs. The organisation should test how the gateway recovers after failure and how it handles old or invalid policy packages. It should also test whether a user with limited access can change a policy, disable logging, or route data around the gateway.

13.3 AI-related threats

AI systems may face evasion, poisoning, model extraction, and membership-inference risks. Evasion changes the input so that a model misses an event. Poisoning adds misleading training examples or updates. Model extraction tries to learn the model's behaviour, while membership inference tries to discover whether a record was used for training [9][47][48]. The exact risk depends on what model is exposed, what outputs are available, and who can send inputs or updates.

Mitigations include strict input validation, signed model packages, controlled training data, robust testing, rate limits, and separate approval for model updates. Federated learning needs checks on participants and aggregation, because local training does not guarantee that each update is benign. Model changes should have a version and test record. The organisation should also monitor for changes in the distribution of inputs and re-evaluate the system when a new device type or usage pattern appears.

Table 8. Threat Categories and Security Measures.

Threat	Possible effect	Example mitigation
Stolen credentials	Unapproved access	Unique identities, revocation, least privilege.
Old firmware	Known weaknesses or hidden behaviour	Inventory, supported updates, integrity checks.
Model evasion	Risk events may be missed	Input validation and adversarial testing.
Model poisoning	Risk scores can be manipulated	Participant checks and controlled model updates.
Insider misuse	Purpose mismatch or data export	Separate duties and audit access.
Audit-log exposure	Logs reveal behaviour	Minimal logs, access control, retention limits.
Gateway compromise	Rules can be bypassed	Hardening, signed policies, monitoring, recovery.

13.4 Incident response

The incident process should define how an alert is assessed, contained, investigated, and closed. An incident record should include time, affected system, event IDs, data categories, containment action, notifications, recovery steps, and lessons learned. It should not copy all affected personal data into the incident tracker unless that is necessary and permitted. The team should be able to identify which policies were active and which devices or services were involved at the time.

After an incident, the organisation should test whether the control worked and whether the evidence was sufficient. If the event happened because metadata were missing, the fix may be an onboarding change rather than a new AI model. If the event occurred because a policy was old, the fix may be the policy update process. This makes incident learning specific. Table 8 maps common threats to example controls, while the actual response plan must match the organisation's risk and legal duties.

13.5 Supply-chain and vendor risk

An IoT device depends on firmware, libraries, cloud services, and vendor support. A vulnerability in a dependency can affect the gateway or the system that applies privacy rules. The organisation should keep a device and software inventory, track support dates, and review vendor updates. It should also know who can access the data through the vendor's service. Contracts and technical controls should agree about data handling, incident reporting, and deletion of data when service ends [15][30][36].

Supply-chain risk should be included in the governance record. For each important component, the owner can record the supplier, version, update channel, access rights, known support limits, and recovery plan. A change in vendor or version should trigger regression tests for identity, policy enforcement, logs, and performance. A system that is correct only with one unmaintained library may not remain safe. Future implementation work should test how quickly the organisation can update or isolate a compromised component.

13.6 Privacy-aware incident evidence

Incident investigation needs enough evidence to determine what happened, but unrestricted logging can create new privacy risks. The team should define which details are necessary to investigate authentication failures, unusual exports, policy bypass, or data exposure. Where possible, the log should refer to an event ID rather than copy the full data payload. Only authorised investigators should be able to resolve the ID to the original record, and every such access should be controlled.

An incident exercise should check that the team can find the affected events, identify the active policy version, stop further transfers, and restore service. It should also check that deleted or transformed data are not accidentally copied back during recovery. The lesson should be written as a concrete system or process change. For example, an incident caused by an old device certificate may call for better revocation checks rather than a new AI classifier. This keeps incident response focused on the real cause.

13.7 Threats to the AI and policy path

An attacker may try to fool the model by changing the input pattern, poison training data, steal a model file, or send a fake model update. A compromised administrator account may also change a policy or lower a threshold. These threats affect the decision system itself, not just the IoT data. The system should therefore protect model packages and policy files with controlled access, signed versions, integrity checks, and a history of changes. Training data and model updates need their own provenance records.

Threat testing should include a model that returns a timeout, a malformed score, a value outside its expected range, and an unrecognised model version. The policy engine should not accept a result from an unknown model as if it were normal. If the AI service is unavailable, mandatory rules must still run. A strict policy failure must remain a denial. These conditions should be tested with the same care as ordinary user-facing features because they define how the system behaves under attack or failure.

13.8 Measuring the incident response

Incident response should be measured from detection to containment, not just from the first alert to the first log entry. The workflow may include alert generation, triage, owner assignment, containment, assessment of affected data, notifications where required, recovery, and a post-incident review. Each step should have an owner and a time stamp. The measures can show whether the system provides useful evidence quickly enough for the response team.

The system should also distinguish confirmed privacy violations from security alerts that remain uncertain. A suspicious outbound connection may be a security incident, but the team still needs evidence to decide whether personal data were exposed or used for an unapproved purpose. The report should not treat every anomaly alert as a confirmed privacy incident. Instead, the audit workflow should record the initial alert, later findings, and final classification. This helps maintain honest incident statistics and allows the organisation to improve its detection rules over time.

14. Experimental Setup

14.1 Purpose of the evaluation

No live system was implemented or benchmarked for this report. Section 14 defines an experiment that can be completed later. Its purpose is to test whether the proposed workflow follows the policy, detects labelled risk events, gives clear explanations, and runs within the resource limits of the target devices. The evaluation should be written before the runs begin so the expected results are not changed after seeing the output.

A useful testbed includes a set of IoT event producers, a gateway, an edge privacy service, an AI risk engine, a policy engine, a small data store, and an audit viewer. The exact hardware may vary, but the report should record the CPU, memory, operating system, network, framework versions, and policy version. If the implementation uses a cloud service, the service region and relevant configuration should be stated. Figure 1 represents the logical architecture; the experimental setup should document the actual implementation separately.

14.2 Event set and labels

The evaluation needs two kinds of events. The first kind can be technical events, such as unusual network traffic or repeated authentication failures. The second kind must directly test privacy rules, such as a missing purpose, an unapproved receiver, an expired retention period, an invalid consent state where relevant, or a revoked device. Security datasets do not automatically contain labels for legal privacy compliance. A separate set of policy events is required to test the final compliance decision [1][3][29].

Each test event should have a unique ID, input fields, expected policy outcome, expected action, and a short explanation of the label. Events should include normal, violating, uncertain, and boundary cases. Synthetic events are useful for checking known conditions, but they do not show how frequent those conditions are in a real organisation. Public datasets should be used according to their licence and should be documented by name, version, source, and preprocessing steps.

14.3 Baselines and comparison

The proposed workflow should be compared with at least three alternatives: manual review, rules-only enforcement, and a central AI model with policy checking. All methods should receive the same labelled events and should use the same definitions of success and failure. If different input fields or thresholds are used, the report should explain why. A comparison is not fair when one method is given more information or is tested on easier events than another method.

The test should use a fixed train, validation, and test split for learning methods. Where events from the same device are strongly related, the split should avoid placing near-duplicates across the training and test sets. Otherwise, a model can appear accurate because it has seen very similar examples before. The code, configuration, random seed, event IDs, and policy versions should be saved. These details allow another person to repeat the experiment and understand why a result changed.

Table 9. Proposed Experimental Configuration.

Item	Proposed setup	Details to record
Event set	Public technical events plus labelled policy cases	Source, licence, labels, event counts.
Baselines	Manual, rules-only, central AI, proposed design	Same cases and expected answers.
Hardware	Documented edge gateway and server	CPU, memory, OS, network.
Model	Chosen classifier or anomaly detector	Features, version, seed, model card.
Policy	Purpose, receiver, retention, jurisdiction	Version, effective date, reviewer.
Runs	Repeated runs on fixed configuration	Commands, logs, random seeds.
Release	Code, allowed test cases, result scripts	Commit identifier and access location.

14.4 Failure and recovery tests

The evaluation should test more than a normal run. It should include an expired policy, invalid policy signature, unavailable policy server, missing metadata, clock difference, duplicated event ID, audit-store failure, and a revoked device. It should also test how the system behaves when a model is unavailable or returns an uncertain result. For each test, the expected fallback must be stated in advance. High-risk events should not be silently allowed because one supporting service is down.

A recovery test should verify that the system can resume safely. After a network outage, the edge node should synchronise records, detect missing entries, and retrieve valid policy updates. If the audit service was unavailable, the system should either queue a protected local record or block the action according to the design. The test should measure the delay to recovery and the number of records lost or duplicated. These are important reliability outcomes for distributed IoT environments.

14.5 Reproducible software and configuration

A repeatable experiment needs more than source code. It should record the library versions, operating system, model settings, policy files, event set, preprocessing, random seeds, and the exact command used for each run. The report should say whether the system was run on physical hardware, a virtual machine, or a simulated environment. Network conditions and the way latency was measured should also be documented. These details allow a reader to decide whether another result is comparable.

The result files should be generated by a script where possible, rather than copied manually into a paper. The script should use fixed names for metrics and keep the raw run summary. If the program is updated, the commit identifier should be recorded. Test labels and expected actions should be stored separately from the model output so that an accidental change to the prediction does not also change the reference answer. This helps protect the evaluation against silent or unintended bias.

14.6 Ethics, access, and dataset limits

If real personal data are used, the project must have permission to process them and a documented plan for secure storage, access, retention, and deletion. The team should avoid using health, location, or other sensitive records just because they are easy to obtain. Where possible, use synthetic or de-identified data for early tests. De-identification should be reviewed for linkability, particularly when high-detail time-series data are involved. Data should be shared only when the licence and consent allow it.

Public datasets can contain bias or labels that do not match the intended task. The experimental plan should record the source and known limitations. A dataset for cyberattack detection should not be treated as a dataset of privacy-law violations. If labels are created by the research team, the label guide should be published and difficult cases should be reviewed. This makes it clear what the model was actually trained and tested to identify.

14.7 Full experimental procedure

The proposed evaluation should begin with a written test protocol. The protocol should define the research questions, event categories, data sources, preprocessing steps, model versions, policy rules, test hardware, baselines, metrics, and pass conditions. The same saved event set should be used across the compared methods. Any randomised method should record its seed, and repeated runs should be saved separately. If a reviewer changes an event label, the change should be documented with the reason and the reviewer who approved it.

The experiment should be run in stages. First, unit tests verify individual rules and transformations. Second, integration tests confirm that gateway, risk engine, policy engine, and audit store exchange the correct fields. Third, scenario tests run normal, violating, and uncertain events through the full pipeline. Fourth, load and outage tests measure response under pressure and during failure. Finally, error analysis examines missed violations, false alerts, incomplete audit records, and differences across device categories. A system should not move to deployment only because the normal path passes.

14.8 Data quality, leakage, and reproducibility safeguards

Labels must be defined before the model is evaluated. A privacy violation label should describe the rule that was broken and the evidence required to assign the label. Attack labels from a network dataset should not automatically be treated as privacy violation labels. Unclear cases should have a separate category rather than being forced into compliant or violating. When possible, two reviewers should label a sample independently and resolve disagreements using a written rule. This gives the test a clearer ground truth.

The split between training and testing should prevent near-duplicate or related events from leaking across sets. If records from the same device or time window appear in both training and test data, the result can look better than performance on new devices or later events. Group-based or time-based splits may be more appropriate, depending on the research question. The report should state the split rule and keep the test set fixed while model settings are tuned only on training and validation data. All preprocessing steps should be fitted on the training data when needed, then applied unchanged to the held-out data.

15. Evaluation Metrics

15.1 Detection measures

The system should be evaluated with a confusion matrix. A true positive (TP) is a labelled violation correctly flagged. A false positive (FP) is a normal event wrongly flagged. A false negative (FN) is a violation that the system missed. A true negative (TN) is a normal event correctly left unflagged. Precision answers "when the system raises an alert, how often is it right?" Recall answers "of all real violations, how many did it find?" These questions matter because a system can look good on one measure but fail on the other.

F1 combines precision and recall, while the false-positive rate measures how often normal events are flagged. Accuracy may be misleading when violations are rare. If only one event in a thousand is a violation, a system that always says "normal" could have high accuracy but be useless for finding violations. Reports should include the confusion matrix, class balance, and performance by event type. Thresholds should be chosen based on the harms of missed violations and the cost of reviewing false alerts.

$$Precision = \frac{TP}{TP + FP} \tag{4}$$

Equation (4) defines precision as the proportion of flagged events that are labelled violations. TP means true positives and FP means false positives; labels should be checked by a reviewer.

$$Recall = \frac{TP}{TP + FN} \tag{5}$$

Equation (5) defines recall as the share of labelled violations found by the system. FN means false negatives.

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{6}$$

Equation (6) combines precision and recall in the F1 score. If both terms are zero, define F1 as zero and state the convention.

$$FPR = \frac{FP}{FP + TN} \tag{7}$$

Equation (7) defines the false-positive rate (FPR), which describes how often a normal event is flagged by mistake. TN means true negatives.

15.2 Runtime and resource measures

Table 10. Evaluation Metrics and Definitions.

Metric	Definition / Formula	What it shows
Precision	TP / (TP + FP)	Share of alerts that are correct.
Recall	TP / (TP + FN)	Share of labelled violations found.
F1 score	2PR / (P + R)	Balance of precision and recall.
False-positive rate	FP / (FP + TN)	How often a normal event is flagged.
Latency	t_action - t_arrival (ms)	Time to apply the final action.
Throughput	N_events / T_total (events/s)	Events handled per second.
Compliance coverage	Complete-context decisions / all decisions	Share of decisions with required context.
Audit completeness	Complete audit records / all decisions	Share of decisions that can be reconstructed.
Resource use	CPU, memory, network, energy	Deployment cost on target hardware.

Latency is the time between an event arriving and the final action being applied. The report should measure median and high-percentile latency, not only the mean. Throughput is the number of events handled per second under a stated load. Resource use includes CPU, memory, network traffic, storage, and energy where the target hardware supports those measures. A result from a powerful desktop cannot be assumed to represent a low-power gateway. Table 10 lists the main metrics and what each one means.

Tests should use a defined event rate and should repeat under low, medium, and high load. The report should record the hardware, software, event size, and whether the result includes network time and audit-storage time. If the system uses the cloud for part of a request, the delay should include that round trip. A system may detect events accurately but still be unsuitable if its delay is too high for the service. The report should state these trade-offs plainly.

$$Latency = t_{action} - t_{arrival} \tag{8}$$

Equation (8) defines latency as the time from event arrival to the final action. State the load and report median and high-percentile values.

$$Throughput = \frac{N_{events}}{T_{total}} \tag{9}$$

Equation (9) defines throughput using N_events, the number of processed events, and T_total, the total test time in seconds. Use the same method across baselines.

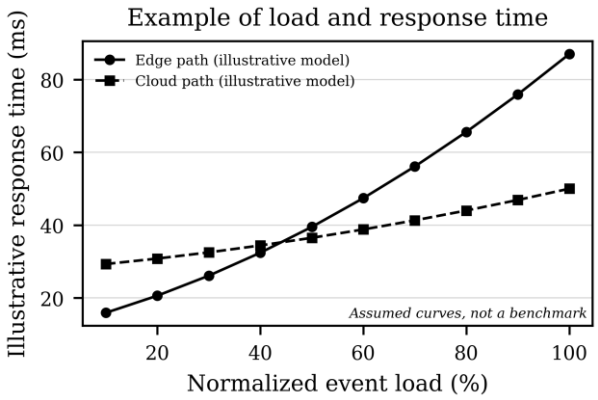


Fig. 6. Illustrative Response Time by Event Load.

The curves use assumed functions to explain a possible edge/cloud trade-off. They are not benchmark results.

15.3 Privacy and governance measures

The privacy evaluation should state what protection method was used and what it can guarantee. If data minimisation is used, measure which fields were removed and whether the remaining data can still complete the task. If differential privacy is used, report ε, δ, the protected unit, and the accumulated privacy budget. If federated learning is used, report what data remain local, what updates are sent, and whether secure aggregation or other protections are enabled [8][9][10][11][47][48].

Governance metrics include the fraction of decisions with complete evidence, the fraction of policies with an owner and current review date, the number of decisions that require human review, and the time needed to reconstruct a decision. These metrics should have clear definitions. For example, evidence completeness may be defined as the number of decisions containing all mandatory audit fields divided by the total number of decisions. A count of completed fields should not be confused with a claim that the process was legally correct.

$$Compliance\ coverage = \frac{N_{decisions\ with\ complete\ context}}{N_{decisions}} \quad (10)$$

Equation (10) defines compliance coverage as the share of decisions with complete context. Complete context means the mandatory metadata and policy fields are available; this measure does not alone prove that the legal interpretation is correct.

$$Audit\ completeness = \frac{N_{complete\ records}}{N_{all\ decisions}} \quad (11)$$

Equation (11) defines audit completeness as the share of decisions with complete audit records. The organisation should define required record fields and report missing entries separately.

15.4 Reporting uncertainty and fairness

A research result should state the number of tests, the way data were split, and the uncertainty around the measured value. One run is not enough to show that a method is stable. Results should be repeated when the method is stochastic or when system load changes. If the sample is small, that fact should be reported. If the labels were uncertain or based on reviewer judgement, the report should explain the label process and disagreements.

Performance should also be inspected by device type and data category. A model may work well for environmental sensors but poorly for a health wearable with irregular measurements. A single overall score can hide that problem. The team should check whether alerts or false negatives are concentrated in one device group, one language, one location, or one kind of data. Such analysis is useful even when the sample is too small to support strong statistical conclusions. It should be presented as a diagnostic, not as proof of fairness.

15.5 Choose metrics that fit the harm

No single metric is enough to assess the system. High recall may be important when a missed violation would cause serious harm, but extremely low precision can overwhelm reviewers. Low latency matters for a safety-critical action, while a monthly audit report may tolerate longer processing. Audit completeness matters when decisions need to be reconstructed. Privacy measures depend on the technique. The evaluation should state which metrics are primary and why, rather than choosing only the values that look strongest after the test.

The team should set acceptance thresholds before running the test. For example, it may require that every known unapproved receiver be blocked, that the audit record contain required fields, and that latency remain below a stated value under a stated load. These are illustrative types of acceptance conditions, not universal values. A real deployment should set the actual limits with operators and risk owners. The result should state which thresholds passed and which did not.

15.6 Statistical reporting and error analysis

When the test includes a learning model, the team should repeat runs where randomness affects the result and report the spread across runs. It should include the number of events, class balance, data split, and how uncertain labels were handled. Results should be inspected per class and per device type where the sample size allows. A single average may hide a weakness in one device class or one sensitive data category. Those weaknesses should be discussed rather than hidden in a summary score.

Error analysis should inspect false positives and false negatives individually. A false positive may come from a legitimate workload change that the model has not seen before. A false negative may come from missing metadata or a new kind of data transfer. The team should classify the causes, not only count them. This makes the analysis useful for fixing the system. It also helps distinguish a model limitation from a problem in device identity, event schema, rule logic, or operational procedure.

15.7 Confidence intervals and repeated tests

A single metric value does not show how stable the result is. If a model is tested on a small number of events, one or two changed labels can have a large effect. Results should include the number of events in each class and, where suitable, confidence intervals or repeated-run variation. The method for calculating the interval should be stated. The same principle applies to response time: report the median and high-percentile values, and describe the load and hardware under which the timing was measured.

Comparison should use paired events so that each method faces the same cases. The report can then count cases where both methods succeed, where only one succeeds, and where both fail. A claimed improvement should include its magnitude and the uncertainty around it. If the difference is small or unstable, that should be reported honestly. Statistical significance by itself is not enough; the result must also be meaningful for the operational task. A reduction in false alerts may be valuable, but not if recall drops below an agreed safe level.

15.8 Class imbalance and cost-sensitive errors

Privacy violation events may be much less common than normal events. In such a dataset, a model could label nearly every event as normal and still achieve high accuracy. This is why precision, recall, F1, false-positive rate, and the full confusion matrix are needed. Precision-recall curves may be more informative than accuracy alone when positive events are rare. The report should also give the number of positive cases because a high percentage based on only a few cases may be unreliable.

The evaluation should define the cost of each error for the target application. A false positive consumes review time and can interrupt useful work. A false negative may allow sensitive data to be transferred incorrectly. The cost is not always equal across cases. In healthcare or other high-impact settings, missed violations may require a more conservative threshold and stronger review. These decisions should be approved by the organisation and reported as design choices, not hidden inside the model. A reader should be able to see how threshold choice changes the balance between missed violations and unnecessary alerts.

16. Results and Findings

16.1 Status of results

This report does not claim results from a completed deployment or experiment. The quantitative figures are either calculated from the proposed risk equation or generated from explicitly stated illustrative assumptions. They are included to show the mathematical behaviour of the proposed method and to demonstrate how a later paper could report its measurements. They must not be described as proof that the framework improves compliance, reduces cost, or increases privacy in practice.

The design analysis still produces useful findings. First, mandatory rules and AI risk ranking have different roles. Second, metadata quality is essential because the system cannot check purpose or receiver if these fields are missing. Third, the audit record must be designed at the same time as the action logic. Fourth, edge processing can reduce raw-data movement but adds security and policy-management work. These findings arise from the structure and assumptions of the design, not from a measured benchmark.

16.2 Mathematical examples

The synthetic risk values in Table 11 show how a formula can rank different illustrative events. A public sensor event with low impact, likelihood, and uncertainty receives a low score. A sensitive event sent to an unapproved receiver receives a higher example score. A revoked device may receive a high risk score, but the important result is that the identity rule independently rejects the request. This example shows why the score should support review while mandatory policy rules determine required actions.

Figure 4 changes uncertainty while holding impact and likelihood fixed. The curves rise in a predictable way because the equation assigns a positive weight to uncertainty. Figure 3 shows how the three weighted components add to one example score. Figure 5 shows a conceptual privacy–utility trade-off. Figure 6 shows assumed response-time curves under increasing load. None of these graphs represents observed system performance. A future test should replace them with plots generated from saved run data and should include axes, units, sample size, and test configuration.

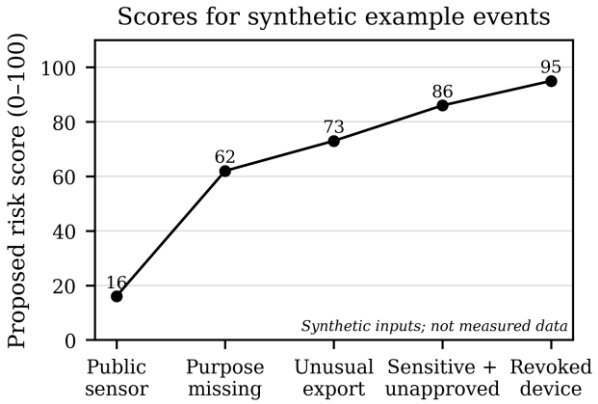


Fig. 7. Risk Scores for Synthetic Example Events.

The values are generated from the stated example inputs. They are not observed events or experimental outcomes.

Table 11. Illustrative Test Cases - Not Measured Results.

Synthetic test case	I, L, U	Formula score	Expected action
Public sensor	0.20, 0.15, 0.10	16	Allow if rules pass.
Purpose missing	0.60, 0.45, 0.90	62	Hold for missing context.
Unusual export	0.75, 0.80, 0.60	73	Review or limit according to policy.
Sensitive + unapproved	0.90, 0.85, 0.80	86	Block because receiver rule fails.
Revoked device	0.95, 0.95, 0.95	95	Reject due to identity rule.

16.3 What must be measured next

The first required measurement is policy correctness: does the system apply the expected action for each event? This should be measured separately from the model's classification accuracy. The second measurement is false-alert workload: how many normal events are sent to people for review? The third is runtime performance, including edge latency and resource use. The fourth is audit quality, including whether reviewers can reconstruct a decision. The fifth is privacy effect, such as how much data movement is reduced and what protection parameters are used.

The evaluation should publish failures as well as successes. If an event is missed, the report should show whether the cause was a poor label, missing metadata, a model error, or an incorrect policy. If the system blocks a normal event, the report should show which condition triggered the block and whether it could be fixed safely. This helps future researchers understand the limits of the design. A result table should never be filled with estimated accuracy or latency merely to make the paper look complete.

16.4 Interpreting outcomes cautiously

A single score cannot represent all privacy outcomes. A system may reduce data transfer but raise processing delay. It may improve recall but create a heavy review queue. It may keep raw data local but increase the risk of local device compromise. A useful report shows these measures together and explains which trade-offs matter most for the target service. Results should be interpreted in the context of the threat model and the data category.

The design should also be compared with what an organisation already does. If a rules-only process already blocks every strict violation with low delay, adding AI may have limited value for those cases. AI may be more useful for unusual activity that is difficult to list in advance. That value must be shown by test results, not assumed. Future work should therefore measure the extra cases found by AI, the false alerts it adds, and whether reviewers can act on its explanations.

16.5 Threats to validity

The design analysis is limited by the lack of an implemented testbed. The illustrative scores show that the equation behaves in a predictable way, but they do not show whether the inputs can be measured reliably. A future model may use different features or weights. Likewise, the conceptual latency curves do not show the speed of a real edge device. These figures are included to make the method transparent and to prevent hidden assumptions, not to replace experiments.

Other threats include label error, dataset shift, different device types, and legal context. A test set may over-represent easy cases or omit rare but harmful events. The event labels may reflect one organisation's interpretation rather than a universal legal rule. The evaluation should state these limits and avoid broad claims beyond the tested environment. Replication across devices, organisations, and domains would provide stronger evidence than a single run.

16.6 Format for reporting future measured results

A measured result table should include the method, number of events, data split, metric, result, uncertainty, and test configuration. If possible, it should provide results for each device class and risk category. A figure caption should state the sample size and whether the values are measured, simulated, or synthetic. The paper should not mix real measurements and illustrative estimates on the same axis without a clear legend. Raw summaries should be retained so that the chart can be regenerated.

If the proposed model does not outperform rules-only enforcement, that is still a useful result. It may show that AI is not required for the tested events or that the model needs better data. If it improves recall but raises false alerts, the report should quantify the trade-off and state whether the review team can handle it. A research paper becomes more credible when it reports negative or mixed findings instead of presenting every graph as evidence of success.

16.7 Expected result tables versus observed results

An experimental results table should contain only values produced by a completed run. Before the implementation exists, the report can include an empty results template that names the metric, the comparison method, and the value to be filled later. It must not place synthetic values in a column labelled “measured accuracy” or “measured latency.” The graphs in this report are used to explain the score equation and possible trade-offs. They are not evidence that the proposed system outperforms another method.

When the future test is complete, each result should state its dataset, event count, label process, split method, hardware, software version, policy version, model version, and number of runs. A good report should include unsuccessful tests and known limits. If a result changes when the event threshold changes, the report should show the sensitivity instead of selecting only the best value. This approach makes it possible for another student or reviewer to understand how the result was obtained.

16.8 Error analysis as a source of research findings

Raw metrics tell the reader how often an event was classified correctly, but an error analysis can explain why. Missed violations may come from missing metadata, an unseen data category, a threshold that is too high, a weak feature, or a policy mapping that is wrong. False alerts may come from a new but valid device, seasonal traffic changes, or a rule that does not match the real purpose. The team should review examples of each important error and connect them to a specific system component.

Errors should be grouped before changes are made. If a schema issue causes the problem, the team should fix validation rather than retrain the AI model. If a receiver rule is missing, the policy needs revision. If the model misses a new behaviour, the training data or anomaly method may need attention. If the audit record is incomplete, the logging interface should be repaired. This mapping avoids using AI as a cure for every system problem. After a correction, the failed case should remain in a regression test so that a later update does not recreate the same error.

17. Comparative Analysis

17.1 Comparison of approaches

Manual review offers human context and can handle unusual situations, but it may be slow and hard to apply consistently at high volume. Rule-only automation is clear for well-defined cases and can be tested directly, but it may miss patterns that were not anticipated. Central AI can find patterns across many events, but it may need more data to be collected in one place and can be difficult to explain. Edge plus policy design attempts to combine explicit rules, local checks, AI assistance, and traceable evidence [1][6][9][14].

The proposed design also has costs. It needs policies and models to be maintained. Edge devices must be updated. The organisation must manage a review queue and resolve cases in time. A policy catalogue may be difficult to keep accurate across multiple regions. AI can create a false sense of automation if staff stop checking the quality of its output. Therefore, the framework should be considered a design option whose benefits must be measured against its operating cost and risks.

17.2 Comparison dimensions

The comparison should include explainability, detection of new patterns, response delay, data movement, audit evidence, maintainability, and failure handling. These dimensions describe different parts of the system. One approach can be strong in explainability but weak in anomaly detection. Another can identify unusual behaviour but require too much data movement. Table 12 compares design features in qualitative terms. The ratings are not benchmark results and should not be converted into a numerical performance claim.

A fair benchmark should use the same event set, policy rules, labels, hardware, and time period for each method. The test should separate the effects of the policy engine from the effects of the AI model. If the proposed method performs better, the report should show which events caused the difference and whether the difference is large enough to matter. If it performs worse under some conditions, those cases should be reported and discussed rather than hidden in an average score.

Table 12. Comparative Design Assessment.

Dimension	Manual review	Rules only	Central AI	Proposed hybrid
Known rules	Depends on staff	Strong	Needs policy too	Explicit rule control
New patterns	Human-dependent	Limited	Potential strength	AI flags for review
Data locality	Varies	Possible	May centralise data	Edge checks designed in
Explanations	Can vary	Clear rules	Model explanation needed	Rule and score reasons
Maintenance	Staff procedures	Rule updates	Model updates	Policy, model, and edge updates
Evidence	May be uneven	Repeatable logs possible	Must link to policy	Audit trail built into flow

17.3 When a simpler method is better

AI should not be added merely because it is available. A clear rule is better when the condition is already known and can be tested directly. For example, an expired retention period or a revoked device identity does not require an AI prediction. Using AI for that check could make the result harder to explain without adding value. AI is more appropriate when the event contains complex patterns that fixed rules cannot describe easily and when there is enough high-quality data to test the model.

The system should therefore use the simplest method that satisfies the requirement. A small gateway may need only signed rules and basic anomaly checks. A central service may run a larger model when the task requires it. A high-impact decision may always need human review. This selective use of AI reduces cost and limits the number of components that can fail. It also gives the evaluation a stronger question: what extra benefit does AI provide beyond the rules that were already present?

17.4 Compare total operating cost, not only model quality

The cost of a system includes data movement, processing, key management, policy maintenance, alert review, audit storage, and updates. A large central model may require more infrastructure and data movement. An edge model may reduce transfer but require management across many devices. Rules-only enforcement may be cheaper where all cases are known, but manual rule creation may become expensive as services grow. The proposed hybrid has the benefit of separating hard rules from risk ranking, but it also needs a process to maintain the model and policies.

A cost comparison should use the same target service and account for operations over time. A small prototype may look inexpensive because staff time and security review are not counted. A production system needs monitoring, incident response, version control, backups, and training. The business impact table therefore lists measures rather than claiming that the hybrid approach is automatically cheaper. The correct choice should be based on the service's risk, event volume, hardware, and available governance resources.

17.5 Conditions under which the hybrid approach may help

The hybrid approach may be useful when mandatory rules can be written clearly but unusual behaviours are also important. The rules provide a stable boundary for known requirements, while AI helps focus attention on patterns that are difficult to enumerate. Edge processing can reduce raw-data movement when the data are not needed centrally. Audit records connect the system's action to its rule and model versions. This combination is a design hypothesis that should be tested, not a guaranteed advantage.

The approach may be less useful when the event volume is small, the rules already cover all relevant cases, or the device cannot support the required processing. It may also add too much complexity for a low-risk service. A sensible implementation should start with the simplest acceptable control and add AI only when its contribution can be demonstrated. This avoids adding an opaque component when a clear policy rule would solve the problem more reliably.

17.6 Comparing total cost and operational burden

A fair comparison should include more than model accuracy. Manual review uses staff time and can vary between reviewers. A rules-only system is easier to explain, but it requires careful rule maintenance and may miss patterns that were not anticipated. A central AI service may use more computing power and may require data to be transferred away from the edge. A hybrid design adds policy, model, and edge-management work. The total cost includes engineering, devices, integration, updates, cloud use, audits, and the time spent resolving alerts.

The proposed framework should therefore be compared using both benefit and burden. Useful measures include setup time, policy update time, review queue size, rate of missed violations, false alerts per thousand events, energy use, and audit preparation time. Each organisation should set acceptable thresholds for the measures that matter to it. A system may be a good fit when the data are sensitive and local response is important, but a simple rules engine may be enough for a small sensor application with a few stable conditions. The report should say when its approach is not needed.

17.7 Domain fit and migration cost

Different settings have different needs. A smart-city system may care about location detail, aggregation, and public purpose. A healthcare system may need strict access controls, clear clinical roles, and careful handling of sensitive records. An industrial system may need to protect trade secrets and keep real-time control separate from privacy review. A farm sensor system may have limited connectivity and simple data structures. A shared architecture can provide a common workflow, but data labels, rule definitions, performance targets, and test cases must still match the domain.

The cost of moving from an existing system should be part of the comparison. An organisation may already have a device inventory, an identity service, a data catalogue, or an audit platform. The proposed framework should connect to these systems where safe instead of replacing them without reason. Migration tests should check whether old event formats can be mapped, whether policy rules preserve their intended meaning, and whether historical audit records can still be read. A realistic comparison considers the cost of adoption as well as the design's theoretical strengths.

18. Trustworthiness, Accountability, and Governance Assessment

18.1 Accountability

Accountability means that an organisation can identify who is responsible for a policy, a model, an event, and a final decision. Every policy should have an owner. Every model should have a technical owner and an approved use. Every high-risk action should have a reason and a record. This makes it possible to respond when an event is mishandled. Without clear ownership, teams can pass responsibility between the device vendor, cloud provider, and data user without fixing the underlying problem [17][18][19][41][42].

The roles should be separated where possible. The policy owner decides the intended rules, the model owner tests the model, the security team protects the infrastructure, and the reviewer decides uncertain cases. In a small project, one person may hold several roles, but the tasks should still be documented. A change to a policy or model should have a review record so that future auditors can distinguish an approved change from an unapproved modification.

18.2 Trustworthiness and human oversight

A trustworthy system should behave consistently, explain its actions, protect data, and fail safely. It should also recognise uncertainty and allow people to challenge decisions. The NIST AI Risk Management Framework provides a voluntary way to organise AI risk activities through governance, context mapping, measurement, and management [42]. It can guide a team in deciding who is responsible, which harms matter, and what evidence is needed before the system is used.

Human oversight needs a real purpose. A reviewer must see enough evidence to understand the event and must have authority to stop an action. A button that simply accepts the model's recommendation does not provide meaningful oversight. The organisation should record how often reviewers disagree with a model and the reasons for the disagreement. These records can reveal bias, poor data quality, missing policy rules, or a threshold that is too aggressive.

18.3 Governance maturity

A simple maturity path can begin with informal manual checks, move to documented policies, then to tested automation, measured performance, and continuous improvement. Progress should be based on evidence, not on a dashboard's appearance. A team should not claim a high maturity level if policies have no owners, model performance is not checked, or exceptions are never reviewed. Table 13 lists evidence that can be used to assess accountability, transparency, robustness, privacy, security, and resilience.

Governance measures should be reviewed at a regular interval and after important changes. A new law, a new device class, a new vendor, or a significant model update can change the risk. The team should define who is informed and who has the right to approve the change. This avoids the problem of a technically working system continuing to use outdated rules. A mature process can show not only that controls exist, but also that they are tested and improved when they fail.

Table 13. Trustworthiness Assessment Matrix.

Area	Evidence to inspect	Owner
Accountability	Named owner and approved policy version	Service owner
Transparency	Decision reason is understandable	Privacy team
Robustness	Stress tests and drift tests pass	Model owner
Privacy	Method and assumptions are recorded	Privacy engineer
Security	Identity and update controls are tested	Security team
Resilience	Safe operation during outages is shown	Operations team
Human oversight	Reviewer can challenge the action	Governance lead

18.4 Governance review cycle

A governance review should occur on a schedule and after important changes. It should examine open incidents, model drift, policy exceptions, audit completeness, device support, and changes in law or business purpose. The review should create actions with owners and due dates. If a high-risk action is repeatedly overridden, the team should assess whether the model, threshold, policy, or workflow is wrong. Governance is effective only when evidence leads to decisions and corrections.

The record of a review should include what was checked, what evidence was used, what was approved, and which risks remain. It should not simply say "review completed." The review process can use the same traceability framework as the software tests, linking each risk to a control and each control to its evidence. This allows an internal auditor or project supervisor to inspect the process without relying only on oral explanations.

18.5 Practical maturity indicators

Maturity can be assessed with observable indicators. A basic level has an inventory and named owners. A defined level has written policies and documented test cases. A measured level records model and rule performance. An improving level uses incidents and review results to change controls. These steps are not a legal certification. They are a way to decide what needs to be strengthened next. Table 13 gives example evidence for each governance area.

Maturity should not be judged by the number of tools installed. A dashboard, model registry, or policy engine is useful only if people maintain it and act on its outputs. A small system with clear owners, tested rules, and reliable evidence may be better governed than a complex system with unclear responsibility. The framework should therefore measure practices and outcomes rather than reward technology for its own sake.

18.6 Responsibility across people and systems

Accountability requires a clear answer to the question “who is responsible for this decision?” A model may generate a risk score, but the service owner is responsible for the way the score is used. A privacy owner approves the purpose and data controls. A security owner manages credentials and incident response. A policy owner approves rules and exceptions. An operator monitors availability and failed actions. An auditor checks whether evidence is complete. In a small project, one person may hold multiple roles, but the responsibilities should still be stated.

The system should not allow a model to become a hidden decision-maker. High-impact or unclear cases should have a review path, and the reviewer should have enough evidence to understand the system output. A person should be able to correct a wrong data label or request a policy review without erasing the history of the original decision. Any override should be visible, time-limited where possible, and examined later. These controls make accountability part of the workflow rather than a statement placed only in the introduction.

18.7 A simple assurance case

An assurance case is a structured statement of what the organisation believes is safe and why. For this framework, one claim might be: “The system blocks transfers to unapproved receivers.” Evidence could include the policy rule, automated tests, integration test results, and a sample audit trail. Another claim might be: “The system can explain high-risk alerts.” Evidence could include reason codes, reviewer studies, and examples of correct and incorrect alerts. A claim without supporting evidence should be labelled as an assumption or a goal, not as a finding.

The assurance case should also record limits. For example, the system may block a destination that is not in the approved list, but it may not know whether a listed receiver later misuses data. It may flag unusual traffic but miss a new pattern. It may record an audit event but depend on the integrity of the host that generated the event. Writing these limits beside the claim makes the safety argument more honest and shows which external controls are still required. The assurance case should be updated whenever a policy, model, device class, or important workflow changes.

19. Industry Use Cases and Case Study Evaluation

19.1 Smart healthcare

A wearable may send heart-rate or movement data to a care service. The gateway checks the device, verifies the declared purpose, and labels the event as health-related. If the approved task needs only a summary, the system may send an aggregate rather than the full stream. A request to send the same record to an unapproved analytics service should fail the receiver rule. The AI score may flag the request as unusual, but the strict policy is enough to block it. The audit record stores the reason and policy version, not the full health record [11][24][26][46].

A test for this use case should include an approved care receiver, an unapproved receiver, missing purpose, a revoked device, and a policy server outage. The team should check whether the decision is correct and whether the saved record explains it. It should also test whether a reviewer can find the event without seeing unnecessary patient details. The use case should not be described as a hospital deployment unless it has actually been built and approved by a real healthcare organisation.

19.2 Smart city and transport

A city may use traffic sensors, parking cameras, utility meters, and public-space counters. A planning task may need a number of cars by time interval rather than individual faces or long movement histories. Edge processing can count vehicles locally and send only counts and quality indicators. If a service requests detailed tracking that is not included in its approved purpose, the policy can hold or block it. This can reduce unnecessary data movement while keeping the intended city service available [6][23][25].

The test should consider crowded events, unusual weather, device failure, and a change in the destination service. It should also check whether aggregated data can still reveal individuals when the groups are too small. A privacy rule should define a minimum group size or other condition where needed. The city must still consider local law, public expectations, retention, and access to the raw data. Edge aggregation is a useful control but does not make every surveillance use acceptable.

19.3 Manufacturing and energy

Industrial sensors record machine temperature, vibration, energy use, production timing, and sometimes worker activity. Some records are trade secrets; others can reveal a person's work pattern. The system should separate the categories and apply different access and retention rules. A machine-health model may require detailed vibration data, while a business dashboard may only need a daily summary. The purpose and user role should determine which version is provided [15][16][30].

Energy readings may reveal when a building is occupied. A supplier may need aggregate demand to plan resources, but it may not need a household's exact usage history. A policy can reduce the time resolution or send an aggregate. Tests should confirm that the reduced data still support the intended task. This is important because privacy transformations can also reduce usefulness. The report should measure the task's quality after transformation rather than assuming that minimisation has no effect.

19.4 Logistics, transport, and agriculture

Logistics systems may track vehicles, packages, drivers, and delivery times. Exact routes can reveal where a person works or lives. An agricultural IoT system may collect land conditions, equipment activity, irrigation events, and location data. Some of these records are operationally sensitive even when they do not identify a person. The framework can label the data, check the declared purpose, and limit which services may receive raw values. It can send totals or ranges when those are enough for planning.

The same high-level pipeline can support these settings, but the policy catalogue must be adapted. A logistics service may need a short-term precise route, while a monthly performance report may only need an aggregate. An agricultural system may share a field moisture summary without sharing equipment identifiers or exact operational timing. Each use case needs stakeholder review and test cases that represent its actual risk. A generic rule set should not be treated as a full policy for every industry.

Table 14. Industry Use Cases Matrix.

Use case	Main risk	Example control
Healthcare IoT	Health record sent to wrong service	Purpose check, strict access, minimum fields.
Smart city	Linked sensor data tracks people	Local aggregation and reduced precision.
Manufacturing	Trade secrets or worker monitoring	Separate access roles and narrow data collection.
Energy	Usage data reveals occupancy	Aggregate readings and limit history.
Transport	Route history retained too long	Purpose and retention limits.
Agriculture	Operational data shared too widely	Owner labels and approved receivers.
Logistics	Personal location exposed	Limited precision and short retention.

19.5 Worked decision example

Consider an event with impact I = 0.90, likelihood L = 0.85, and uncertainty U = 0.80. Under the proposed formula the score is 85.75, displayed as 86. These are example values only. The event also requests to send a health record to a receiver that is not on the approved list. The policy engine rejects the transfer because the receiver check fails. The score helps explain why the case should be reviewed, but the mandatory receiver rule is the decisive reason.

The output should show the event ID, receiver rule, policy version, action, and review status. The audit log should not store the full health payload if the decision can be explained without it. A reviewer should be able to inspect the reason and confirm the result. Figure 8 shows this example. It is a diagram of the proposed behaviour, not evidence of a live system event. A future implementation should run the scenario and include the actual log record as supplementary evidence.

19.6 How to evaluate the case studies

Each industry use case should be turned into test cases before deployment. The test should list the event, data category, declared purpose, destination, mandatory rules, expected action, and audit record. The same pipeline can be reused, but the expected answer should change with the context. A healthcare test may focus on a sensitive measurement and approved care service, while a smart-city test may focus on location detail or aggregate data. These differences should be explicit rather than hidden in generic examples.

The results should be reviewed with the people who understand the domain. A clinician can explain whether a health data reduction preserves the intended task. A city privacy officer can evaluate whether an aggregate still carries re-identification risk. A manufacturing operator can explain which fields reveal trade secrets. This domain review complements technical testing. It does not replace legal advice but can uncover practical harms that a software-only team may not anticipate.

19.7 User involvement and acceptable errors

Privacy is about people, so the system should consider the experience of those affected by the data use. The available notice, access, correction, objection, or deletion pathways depend on the applicable law and service. The system should provide a clear explanation when a request is blocked and, where appropriate, a path for review. It should not expose confidential rule details to an unauthorised person, but it should avoid vague messages that leave people unable to understand what happened.

Error costs differ between applications. A false block in a safety-critical healthcare path may delay care, while a missed privacy violation may expose sensitive information. A city sensor may tolerate a small delay but not unnecessary tracking. The project should work with domain owners to rank these harms and set action rules. The error trade-off should be documented before the thresholds are chosen. This helps ensure that model performance targets reflect the needs of the service rather than a generic score.

19.8 Smart-city scenario: aggregate rather than track

Consider a city that wants to estimate traffic congestion at a junction. Cameras or road sensors produce time-stamped observations. The task may require vehicle counts by time window, but it may not require storing identifiable images or following one vehicle over a long route. The edge service can count vehicles locally, discard image frames after the approved processing step, and send aggregate counts to a planning service. The policy should state the permitted purpose, output granularity, retention time, and approved receiver. If a request asks for person-level tracking without the required authorisation, the system should hold or block the request according to policy.

The test should compare the aggregate output with a suitable reference count, check whether the task still works after detail is reduced, and verify that raw frames are not kept beyond the approved time. It should also test a camera that is not registered, a change in the destination, and an outage at the edge gateway. The audit record should show the policy and action but should not contain the images themselves. This case demonstrates that privacy and utility must be assessed together: a less detailed output may be sufficient for city planning while exposing less personal information.

19.9 Industrial scenario: machine data and worker privacy

A factory may collect machine vibration, production rate, worker badge events, and maintenance logs. These data have different meanings and should not share one broad "industrial data" label. Machine vibration may be operational data; a badge event can reveal a worker's movement; a production record may contain trade secrets. The same receiver may be approved for one category but not another. The policy engine should therefore consider data category, purpose, recipient role, and retention rule for each event.

An industrial test should check the system under normal load and during a machine fault. The system must not delay a safety-critical control action while waiting for a non-critical privacy review. Instead, control and compliance workflows should have a clear boundary. If a privacy check is needed for a later analytics copy, the raw operational control path can remain separate while the analytics path is minimised and governed. The test must document this separation and verify that the privacy mechanism does not create unsafe control delay. This example shows why system design must account for domain-specific safety as well as data protection.

20. Business Impact Assessment

20.1 Expected types of value

A privacy compliance service may reduce manual repetition, help staff find risky events sooner, and make audit evidence easier to prepare. It may also reduce unnecessary central data storage if edge minimisation is applied. These benefits are possible, not guaranteed. If the model produces too many alerts, staff workload may increase. If metadata are incomplete, automation may hold many events until a person supplies the missing facts. The business case must account for both gains and new operating duties.

The organisation should measure baseline effort before implementation. Useful measures include time spent reviewing requests, time spent collecting audit evidence, number of policy exceptions, number of unresolved events, and the cost of running the system. A comparison after implementation should use the same definitions and a similar period. Without a baseline, a claim of cost reduction is not well supported. The report therefore describes a measurement plan instead of inventing a financial return.

20.2 Costs and return on investment

Costs may include software development, gateway hardware, cloud services, security testing, model training, policy review, staff training, maintenance, and incident handling. Benefits may include less manual work, lower data-storage requirements, fewer repeated checks, and better evidence quality. Some benefits are hard to express as money, such as clearer accountability or improved trust. These values should be described separately rather than assigning a made-up monetary amount.

Return on investment can be calculated only when the benefit and cost estimates are based on stated assumptions or actual records. Let B be total yearly benefit and C be total yearly cost. The proposed calculation uses ROI = 100 × (B − C) / C, when C is greater than zero. A report should explain how B and C were calculated, which time period was used, and which costs were excluded. This report has no operational cost data, so it does not claim a numerical ROI.

$$ROI = 100 \times \frac{(B - C)}{C} \quad (12)$$

Equation (12) expresses return on investment (ROI) as a percentage, where B is total yearly benefit and C is total yearly cost. Use it only when C is greater than zero and both values have a documented basis.

Table 15. Business Impact Assessment Framework.

Item	How to measure	Status in this report
Review time	Work logs before and after implementation	Not measured
Audit preparation	Time needed to gather evidence	Not measured
System cost	Hardware, cloud, staff, testing, maintenance	No operational cost data
Net benefit	Measured benefit minus yearly cost	Not calculated
ROI	100 × (benefit − cost) / cost	Not calculated
Trust and rights	Review outcomes and user feedback	Requires field study

20.3 Operational adoption

A system can fail if its users do not understand the workflow or if the review queue is too hard to use. Adoption planning should include training for device operators, privacy officers, security staff, and auditors. Each role should know what it can approve and what it must escalate. The organisation should also decide how quickly high-risk cases must be reviewed and what happens when no reviewer is available.

A rollout should begin with a small pilot that uses non-sensitive or carefully controlled data and has a clear stop procedure. The pilot should test rules, logs, and failures before the system is allowed to act on high-impact requests. The team should compare the pilot results with the baseline and publish the limitations. Moving directly from a conceptual design to full operation would create avoidable risk, especially where health, location, or critical infrastructure data are involved.

20.4 Cost profile across the system life cycle

Costs should be grouped into initial build, integration, operation, maintenance, and review. Initial costs include device adapters, policy mapping, secure key setup, and the first model evaluation. Operating costs include compute, network use, log storage, review staff, and monitoring. Maintenance costs include policy changes, model retraining, device updates, and regression tests. Incident costs may include investigation and recovery. A realistic business case should state which categories were measured and which are only estimates.

Benefits should be measured with the same care. Time saved in evidence gathering can be counted from work logs. Reduced central data movement can be measured from network and storage records. Fewer manual checks can be measured from workflow data. Improved trust or reduced harm may require a separate study and should not be assigned a financial value without a reason. The report does not have actual cost records, so it offers a structure rather than a numerical estimate.

20.5 Decision rules for business adoption

An organisation should adopt the framework only if it meets technical, privacy, operational, and governance requirements. The pilot should show that mandatory rules are correctly enforced, audit evidence is complete, and the review queue is manageable. The team should also assess whether the controls add unacceptable delay or block legitimate work. If an approach does not meet its acceptance conditions, the correct response is to fix it or use a simpler method, not to claim a benefit that has not been measured.

The business decision should state which risks remain and who accepts them. It should identify the resources needed to maintain the system and the conditions that trigger re-evaluation. A system that works during a short pilot may need additional controls before it is used across different locations or vendors. The cost-benefit analysis should therefore be updated after a period of real operation and after material changes to data, models, rules, and suppliers.

20.6 Establishing a baseline before claiming benefit

Business impact should be measured against a baseline. If an organisation wants to claim that automated checks reduce review time, it should first record how long the same kind of review takes under the current process. If it wants to claim that audit preparation becomes easier, it should record the work required to gather evidence before the framework is introduced. A before-and-after comparison should use a similar workload and explain changes in staffing, policy complexity, and event volume. An estimate can be useful, but it must be labelled as an estimate and its assumptions should be shown.

Benefits can include fewer repeated manual checks, earlier detection, better traceability, reduced duplication of raw data, and more consistent policy enforcement. Costs include development, integration, model maintenance, device management, legal review, storage, training, and incident response. The total should also include the time needed to investigate false alerts. A dashboard that shows more alerts is not automatically a better outcome; the organisation must measure whether the alerts lead to useful action.

20.7 Adoption targets and stop conditions

Before a pilot, the organisation should set target values and stop conditions. It may require a minimum recall for specified high-risk cases, a maximum false-alert rate, a response time within a limit, and complete audit records for all decisions. It may also require a safe fallback during outages. If a target is not met, the team should not describe the pilot as successful just because other measures are strong. It should record the issue, identify the cause, and decide whether the system needs redesign or is unsuitable for that use case.

Adoption should be gradual. A shadow mode can record what the system would have done without enforcing the action. Reviewers can compare the suggestion with existing decisions. A controlled pilot can then allow low-risk cases before enabling enforcement for more sensitive events. At each stage, the organisation should review errors, user feedback, workload, privacy impact, and operational cost. The final decision to deploy should be owned by the organisation, not inferred from a model score or a positive chart alone.

21. Challenges and Limitations

21.1 Data and model limits

The first limitation is that privacy labels are not easy to obtain. Public network data sets often label attacks, not whether a data use has an approved purpose. Synthetic events can check program logic, but they do not show how often a real organisation faces each case. A future study needs a clear label guide and a process for reviewing uncertain examples. Results from one dataset may not transfer to other devices, countries, or sectors [1][3][29].

The second limitation is model drift. A new device or service can change the meaning of normal behaviour. A model may also perform differently across device classes. If the training data contain fewer examples from one group, the model may miss events in that group. The system needs monitoring and retesting after significant change. It should not rely on one overall average score to claim that it works equally well across all environments.

21.2 Legal and organisational limits

The framework cannot automatically resolve every legal question. Whether a purpose is lawful, whether an exception is available, or which law applies may require expert judgement. The policy catalogue can store the organisation's approved interpretation and its source, but it can become outdated. Legal updates, new products, new vendors, and cross-border changes need a review process. A student paper cannot replace the legal and privacy review required before a real system is deployed [31][44][45][46].

The organisation may also lack the staff needed to maintain policies, models, and devices. Small teams may have difficulty reviewing every alert or checking every vendor update. The design should therefore use risk-proportionate controls and keep the rules understandable. It should also identify which actions can be safely automated and which require a person. Automation can reduce repeated work, but it does not remove responsibility from the organisation that uses the system.

21.3 Performance and privacy trade-offs

More checks can improve coverage but increase delay. Stronger privacy mechanisms can reduce information leakage but also reduce utility or increase resource use. Edge processing may limit central data movement but add the burden of managing local hardware. Federated learning can keep raw data local but still expose model updates. Audit logs improve accountability but can reveal the behaviour of devices and people. These trade-offs must be measured in the target setting rather than assumed from a diagram or a general survey [8][9][10][11][12][13][14][15][16].

A report should show where the chosen design may fail. For example, the model may not run on a low-power sensor, a network outage may prevent a new policy from arriving, or a data category may be incorrectly assigned. For each failure, the system should specify a safe fallback and an owner for correction. The limitations are not a weakness to hide; they help define what further research and implementation are needed.

21.4 Ethics and data protection for the compliance tool

A compliance system can collect sensitive metadata while trying to protect the original data. Logs may show which services a person used, when a device was active, or which request was rejected. The compliance tool needs its own purpose, access control, and retention policy. Model training data, explanations, and decision records should be reviewed for privacy. The system should not collect an entire record merely because it might be useful for debugging.

The organisation should also consider the people affected by false alerts. A false block can interrupt a medical or safety service; an incorrect approval can expose personal data. The response should be proportionate to the risk and the use case. High-impact actions need careful testing, review, and a way to appeal or correct an error. The framework should protect people as well as protect the organisation's ability to show that it has controls.

21.5 Automation risk and over-reliance

Automation can make a process faster, but it can also make mistakes appear official. A reviewer may accept a high-confidence score without checking the underlying facts. An engineer may assume that a signed policy is correct simply because the signature is valid. A dashboard may show that all checks completed even when the underlying rule was not suitable for the situation. The design needs controls for these forms of over-reliance, including human review, explicit uncertainty, and separate testing of the legal mapping.

The interface should distinguish technical success from compliance approval. A model may complete successfully while the prediction is wrong. A rule may execute correctly while the approved rule is outdated. A log may be complete while the event fields were wrong. The system should name these different states and should not collapse them into a single green status. Regular review and challenge mechanisms help prevent staff from treating automation as a substitute for judgement.

21.6 Limits of the present case study

The report is a design-oriented case study and does not include a working deployment, labelled field dataset, measured latency, or cost records. Its numerical examples use the proposed equation or assumed curves. The literature review is focused, not a fully systematic review of every relevant paper. Legal mappings are illustrative and need review for the real organisation, jurisdiction, and effective date. These limits mean that the report cannot claim improved privacy, better detection, or cost savings in production.

The next study should implement the architecture, define a public or permitted dataset, run the planned tests, and compare the system with manual and rules-only baselines. It should publish the configuration, measured results, failures, and limitations. The results should then be reviewed by people who understand both privacy and the domain. This path turns the current report from a proposal into a testable research project without overstating what has already been achieved.

21.7 Ethical concerns in automated monitoring

A system that monitors data flows can itself become a monitoring tool for people. The logs may reveal when a person used a service, when a patient measurement was taken, or which staff member reviewed an event. The system should collect only the information needed for its stated purpose. Access should be limited, retention should be defined, and uses outside the compliance task should be forbidden unless separately reviewed. This principle should apply to the model training data, explanations, audit logs, and dashboards, not only to the original IoT data.

The system should avoid using risk scores to judge a person when the score was designed to assess an event or transfer. A high score may mean that the receiver is unknown or the metadata are incomplete. It does not mean that a person behaved badly. User-facing language should explain the system action without turning uncertainty into blame. The organisation should also provide a way to challenge a wrong label, report an unfair rule, and request a human review where appropriate. These safeguards reduce the risk of the compliance system causing a new privacy or fairness problem.

21.8 Limits caused by data and setting

The proposed risk equation uses impact, likelihood, and uncertainty as simple inputs. Real deployments may need different variables, weights, and thresholds. The illustrative weights are not validated standards. The score may be useful for ranking events, but its meaning depends on the quality of its inputs and the way experts define them. It should not be interpreted as a probability unless a suitable calibration study supports that interpretation. A future report should evaluate the chosen weights and show what changes when each weight changes.

The framework also depends on correct policy metadata. If an approved-purpose list is incomplete or a receiver is incorrectly recorded as trusted, the policy engine may enforce a wrong rule consistently. Better software cannot fully repair a flawed governance process. The system needs owners, review dates, source documents, and a method for reporting wrong rules. This limitation is especially important where laws vary by region or where an organisation has complex relationships with service providers. A technically correct execution of a mistaken rule is still a governance failure.

22. Future Research Directions

22.1 Better data and shared test suites

Future research should develop shared, well-documented data sets for privacy-compliance events. Such a data set should include approved requests, clear violations, missing-information cases, uncertain events, and device failures. It should have a label guide, privacy controls, a licence, and a process for correcting mistakes. Public attack data can still be used for the security component, but privacy-policy labels should be created separately. This would allow researchers to compare methods fairly rather than creating a different test set for every paper.

A shared benchmark should include device heterogeneity. It should test small and large devices, several event rates, different data categories, and changing policies. It should also include negative cases, because a system that blocks everything may appear to catch every violation but cannot support useful services. Benchmark results should report precision, recall, false-positive rate, latency, resource use, audit completeness, and privacy parameters where relevant.

22.2 Privacy-aware digital twins

A digital twin is a digital model of a real system that can be used to examine changes before they are applied. A privacy-aware twin could represent devices, data flows, policy rules, and likely failure points. Researchers could test whether a policy update blocks legitimate work or opens an unintended route for data. The twin would need to represent how data are collected, linked, transformed, and retained, not only the network layout.

The twin should not be treated as proof of the real system's behaviour. Its model may omit important details or use inaccurate event distributions. Research should compare twin predictions with the actual results from a controlled testbed and document the differences. The most useful role is to catch likely problems early and improve test coverage before policy changes are deployed to sensitive data.

22.3 AI agents and policy assistance

AI agents may help locate regulatory updates, compare a proposed rule with an approved policy, suggest test cases, or summarise an audit record. These tasks can save time when the source documents are large. However, an agent can misunderstand legal language, miss a date, or follow misleading input. A suggested policy should therefore remain a draft until a responsible person checks it. The agent should state its source, the evidence it used, and any uncertainty.

Future systems should test whether policy agents can be fooled by malicious text, conflicting documents, or outdated sources. High-impact policy changes should require approval and have a rollback path. The agent's actions should be logged and versioned. Research should measure the number of correct suggestions, missed changes, false suggestions, review time, and the rate at which humans reject the output. It should not assume that language generation alone makes compliance automatic.

22.4 Quantum-safe and long-term protection

Long-lived IoT records may need protection for many years. Organisations should review whether the cryptographic mechanisms they use can be updated when standards change. Quantum-safe cryptography is one future area of work, but adopting a new algorithm requires compatibility testing, resource measurement, and key-management planning. Small devices may not have the same capabilities as a cloud service, so the update path should be planned before the system is deployed.

Privacy rules also need long-term maintenance. A record may remain stored after the device that generated it has been retired. A policy may remain in a gateway after the organisation has changed its purpose. Research should examine how to expire old rules, migrate audit evidence, rotate keys, and delete records safely across edge and cloud systems. These tasks matter as much as initial setup because a system that cannot be updated may become unsafe even if it was designed well at the start.

22.5 A staged research roadmap

The first stage should implement the event schema, device identity check, hard-rule engine, and audit record. These components can be tested without a machine-learning model and provide a clear baseline. The second stage should add the risk score and a review queue. The third stage can test a trained anomaly detector using properly labelled events. Later stages can compare edge inference, federated learning, secure aggregation, and differential privacy where they suit the use case. Each stage should have a documented acceptance condition before moving to the next.

The roadmap should include security review and privacy review at every stage. A new AI feature can introduce new data needs and new attack surfaces. A new privacy mechanism can change utility and latency. A new legal mapping can change policy actions. The project should measure the effect of each addition rather than introduce many components at once. This staged process helps identify which part provides value and which part adds cost without measurable benefit.

22.6 Open research questions

Several questions remain open. How can a shared policy language represent different laws without hiding important differences? How can the system detect when a policy mapping is outdated? Which explanation types help reviewers make correct decisions rather than merely increase confidence? How can privacy guarantees be accounted for across repeated releases and multiple model updates? How should the system operate when devices have weak hardware or intermittent connectivity? These questions are suitable for separate experiments rather than one broad claim about AI compliance.

Future work should also study how to share evaluation data safely. Researchers need labelled privacy-policy cases, but the records may contain sensitive context. Synthetic data can help test rule logic, while de-identified or controlled data may be needed to evaluate real behaviour. A shared benchmark should include limitations and an ethical data-use process. Better evidence and shared test methods could make future studies easier to compare and reduce the temptation to rely on single, non-comparable performance numbers.

22.7 Controlled use of AI agents for policy support

An AI agent may help locate a new legal document, summarise a change, suggest a control, or create candidate tests. These tasks may reduce drafting effort, but the agent should not silently change a high-impact rule. A safe process would store the source document, the date it was checked, the generated suggestion, the reviewer’s edits, the approval, and the version deployed. The suggestion should be compared with the current policy and the affected system components before it is accepted.

Future tests should examine whether an agent can misread a legal exception, follow malicious instructions hidden in a document, cite a source that does not support its statement, or generate a rule that conflicts with an existing policy. The evaluation should measure source accuracy, reviewer correction time, dangerous suggestions, and the quality of generated test cases. The agent should operate with limited permissions and should not have direct write access to production policy stores. This keeps the possible benefit while reducing the risk of autonomous errors.

22.8 Digital twins and long-term protection

A digital twin can represent part of an IoT environment in a controlled test setting. It may help a team simulate a new sensor type, a network outage, a policy change, or a sudden increase in events before changing the live system. The twin should include the relevant data flow and policy conditions, not just the physical device. It should also be clear which parts are simulated and which measurements came from a real system. A simulation can reveal likely failures, but it cannot prove that a live deployment behaves in exactly the same way.

Long-term protection also needs planning for cryptographic change. A system may keep records for many years, while cryptographic recommendations and device capabilities change over time. Future work should document key lifetime, algorithms, update methods, and how old signed records will be verified. Quantum-safe techniques may become relevant for long-lived sensitive information, but any migration should be based on current standards, the actual threat model, and tested interoperability. The research goal is a controlled upgrade path rather than an unsupported claim that one algorithm protects every future system.

23. Conclusion

23.1 Summary of the proposal

This case study proposed a layered framework for AI-supported privacy compliance in heterogeneous IoT environments. It combines identity checks, data classification, edge privacy controls, a risk engine, a policy engine, and audit evidence. Its main design rule is that mandatory policy checks remain in control. AI can flag unusual events and help prioritise review, but it cannot make an unapproved data use acceptable simply by giving it a low score.

The report also explained the strengths and limits of common privacy tools. Encryption protects data during storage and transfer. Data minimisation reduces the fields used. Pseudonymisation replaces direct identifiers but does not always make data anonymous. Differential privacy provides a mathematical guarantee for a defined mechanism and budget. Federated learning can reduce raw-data movement but may leak information through updates. Each tool must be selected for a clear risk and tested in its intended setting.

23.2 Main research lessons

The literature shows that AI, edge computing, privacy-enhancing technology, and legal mapping solve different parts of the problem. Their value depends on how they are connected. A clear system needs reliable metadata, explicit rules, a safe path for uncertain events, explanations, and records that can be reviewed later. It also needs owners who update policies and models as devices and rules change. These are engineering and governance tasks, not only machine-learning tasks.

The proposed equations and graphs are included to support design reasoning. Their values are synthetic or formula-based and must not be reported as real results. The next research step is to implement the architecture, test it with labelled events, compare it against fair baselines, measure the trade-offs, and publish the run configuration and actual results. That future experiment will determine whether the proposal improves detection and evidence quality without creating too much delay, overhead, or false-alert workload.

23.3 Practical implications

For practitioners, the immediate step is to document device identities, data purposes, data categories, receivers, retention rules, and policy owners. The second step is to test clear rules before adding AI. The third is to design audit evidence and safe failure behaviour at the same time as the decision logic. The fourth is to run a controlled pilot and measure both the benefit and the burden. A system should move into production only when the evidence shows that it meets the requirements for its use case.

The framework is intended as a practical starting point rather than a claim of universal compliance. Different organisations will need different rules, and laws may change over time. A human legal or privacy review remains necessary. With that caution, the report offers a complete plan for connecting privacy engineering, AI risk support, local processing, enforcement, and accountability. The final example in Figure 8 shows the expected form of an output: a decision, its reason, and an audit record that can be checked later.

23.4 Implementation sequence

The practical implementation sequence should begin with inventory, purpose, data categories, receiver lists, and retention rules. It should then add a common event schema and validate the required fields. The next stage should implement hard policy rules and minimal audit records. Only after these controls pass their tests should the project add AI risk ranking. The AI component should be evaluated against the rules-only baseline, and the output should be used only within a clearly approved purpose and scope.

The first pilot should use a controlled set of events and have a stop procedure. The team should measure correctness, false alerts, latency, resource use, evidence completeness, and the number of cases sent for review. Failure cases should be retained and analysed. A production decision should depend on the observed evidence and an explicit acceptance threshold. This sequence makes it less likely that the project adopts a complex model before the basic privacy rules and records are reliable.

23.5 Final recommendations

The main recommendation is to keep privacy rules simple, explicit, and testable. Use AI to support review rather than to replace mandatory checks. Keep raw data close to the source when practical. Make each action explainable. Record the policy and model versions used. Set a safe fallback for missing information and unavailable services. Review the legal mapping when the purpose, organisation, device, or jurisdiction changes. These practices help maintain control as a heterogeneous system evolves.

The final recommendation is to report evidence honestly. The examples in this report show how the design works on paper, but they are not experimental findings. Future work must run the code, save the measurements, compare baselines, and publish errors and limitations. A paper becomes stronger when readers can reproduce its method and see the limits of its claims. This case study supplies the detailed plan needed for that next stage.

23.6 What the design can and cannot establish

The design provides a clear order for event checking, a risk-score example, policy actions, and a plan for recording decisions. It also explains where edge processing, AI, privacy-enhancing technology, and human review fit together. These elements are useful because they make later testing more specific. A reviewer can check a strict receiver rule, inspect an explanation, rebuild a past decision, and test safe behaviour when a service is unavailable.

The design does not establish that a deployed system has achieved a particular accuracy, reduction in privacy risk, speed, or cost saving. Those claims require a working implementation, labelled test events, measured baselines, and repeated runs. The graphs in this report are illustrative and are identified as such. The main academic contribution is the testable design and its detailed evaluation plan, not a claimed performance result that has not been measured.

23.7 Practical next steps

The next step is to build the event schema, device adapter, strict policy engine, audit record, and the simplest working risk scorer. The team should first test individual rules, then the full event pipeline with synthetic examples. It should next add a labelled evaluation set and run manual-review, rules-only, and combined methods on the same events. Only after those tests work should the system be tried on a controlled dataset from a real application.

The implementation report should include the source-code version, environment, dataset licence, preprocessing, event count, label guide, model settings, policy versions, baseline definitions, raw metric results, and failures. It should also explain how the system handles personal data during the test. These steps make the work easier to repeat and give the next reader a fair way to judge the claims. The proposed framework is therefore a starting point for measurable research rather than the final proof of privacy compliance in every IoT environment.

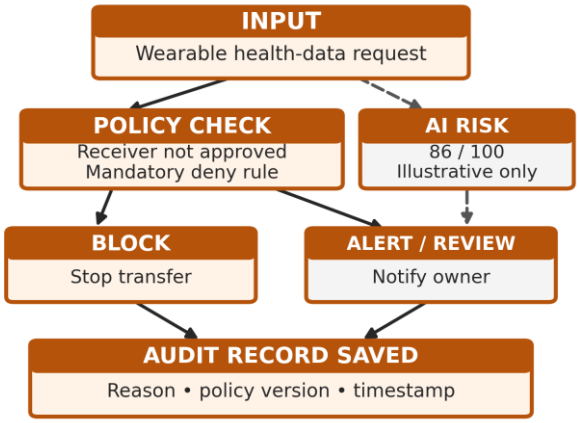


Fig. 8. Example System Output: Compliance Decision.

Worked example only. The score is illustrative, and no live transfer or production system is claimed.

DATA AVAILABILITY

The synthetic illustrative event data generated for this design-focused case study are provided in the supplementary Datasets folder. No empirical or field dataset was collected or analysed, and no personal or patient data were used. The numerical examples and figures are synthetic or formula-based and are not measurements from a deployed system. The supplementary event records are provided only to demonstrate the proposed scoring and policy logic; they do not establish model accuracy, operational performance, or legal compliance.

Appendix A. Policy Rule Template

A policy record should be short enough for engineers to implement and detailed enough for a reviewer to understand. The following template is a starting point. Each field should be completed for the real use case; the examples are not legal advice.

Policy ID: PRIV-IOT-EX-01. Policy owner: named privacy or service owner. Version: 1.0. Status: draft until approved. Source requirement: the relevant law, contract, standard, or internal policy. Effective date: date confirmed by the owner. Review date: date by which the rule must be checked again. Jurisdiction: the location and role for which the rule applies. Data category: the approved data label. Purpose: the specific allowed task. Receiver: the service or organisation allowed to receive the data. Retention: the period or event that ends storage. Required controls: encryption, minimisation, pseudonymisation, access control, or other measures. Expected evidence: event ID, policy version, action, reason, and reviewer if used.

Example condition: if device identity is invalid, reject the event. Example condition: if receiver is not approved, block the transfer. Example condition: if required purpose metadata are missing, hold the request and ask for review. Example condition: if the retention period is expired, invoke the approved deletion or isolation workflow. Example condition: if all mandatory rules pass but the AI engine flags an unusual pattern, route the event to an appropriate risk action based on the approved thresholds.

Every rule needs at least one positive and one negative test. The positive test checks that the rule permits a valid case. The negative test checks that the rule rejects or holds a case that does not meet the requirement. Boundary tests should include expired approvals, missing fields, mismatched policy versions, and service outage. The result must be saved with the expected outcome, actual outcome, software version, policy version, and a short explanation. If the test fails, the policy should not be deployed until the problem has been reviewed.

A.1 Example machine-readable policy record

A policy record may be represented as a JSON object. A real implementation should validate it against a schema and reject unknown or unsafe fields. The record should include policy_id, version, owner, effective_at, expires_at, data_class, allowed_purposes, allowed_receivers, retention_rule, hard_conditions, actions, evidence_fields, review_owner, and source_reference. The JSON should not contain secret keys or copies of personal data. It should describe the control, while access tokens and protected data remain in their own secure stores.

A.2 Review workflow

A policy should move through draft, technical review, privacy/legal review, test, approval, staged release, active use, and retirement. The record should preserve who performed each step and when. A rejected policy should remain available for audit but must not be active. If a policy changes an action that can affect a person or a critical service, the change should be assessed for harm before deployment. The workflow must define who can approve an exception and how long the exception remains valid.

Appendix B. Reference Pseudocode for the Event Pipeline

The pseudocode below describes the proposed decision order. It is intended to make the implementation plan easier to follow. It is not executable code and does not provide a security guarantee on its own.

```
FUNCTION process_event(event):
1. verify_schema(event); if invalid,
reject_and_log("invalid_schema")
2. verify_device(event.device_id); if
revoked_or_unknown,
reject_and_log("invalid_device")
3.
load_current_signed_policy(event.conte
xt); if missing_or_expired,
apply_safe_fallback()
4. classify_data(event); if
unknown_sensitive_context,
hold_for_review()
5.
verify_purpose_receiver_retention(even
t, policy); if any
mandatory_rule_fails,
apply_policy_action()
6. minimise_or_transform(event) when
the policy requires it; if
transformation fails, stop transfer
7. compute_risk_score(event); record
reasons and uncertainty
8. if risk exceeds approved review
threshold, send to reviewer or apply
the approved action
9. save_minimal_audit_record(event_id,
policy_version, model_version, reason,
action, result)
10. return the final state and
explanation.
```

The order matters. Schema and identity checks run before AI because an event with a malformed or invalid source should not be treated as a normal model input. Mandatory rules run before a low risk score is allowed to influence the action. A required transformation must be confirmed before the transfer continues. Audit evidence is generated only after the system knows which action actually happened. Where a step fails, the system follows a defined failure path instead of silently continuing.

The implementation should make each step measurable. The event should contain a trace ID that links gateway logs to the policy engine and audit store. Error codes should be stable across software releases. Logs should avoid raw personal payloads. The test suite should simulate policy service outages, model timeouts, malformed data, and audit-store failures. Each result should state whether the event was safely rejected, held, transformed, or allowed.

B.1 State machine for one event

The event state can be represented as RECEIVED, VALIDATED, CLASSIFIED, POLICY_CHECKED, RISK_REVIEWED, ACTIONED, and LOGGED. An event should not jump directly from RECEIVED to ACTIONED. A failed stage should create a named error state such as REJECTED_INVALID_DEVICE, HELD_MISSING_PURPOSE, BLOCKED_RECEIVER, or REVIEW_REQUIRED. A transition should include the rule that permits it. This state machine can be used for unit tests and for explaining the process to an auditor.

B.2 Idempotency and retries

Networked systems may receive the same event more than once because a sender retries after a timeout. The processing system should use a stable event ID and define how duplicates are handled. A duplicate should not create repeated exports or multiple contradictory decisions. If the audit service fails after an action is applied, the system needs a safe recovery path so that a retry does not repeat a harmful action. These conditions should be included in testing rather than treated as rare implementation details.

Appendix C. Event Data Dictionary

The event data dictionary defines common fields used by the proposed architecture. The fields below are a minimum starting point. Some deployments will need additional fields, while others should remove fields that are not needed. The purpose is not to collect as much metadata as possible; it is to record enough context to make and explain the decision.

event_id: unique ID for the event. device_id: reference to the registered device. event_time: timestamp and stated time zone or clock source. event_type: normalised event category. data_class: public, operational, personal, sensitive personal, or unknown. purpose_id: identifier for the approved purpose. receiver_id: approved destination or service. retention_class: reference to the rule that controls storage time. policy_version: the policy version checked. model_version: the model used, if any. risk_score: optional numeric score with its scale. reason_codes: machine-readable reasons such as missing_purpose or receiver_denied. action: allow, minimise, transform, block, hold, or review. action_status: whether the action completed. reviewer_id: the authorised reviewer where a person decided or overrode the result.

The record may also include a data provenance pointer, transformation summary, device software version, and security state. These fields should only be retained where they support a defined task. A provenance pointer must be protected because it may allow access to the original data. If the audit service stores a reference rather than a copy, the underlying data store still needs its own access and retention controls.

A schema test should check required fields, data types, time format, range, and identifier format. It should also check that unknown categories do not automatically become low-risk categories. The test should verify that sensitive values are not copied into the audit record unless explicitly permitted. A versioned schema can help different device adapters produce records that the policy service can process in a consistent way.

C.1 Example event schema

A minimal event might contain: event_id = a unique reference; device_id = registered source; event_time = timestamp; event_type = normalised category; data_class = the reviewed sensitivity label; purpose_id = approved task; receiver_id = target service; retention_class = the storage rule; policy_version = the applicable rules; and data_pointer = a protected reference if a full payload is held elsewhere. Optional fields should be included only where they support the decision or its evidence. The event should not carry secrets in plain text.

C.2 Schema version changes

If a new field is introduced, the team should define whether older devices can continue to send events and how missing values will be handled. A field that changes its units or meaning should not retain the same version without a compatibility plan. The adapter should state which transformation was applied. The policy engine should reject a field that is required but absent. These rules prevent an apparently valid event from being interpreted under the wrong assumptions.

Appendix D. Extended Test Cases and Expected Behaviour

Test cases should be defined before evaluation. The table in the main report contains examples, and this appendix expands the set into a repeatable checklist. Each case should include the input event, expected policy result, expected system action, expected audit fields, and observed result. The observed result must be completed only after the software has been run.

D-01 Valid device and approved receiver: the identity and policy checks should pass. The result may be allowed if all other required checks pass. D-02 Revoked device: the event should be rejected before AI scoring. D-03 Missing purpose: the event should be held or sent for review. D-04 Unapproved receiver: the transfer should be blocked. D-05 Expired retention: the system should start the approved deletion or isolation process and record the result. D-06 Unusual transfer rate: the AI engine may flag the event, but the final action must follow the policy.

D-07 Model unavailable: mandatory rules should still run; sensitive uncertain events should follow the safe fallback. D-08 Policy server offline with valid signed local policy: the edge may continue within the local policy scope and must record that it operated offline. D-09 Policy server offline with expired local policy: the system should hold or block the high-risk action. D-10 Audit storage unavailable: the system should follow a pre-approved safe path, such as queueing protected local evidence or stopping the action. D-11 Invalid event schema: the event should be rejected with a schema reason. D-12 Successful transformation: the system should verify that the transformed data match the approved target form before sending them.

After the test, the team should compare the expected and observed actions. A mismatch is a failure even if the output appears reasonable. The team should group failures by cause: identity, metadata, rule logic, model result, transformation, fallback, or audit recording. This grouping helps decide whether the correction belongs in the device adapter, policy engine, model, or operations process. The full set of failed cases should remain in the report; it should not be deleted because a later code change passes them.

D.1 Expanded table of negative tests

Additional tests should cover: an invalid policy signature; a valid but expired local policy; an event with a timestamp far outside the expected window; a duplicate event ID; an unknown receiver; a purpose that is not permitted for the data class; an attempted direct transfer that bypasses the gateway; an incomplete audit record; and a model input outside the training range. For each case, the test should state the expected status, expected action, and minimum audit fields. The actual result should be completed only after the test is run.

D.2 Regression testing

When a rule, model, schema, or adapter changes, the full set of relevant test cases should run again. The test report should identify the changed component and compare the new results with the saved baseline. A fix for one event type can break another path, especially when rules share fields or use precedence. Regression tests help reveal these effects before a new version reaches a live service.

Appendix E. Reproducibility and Deployment Checklist

A future implementation should keep a reproducibility package. At minimum it should contain the application source code, dependency versions, configuration, policy files, model card, event schema, test data or permitted links, preprocessing scripts, evaluation scripts, and a record of the commands used to run each experiment. If the data cannot be shared, the report should state why and provide a synthetic test set when possible. A reviewer should be able to run the public portion of the test without access to private data.

Before deployment, the team should confirm the following: device inventory is complete; owners and purposes are recorded; data categories are reviewed; policy rules have tests; policy updates are signed and versioned; model limitations are documented; the audit store has access and retention controls; the safe fallback has been tested; incident owners are named; the review queue has a service-level target; and the regulatory mapping has a review date. A missing item should be recorded as a risk rather than assumed to be complete.

After deployment, the team should monitor unresolved high-risk events, false-alert workload, policy exceptions, incomplete audit records, model drift, device update status, failed authentication, and retention failures. The team should review these measures at an agreed interval and after important changes. The monitoring process must also be minimised and protected. The goal is not to collect everything about the system forever; it is to retain the evidence needed for safe operation and accountability.

For research publication, the final paper should replace illustrative graphs with plots derived from actual run records. The methods section should name the datasets, event counts, label process, hardware, software versions, model settings, baselines, and repeated runs. It should report errors as well as improvements. Any privacy claim should state the method and assumptions. These steps make the work easier to repeat and reduce the risk that a reader mistakes an example for a measured result.

E.1 Release checklist

Before release, the team should verify policy ownership, model approval, device identities, audit storage, update signatures, safe fallback, role permissions, and rollback. The check should include negative tests and outage tests. Every unresolved issue should have an owner, severity, mitigation, and target date. Critical issues should block release. A checklist is useful only if it leads to a decision and preserves evidence of that decision.

E.2 Publication checklist

Before a research submission, verify that the bibliography entries are complete, the in-text citations point to the intended source, the plots carry units and labels, illustrative values are identified, and all tables have a clear source or method. The paper should state which parts were implemented, which were simulated, and which remain proposed. If a dataset or code package is shared, its version, licence, and access method should be stated. The text should not claim measured accuracy, improved compliance, cost savings, or stronger privacy without measured evidence.

Appendix F. Detailed Research Questions and Hypotheses

F.1 Research questions. RQ1 asks whether a shared event schema can preserve the metadata needed to apply privacy rules across different IoT device families. RQ2 asks whether combining strict policy checks with an AI risk signal improves decision support compared with policy rules alone, without allowing the model to override mandatory restrictions. RQ3 asks whether edge-based checks can complete core decisions during limited connectivity while maintaining traceable evidence. RQ4 asks whether a human reviewer can reconstruct a decision from the saved event, policy version, model version, reason codes, and action record. These questions are deliberately limited so they can be tested with a defined event set.

F.2 Testable hypotheses. H1: for events with complete metadata, the schema validator will produce the same normalised field meanings across supported device adapters. H2: the strict policy tests will correctly block all test events with a known mandatory rule failure. H3: the combined workflow will identify additional unusual cases beyond the rules-only baseline, but the size of that change must be measured. H4: the audit record will contain all required fields for each completed decision. H5: during a network outage, the edge component will follow the pre-approved offline policy and will not use an expired or invalid policy package. These are hypotheses to test, not findings from completed experiments.

F.3 Null outcomes and failure reporting. H1 is not supported if one adapter changes or loses a required field. H2 is not supported if any known rule failure is allowed. H3 is not supported if the model adds no useful detections or creates too many false alerts to be practical. H4 is not supported when a reviewer cannot reconstruct a decision. H5 is not supported if the system silently continues under an invalid policy. The final report should state the result for each hypothesis, list the failing cases, and describe any change made after the first run. A successful retest should not erase the original failure from the research record.

F.4 How these questions connect to the literature. Reviews of AI for IoT privacy report differences in task definitions and evaluation approaches, which makes comparison difficult [1][3][4]. Studies of federated and edge learning describe privacy benefits along with leakage and coordination risks [7][8][9][10][26][27][48]. This appendix turns those broad concerns into questions about data context, rule priority, outage behaviour, and audit reconstruction. It also makes a clear distinction between detecting unusual behaviour and confirming a privacy violation. That distinction should guide the label process and the metrics chosen for the experiment.

Appendix G. Step-by-Step Experimental Protocol

G.1 Before a test run. Record the source-code commit, dependency versions, hardware, operating system, policy version, model version, dataset source, licence, and random seed where relevant. Freeze the event schema and the label guide. Define the required fields for a complete audit record. Decide which test categories are mandatory and write the expected result for each event before executing the system. Store the plan in a version-controlled file so the event labels and expected answers are not changed after model outputs are seen.

G.2 Prepare the event set. Include normal permitted events, known rule violations, missing-context events, unusual but permitted events, boundary cases, duplicate messages, late messages, clock differences, revoked devices, expired policies, and malformed inputs. Make sure that the set contains enough examples from each category to make the metric useful. If the set is synthetic, mark it as synthetic and document the generator. If public data are used, retain the original source and licence information and document the transformation into compliance events. Network-attack labels must not be treated as privacy-violation labels unless the test definition explicitly supports that connection.

G.3 Run the baselines. The manual-review method should use a written decision guide and record reviewer time. The rules-only method should use the same strict policy rules as the proposed system but no AI risk ranking. The combined method should use the same rules plus the AI output. A centralised variant can be included if it represents a meaningful deployment alternative. All methods must see the same event records and use the same ground-truth labels. The order of runs may be randomised to reduce the effect of changes in hardware load or reviewer fatigue.

G.4 Collect results. Store the decision, reason, policy version, model version, event time, action time, runtime error, resource use, and audit-completeness status for each event. Record when the action did not complete. Do not count an attempted block as a successful block unless the transfer was actually stopped or prevented. Record both the event-level result and any end-to-end failure, such as a missing audit entry. Keep raw personal payloads out of the research log unless the test specifically requires them and an approved protection plan exists.

G.5 Analyse errors and preserve the record. Calculate the metrics defined in Section 15. Report the event count, class distribution, threshold, and uncertainty method. Inspect false negatives and false positives, group them by root cause, and link each failure to a system component. Save scripts and environment settings needed to repeat the result. If a fix is made, rerun the fixed test and retain both the original and corrected results. This helps distinguish the behaviour of the original system from the behaviour of the updated system.

Appendix H. Event Labels and Annotation Guide

H.1 Label definitions. A “permitted” event passes all required identity, purpose, receiver, retention, and other mandatory checks. A “violation” event contains evidence that at least one mandatory rule has failed. An “uncertain” event lacks information required to decide safely, or has conflicting evidence that has not been resolved. “Not assessed” should be reserved for events excluded from the experiment for a documented reason. Unknown should not automatically be treated as permitted, because the absence of evidence is not evidence that all rules have passed.

H.2 Evidence required for labels. A violation label should name the failed rule, the field or record that supports the label, and the expected action. A purpose mismatch should point to the declared purpose and the policy that permits or denies it. A receiver violation should identify the requested receiver and the approved receiver list or rule version. An expired-retention case should include the applicable time limit and the time the event was checked. The test dataset can use pseudonymous IDs for these records. It should not include a real person’s name just to make the label easier to understand.

H.3 Reviewer agreement. Where feasible, two reviewers should independently label a sample of events. The team should report how many labels agreed and describe how disagreements were resolved. It should not hide hard cases by removing them without explanation. If the event cannot be labelled from the available evidence, the correct label may be uncertain. A label guide should include examples and counter-examples so reviewers apply the same rule. Any changes to the guide should have a version and a reason.

H.4 Avoiding leakage and bias. The model should not receive the field that directly reveals the answer if that field would not be available at decision time. For example, a retrospective “violation confirmed” flag belongs in the label, not among the predictive features. Duplicate events from one device should not be split randomly between training and test sets if this would allow the model to memorise them. Results should be checked by relevant device or event category so good average performance does not conceal a group that fails. Sensitive attributes should only be used for evaluation when there is a clear legal and ethical basis.

H.5 Synthetic event handling. Synthetic events are useful for testing exact policy behaviour because the expected outcome can be known in advance. They are less useful for estimating real-world frequency or performance because generated events may not match actual behaviour. The report should state which tests are synthetic and which use recorded data. A synthetic suite can demonstrate that the rule engine follows a condition, but it cannot by itself prove that the AI detector generalises to a live environment.

Appendix I. Policy Rule Examples and Review Checklist

I.1 Example rule record. A rule can contain: rule identifier; short name; source requirement; applicable jurisdiction and role; data category; allowed purpose; approved receiver; required transformation; retention class; denial or fallback action; effective date; owner; approval record; and test cases. The full record should include only fields that are needed for the use case. Each deployed rule should be linked to the version used by the decision engine. This makes a past decision explainable even after the policy changes.

I.2 Example: approved health-data receiver. The rule states that a wearable health record may be sent to an approved care service for the stated care purpose. The gateway validates the device identity and event metadata. The policy engine checks the purpose, destination, data class, and retention rule. If the destination is not approved, the request is denied regardless of the AI score. If information is missing, the event is held for review. The audit record states the event reference, policy version, reason, action, result, and time, without copying the entire health record.

I.3 Example: city traffic counts. The rule states that a traffic-planning service may receive vehicle counts aggregated over an approved time window. The edge node processes images locally when possible and transmits the count rather than retaining image detail that is not required for the task. The system records the aggregation method and policy version. If a downstream request asks for individual tracking outside the approved purpose, the policy engine holds or blocks it. The test should verify both that the permitted aggregate reaches the correct service and that unnecessary raw images are not retained.

I.4 Policy review checklist. Before approval, check that the requirement source is correct, the purpose is specific, the data category is justified, the receiver is identified, the required transformation is clear, retention is defined, and an owner is assigned. Check that normal, violating, uncertain, expiry, and outage cases have tests. Check that the fallback action is safe and that an exception has an expiry and approver. Confirm that the action can be explained in plain words. A rule that cannot be tested or assigned to an owner is not ready for deployment.

I.5 Change control. A change request should state what changed and why. The team should identify affected devices, data categories, model features, tables, audit fields, and test cases. The new rule should be tested against existing events before rollout. If it is deployed in stages, the policy version used by each stage should be recorded. If the change fails, rollback should restore a known-good version. The organisation should be able to answer which rule was active at a specific time and who approved it.

Appendix J. Implementation and Deployment Readiness Plan

J.1 Development stages. Stage 1 defines the event schema, device inventory, policy model, and audit record. Stage 2 implements identity and strict policy checks without AI. Stage 3 adds data classification and safe transformations. Stage 4 adds the risk model and explanation output. Stage 5 connects review queues, policy versioning, and dashboard reporting. Stage 6 runs the full test protocol, including outage and recovery cases. This order gives the team a working baseline before it adds AI complexity. The first deliverable should be a small pipeline that correctly rejects known invalid requests.

J.2 Roles and access. Use separate permissions for adding devices, editing policies, updating models, reading audit logs, and approving exceptions where possible. A developer should not be able to silently approve a rule change that affects their own code. A person reviewing a high-risk event should see only the data needed to make that review. Administrative changes should be logged and checked. Secret keys should be stored outside source code, and test credentials should not be reused in production.

J.3 Operational monitoring. Track failed identity checks, schema errors, stale policies, model timeouts, high-risk alerts, reviewer backlog, failed transformations, incomplete audit records, and retention errors. A high number of alerts may indicate a real problem, but it may also show that the model has drifted or that a policy is too broad. Monitoring should include a review process, not only a dashboard. It should also minimise sensitive information and define who can access each measure.

J.4 Readiness decision. Before live use, confirm that the system has passed rule tests, integration tests, load tests, outage tests, audit reconstruction tests, and a privacy review. Confirm that every policy has an owner, model limitations are documented, retention jobs have been tested, and incident handling has been rehearsed. Set a rollback path and an owner who can stop processing safely. Any unresolved failure should be written down and approved by the responsible organisation before a limited pilot begins.

J.5 Publication checklist. A research report should include the actual code version, dependency list, dataset source and licence, event counts, class balance, label guide, preprocessing, model parameters, policy versions, baselines, metric definitions, repeated-run settings, and all important errors. It should separate measured results from synthetic examples and state whether the data can be shared. A diagram of a proposed system should not be described as proof of a working deployment. These steps help a reader reproduce the experiment and judge its limits.

Appendix K. Risk Model Calibration and Sensitivity Protocol

K.1 Purpose of calibration. The proposed score combines impact, likelihood, and uncertainty. Its weights are design assumptions, so they must be reviewed before operational use. Calibration asks whether the score has a useful relationship with observed outcomes on a labelled validation set. It does not turn the score into a legal verdict. The team should state the event types used, who labelled them, what “high risk” means in that context, and whether labels reflect confirmed events or reviewer judgements. If labels are uncertain, the report should show that uncertainty rather than treating every label as a fact.

K.2 Sensitivity test. Change one input at a time while holding the other two at fixed values. Then change the weights within a documented range and observe how the score ordering changes. The purpose is to check whether a small change in assumptions reverses the decision for many events. If a score changes greatly when a weight moves slightly, that instability should be reported and may require a different model or a rule-based decision. Any plotted values should come from the stated formula and should be labelled as formula-derived until the model has been tested on real events.

K.3 Calibration and threshold choice. If the model outputs probabilities, evaluate calibration using a held-out set and an appropriate calibration plot or score. If it outputs anomaly scores, describe them as ranking values unless there is evidence that they represent probability. Choose thresholds using the cost of false positives and false negatives in the intended use case. A high-impact setting may need a conservative threshold and human review. Thresholds should be fixed before final testing; tuning them on the test set makes the final score over-optimistic.

K.4 Subgroup and device checks. Where appropriate and lawful, report model performance by device family, event class, and relevant operating condition. Do not assume that an overall metric represents every device. A model may perform well for regular sensor signals but poorly when a wearable sends intermittent data. Any evaluation of sensitive user groups must have a clear basis, safeguards, and a minimum necessary data plan. The aim is to detect uneven performance without collecting additional sensitive data without need.

K.5 What to report. Include the formula, weights, input definitions, input scale, handling of missing values, threshold, calibration method, event count, class balance, and results under the main sensitivity settings. Show failures and explain which component caused them. A reader should be able to reproduce the curve from the stated formula or the score from the saved model. These steps make the risk model transparent and help separate a design illustration from evidence of real-world performance.

Appendix L. Failure Mode and Effects Analysis

L.1 Why failure-mode analysis is useful. A privacy workflow can fail through a broken component or a wrong assumption. A failure-mode and effects analysis (FMEA) lists possible failures, their effects, their likely causes, and the controls that lower the risk. It does not provide a probability unless the team has evidence to estimate one. It is a practical way to decide which tests and safeguards should be completed before a pilot.

L.2 Identity and metadata failures. An unknown or revoked device may be accepted if the identity service is stale. A malformed event may be assigned a default low-risk label if schema validation is weak. The effect can be an unauthorised transfer or an incorrect risk score. Controls include credential expiry, revocation checks, schema validation, and tests for missing values. Evidence should show the rejection reason and whether the payload reached any downstream component.

L.3 Policy and model failures. Conflicting rules may produce inconsistent actions. An expired policy may remain active after an outage. A model may time out or return an invalid score. The effect can be an allowed transfer that should have been held, a false alert, or a system that stops useful work. Controls include policy conflict testing, signed versions, expiry rules, schema checks for model responses, and a safe fallback. The design should ensure that a model failure never changes a mandatory deny into an allow.

L.4 Action and audit failures. A service may log that an event was blocked before it confirms that the transfer was stopped. An audit store may fail and lose evidence. A reviewer may override a decision without recording a reason. Controls include confirmation from the action executor, minimal durable logs, queueing protected audit records during defined outages, and required fields for overrides. Tests should intentionally create these failures and confirm that the final state reflects what really happened.

L.5 Prioritising remediation. Teams can rank failure modes using a documented severity and likelihood judgement, but they should not present a simple rank as a proven statistical risk. High-severity failures that could expose sensitive data or bypass a strict rule should be fixed or formally accepted by the responsible owner before deployment. The FMEA should be updated after incidents, major changes, or a failed test. A living record gives the team a reasoned way to prioritise limited engineering time.

Appendix M. Privacy Impact Assessment Checklist

M.1 Describe the processing. State the purpose, the people or groups whose data may be involved, the data categories, the devices that create the data, each destination, the retention period, and the parties that can access it. Describe the full path from sensor to final receiver. Include copies in queues, caches, backups, analytics stores, and model-training material where they are relevant. A simple data-flow map helps the reviewer see where data move and where controls are applied.

M.2 Test necessity and proportionality. For each field, ask whether it is needed for the purpose. If an aggregate, lower precision, or shorter retention period can meet the task, prefer that option unless there is a documented reason not to. Check whether the system could complete the same task without an identifier, continuous location, or raw image. Where a field is required, state why and define who can access it. The assessment should not assume that more detailed data automatically produce a better service.

M.3 Identify risks and controls. Consider unauthorised access, purpose drift, over-retention, re-identification, unsafe sharing, model leakage, insider misuse, unclear decisions, and failure to honour a user's rights where applicable. Map each risk to one or more controls and to evidence that the control works. For example, retention risk should map to a timer and a tested deletion procedure, while receiver risk should map to a list of approved destinations and a blocking test.

M.4 Review residual risk. After safeguards are selected, record what risk remains, who owns it, and whether more action is required before use. A control can reduce risk without removing it. Encryption does not stop an authorised receiver from misusing data, and a model explanation does not prove that a decision is correct. If the residual risk is high, the responsible organisation should decide whether the processing should be changed, delayed, or not performed.

M.5 Keep the assessment current. Review it when the purpose changes, a new device or receiver is added, the model is updated, a data category changes, the processing moves to another region, or an incident reveals a missing control. The assessment should be dated and versioned. A dashboard can identify a change, but the review still needs an owner who can confirm the impact. This checklist supports a formal process; it does not replace a legal assessment required by a specific jurisdiction.

Appendix N. Experimental Result Reporting Template

N.1 Study identification. Each experiment should have a unique run ID, date, code version, configuration file, event-set version, label-guide version, policy version, model version, and hardware description. The run record should also state whether the data are public, synthetic, or private and how the team is allowed to use them. These fields make it possible to connect a result in a paper to the exact run that produced it.

N.2 Report the event set. State the total number of events, the number in each label category, the way labels were assigned, the number of uncertain cases, and any exclusions. Explain whether the split was random, time-based, or by device group. If data were sampled, describe the sampling process. If synthetic events were used, provide the generation rules and keep them separate from any real observations.

N.3 Report each baseline. Give the name and short description of each method, its features, thresholds, and settings. All methods should use the same test events where possible. If one method cannot use a field that another method uses, state that difference. Report model-training time separately from per-event response time so the measurements do not mix setup costs with inference costs.

N.4 Report results and uncertainty. Include the confusion matrix and the relevant precision, recall, F1, false-positive rate, latency, throughput, resource use, and audit-completeness measures. Report sample sizes and repeated-run variation. If a metric is undefined because a class has no examples, state that rather than inserting zero without explanation. Show important failures, not just the best run. Discuss whether the observed difference is large enough to matter in the target use case.

N.5 Interpretation statement. Every final result table should end with a plain-language interpretation. The interpretation should state what improved, what became worse, what the result cannot prove, and what follow-up test is needed. It should not claim legal compliance from detection accuracy alone. A complete result must connect model behaviour, policy correctness, operational performance, privacy protection, and governance evidence. Until a real experiment is run, this appendix is a reporting template and should not be filled with estimated or fabricated numbers.

Appendix O. Operations, Incident Response, and Recovery

O.1 Daily operations. The operations team should check device registration, policy expiry, model service health, failed transformations, unprocessed audit records, and review-queue size. The team should know which alarms require immediate action and which can be reviewed during normal working hours. The monitoring record should not contain more personal data than needed to diagnose the problem. The organisation should document the person on call and the safe action if the owner cannot be reached.

O.2 Incident handling. When a suspected privacy violation occurs, preserve the minimum evidence needed to investigate, identify the affected event IDs and policies, and prevent further exposure where required. Confirm whether data were actually transferred, who could access them, and whether the log is complete. Do not call an AI alert a confirmed incident until it has been reviewed according to the incident process. Notifications and other duties depend on the applicable law and facts; they should be handled by the responsible organisation.

O.3 Recovery. After containment, restore a known-good policy or software version, rotate credentials where needed, check the integrity of the gateway and audit store, and replay queued events only if they remain permitted. A queued event may have become invalid while offline because the policy or receiver status changed. The system should therefore re-check the current rule before it performs a delayed transfer. It should not assume that an event which was once allowed remains allowed indefinitely.

O.4 Lessons learned. After an incident, review the cause, time to detect, time to contain, missing evidence, policy or model gaps, and how the response affected users. Convert important failures into regression tests. Update the policy, threat model, and training guidance when there is evidence that they were incomplete. Assign each corrective action an owner and due date. An incident report should avoid unnecessary personal details and should be stored according to an approved retention policy.

O.5 Resilience test. Run a controlled exercise with a policy outage, a revoked credential, an invalid model response, and an audit-store interruption. Verify that the system applies the approved fallback, records the failure when possible, and restores normal operation without silently permitting blocked traffic. Test recovery from backups and check that restored logs remain linked to their original event IDs. These tests are part of deployment evidence, not optional extras.

REFERENCES

[1] G. Giordano, F. Palomba, and F. Ferrucci, “On the use of artificial intelligence to deal with privacy in IoT systems: A systematic literature review,” Journal of Systems and Software, vol. 193, art. 111475, 2022, doi: 10.1016/j.jss.2022.111475.

[2] A. D. Kounoudes and G. M. Kapitsaki, “A mapping of IoT user-centric privacy preserving approaches to the GDPR,” Internet of Things, vol. 11, art. 100179, 2020, doi: 10.1016/j.iot.2020.100179.

[3] E. Rodríguez, B. Otero, and R. Canal, “A survey of machine and deep learning methods for privacy protection in the Internet of Things,” Sensors, vol. 23, no. 3, art. 1252, 2023, doi: 10.3390/s23031252.

[4] R. Kaur, T. Rodrigues, N. Kadir, and R. Kashef, “A survey on privacy preservation techniques in IoT systems,” Sensors, vol. 25, no. 22, art. 6967, 2025, doi: 10.3390/s25226967.

[5] A. Marengo, “Navigating the nexus of AI and IoT: A comprehensive review of data analytics and privacy paradigms,” Internet of Things, vol. 27, art. 101318, 2024, doi: 10.1016/j.iot.2024.101318.

[6] M. Gheisari, G. Wang, and S. Chen, “An edge computing-enhanced Internet of Things framework for privacy-preserving in smart city,” Computers & Electrical Engineering, vol. 81, art. 106504, 2020, doi: 10.1016/j.compeleceng.2019.106504.

[7] T. M. Mengistu, T. Kim, and J.-W. Lin, “A survey on heterogeneity taxonomy, security and privacy preservation in the integration of IoT, wireless sensor networks and federated learning,” Sensors, vol. 24, no. 3, art. 968, 2024, doi: 10.3390/s24030968.

[8] M. A. Husnoo, A. Anwar, R. K. Chakrabortty, R. R. M. Doss, and M. J. Ryan, “Differential privacy for IoT-enabled critical infrastructure: A comprehensive survey,” IEEE Access, vol. 9, pp. 153276–153304, 2021, doi: 10.1109/ACCESS.2021.3124309.

[9] V. Mothukuri, R. M. Parizi, S. Pouriyeh, Y. Huang, A. Dehghantanha, and G. Srivastava, “A survey on security and privacy of federated learning,” Future Generation Computer Systems, vol. 115, pp. 619–640, 2021, doi: 10.1016/j.future.2020.10.007.

[10] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, “Federated learning for Internet of Things: A comprehensive survey,” IEEE Communications Surveys & Tutorials, vol. 23, no. 3, pp. 1622–1658, 2021, doi: 10.1109/COMST.2021.3075439.

[11] F. Mosaiyebzadeh, S. Pouriyeh, R. M. Parizi, Q. Z. Sheng, M. Han, L. Zhao, G. Sannino, and D. M. Batista, “Privacy-enhancing technologies in federated learning for the Internet of Healthcare Things: A survey,” Electronics, vol. 12, no. 12, art. 2703, 2023, doi: 10.3390/electronics12122703.

[12] A. Coiduras-Sanagustín, E. Manchado-Pérez, and C. García-Hernández, “Understanding perspectives for product design on personal data privacy in Internet of Things (IoT): A systematic literature review,” Heliyon, vol. 10, no. 9, art. e30357, 2024, doi: 10.1016/j.heliyon.2024.e30357.

[13] S. Salim, N. Moustafa, and B. Turnbull, “Privacy preservation of Internet of Things-integrated social networks: A survey and future challenges,” International Journal of Web Information Systems, vol. 21, no. 4, pp. 372–431, 2025, doi: 10.1108/IJWIS-04-2024-0120.

[14] L. Kong, J. Tan, J. Huang, G. Chen, S. Wang, et al., “Edge-computing-driven Internet of Things: A survey,” ACM Computing Surveys, vol. 55, no. 8, art. 174, 2023, doi: 10.1145/3555308.

[15] E. Fazeldehkordi and T.-M. Grønli, “A survey of security architectures for edge computing-based IoT,” IoT, vol. 3, no. 3, pp. 332–365, 2022, doi: 10.3390/iot3030019.

[16] M. Laroui, B. Nour, H. Moungla, M. A. Cherif, H. Afifi, and M. Guizani, “Edge and fog computing for IoT: A survey on current research activities & future directions,” Computer Communications, vol. 180, pp. 210–231, 2021, doi: 10.1016/j.comcom.2021.09.003.

[17] M. H. Shirvani and M. Masdari, “A survey study on trust-based security in Internet of Things: Challenges and issues,” Internet of Things, vol. 21, art. 100640, 2023, doi: 10.1016/j.iot.2022.100640.

[18] A. Sharma, E. S. Pilli, A. P. Mazumdar, and P. Gera, “Towards trustworthy Internet of Things: A survey on trust management applications and schemes,” Computer Communications, vol. 160, pp. 475–493, 2020, doi: 10.1016/j.comcom.2020.06.030.

[19] S. Alam, S. Zardari, S. Noor, S. Ahmed, and H. Mouratidis, “Trust management in Social Internet of Things (SIoT): A survey,” IEEE Access, vol. 10, pp. 108924–108954, 2022, doi: 10.1109/ACCESS.2022.3213699.

[20] Z. Cheng, Y. Liu, C. Wu, Y. Pan, L. Zhao, X. Deng, and C. Zhu, “Decentralized IoT data sharing: A blockchain-based federated learning approach with joint optimizations for efficiency and privacy,” Future Generation Computer Systems, vol. 160, pp. 547–563, 2024, doi: 10.1016/j.future.2024.06.035.

[21] Y. Shen, C. Shepherd, C. M. Ahmed, S. Shen, X. Wu, W. Ke, and S. Yu, “Game-theoretic analytics for privacy preservation in Internet of Things networks: A survey,” Engineering Applications of Artificial Intelligence, vol. 133, pt. E, art. 108449, 2024, doi: 10.1016/j.engappai.2024.108449.

[22] G. Yang, “An overview of current solutions for privacy in the Internet of Things,” Frontiers in Artificial Intelligence, vol. 5, art. 812732, 2022, doi: 10.3389/frai.2022.812732.

[23] D. Li, Y. Zhao, Y. Wang, D. An, and Q. Yang, “The privacy-preserving auction mechanisms in IoT-based trading market: A survey,” Internet of Things, vol. 26, art. 101178, 2024, doi: 10.1016/j.iot.2024.101178.

[24] K. Park and Y. Park, “MIoT-CDPS: Complete decentralized privacy-preserving scheme for medical Internet of Things,” Internet of Things, vol. 27, art. 101250, 2024, doi: 10.1016/j.iot.2024.101250.

[25] I. Khan, Y. Majib, R. Ullah, and O. Rana, “Blockchain applications for Internet of Things—A survey,” Internet of Things, vol. 27, art. 101254, 2024, doi: 10.1016/j.iot.2024.101254.

[26] M. Aledhari, R. Razzak, R. M. Parizi, and F. Saeed, “Federated learning: A survey on enabling technologies, protocols, and applications,” IEEE Access, vol. 8, pp. 140699–140725, 2020, doi: 10.1109/ACCESS.2020.3013541.

[27] W. Y. B. Lim et al., “Federated learning in mobile edge networks: A comprehensive survey,” IEEE Communications Surveys & Tutorials, vol. 22, no. 3, pp. 2031–2063, 2020, doi: 10.1109/COMST.2020.2986024.

[28] M. Aggarwal, V. Khullar, S. Rani, T. A. Prola, S. B. Bhattacharjee, S. M. Shawon, and N. Goyal, “Federated learning on Internet of Things: Extensive and systematic review,” Computers, Materials & Continua, vol. 79, no. 2, pp. 1795–1834, 2024, doi: 10.32604/cmc.2024.049846.

[29] S. K. Jain and N. Kesswani, “A noise-based privacy preserving model for Internet of Things,” Complex & Intelligent Systems, vol. 9, pp. 3655–3679, 2023, doi: 10.1007/s40747-021-00489-5.

[30] O. G. Darley, A. A. Adenowo, and A. I. O. Yussuff, “Machine learning intrusion detection as a solution to security and privacy issues in IoT: A systematic review,” FUOYE Journal of Engineering and Technology, vol. 7, no. 2, pp. 148–156, 2022, doi: 10.46792/fuoyejet.v7i2.802.

[31] M. A. S. Amin, S. Kim, M. A. S. A. Rishat, Z. Tang, and H. Ahn, “A systematic literature review of privacy information disclosure in AI-integrated Internet of Things technologies,” Sustainability, vol. 17, no. 1, art. 8, 2025, doi: 10.3390/su17010008.

[32] M. Aqeel, F. Ali, M. W. Iqbal, T. A. Rana, M. Arif, and M. R. Auwul, “A review of security and privacy concerns in the Internet of Things (IoT),” Journal of Sensors, vol. 2022, art. 5724168, 2022, doi: 10.1155/2022/5724168.

[33] R. Roman, P. Najera, and J. Lopez, “On the features and challenges of security and privacy in distributed Internet of Things,” Computer Networks, vol. 57, no. 10, pp. 2266–2279, 2013, doi: 10.1016/j.comnet.2013.04.018.

[34] S. Sicari, A. Rizzardi, L. A. Grieco, and A. Coen-Porisini, “Security, privacy and trust in Internet of Things: The road ahead,” Computer Networks, vol. 76, pp. 146–164, 2015, doi: 10.1016/j.comnet.2014.11.008.

[35] K. Ziegeldorf, O. Garcia Morchon, and K. Wehrle, “Privacy in the Internet of Things: Threats and challenges,” Security and Communication Networks, vol. 7, no. 12, pp. 2728–2742, 2014, doi: 10.1002/sec.795.

[36] D. Miorandi, S. Sicari, F. De Pellegrini, and I. Chlamtac, “Internet of things: Vision, applications and research challenges,” Ad Hoc Networks, vol. 10, no. 7, pp. 1497–1516, 2012, doi: 10.1016/j.adhoc.2012.02.016.

[37] W. Shi et al., “Edge computing: Vision and challenges,” IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637–646, 2016, doi: 10.1109/JIOT.2016.2579198.

[38] M. Satyanarayanan, “The emergence of edge computing,” Computer, vol. 50, no. 1, pp. 30–39, 2017, doi: 10.1109/MC.2017.9.

[39] M. Whitmore, A. Agarwal, and L. Da Xu, “The Internet of Things—A survey of topics and trends,” Information Systems Frontiers, vol. 17, pp. 261–274, 2015, doi: 10.1007/s10796-014-9489-2.

[40] L. Atzori, A. Iera, and G. Morabito, “The Internet of Things: A survey,” Computer Networks, vol. 54, no. 15, pp. 2787–2805, 2010, doi: 10.1016/j.comnet.2010.05.010.

[41] K. Boeckl and N. Lefkovitz, The NIST Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management, Version 1.0, National Institute of Standards and Technology, 2020, doi: 10.6028/NIST.CSWP.01162020.

[42] E. Tabassi, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, National Institute of Standards and Technology, 2023, doi: 10.6028/NIST.AI.100-1.

[43] M. J. Fagan, K. N. Megas, K. Scarfone, and M. Smith, IoT Device Cybersecurity Capability Core Baseline, NISTIR 8259A, National Institute of Standards and Technology, 2020, doi: 10.6028/NIST.IR.8259A.

[44] European Parliament and Council, “Regulation (EU) 2016/679 (General Data Protection Regulation),” Official Journal of the European Union, L 119, pp. 1–88, 2016. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj.

[45] Government of India, Digital Personal Data Protection Act, 2023, Act No. 22 of 2023; Ministry of Electronics and Information Technology, Digital Personal Data Protection Rules, 2025, G.S.R. 846(E), 13 Nov. 2025. Official source: https://www.meity.gov.in/data-protection-framework.

[46] U.S. Department of Health and Human Services, “The HIPAA Privacy Rule,” official guidance. Available: https://www.hhs.gov/hipaa/for-professionals/privacy/index.html.

[47] C. Dwork and A. Roth, The Algorithmic Foundations of Differential Privacy. Hanover, MA, USA: Now Publishers, 2014, doi: 10.1561/0400000042.

[48] K. Bonawitz et al., “Practical secure aggregation for privacy-preserving machine learning,” in Proc. ACM SIGSAC Conference on Computer and Communications Security, pp. 1175–1191, 2017, doi: 10.1145/3133956.3133982.

[49] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in Proc. IEEE Symposium on Security and Privacy, pp. 3–18, 2017, doi: 10.1109/SP.2017.41.

[50] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should I trust you?: Explaining the predictions of any classifier,” in Proc. ACM SIGKDD, pp. 1135–1144, 2016, doi: 10.1145/2939672.2939778.